5DESI MINI-PROJECT · ARCHITECTURE DOSSIER

MedX Technical Solution Design

The technical design of MedX, a platform for teleconsultations, lifelong patient records and prescription tracking. Every choice follows MedX's own priorities: health data is protected at rest and in transit, records cannot be silently rewritten, and authorised professionals reach critical information in seconds during an emergency.

1 **High-level architecture**

2 **Detailed component designs**

3 **Data model & storage strategy**

4 **Scalability & performance**

5 **Security & compliance**

6 **Presentation (slides & script)**

STYLE

8 microservices in security zones, event-driven, zero trust

EMERGENCY ACCESS

Break-glass, 99.99 %, critical summary p95 < 300 ms

ENCRYPTION

TLS 1.3 / mTLS everywhere, per-patient keys in HSM-backed KMS

SCALE TARGET

10 M patients, 200 k consultations/day, 10 k concurrent calls

Contents

0 Context & targets

1 High-level architecture

2 Component designs

- 2.1 User Management
- 2.2 Scheduling
- 2.3 Consultation
- 2.4 Patient Records & Emergency Access
- 2.5 Prescription
- 2.6 Pharmacy Interface
- 2.7 Notification
- 2.8 Billing & Payment
- 2.9 Platform components

3 Data model & storage

4 Scalability & performance

5 Security & compliance

6 Decisions & trade-offs

7 Traceability matrix

8 Roadmap & risks

0. Context, Assumptions & Design Targets

Part of the **MedX Technical Solution** design package. Start here: this document fixes the scope, the assumptions and the numbers that every other document relies on.

#	Document	Deliverable
0	Context, assumptions & design targets (this document)	Foundation
1	High-level architecture	Deliverable 1
2	Detailed component designs	Deliverable 2
3	Data model & storage strategy	Deliverable 3
4	Scalability & performance	Deliverable 4
5	Security & compliance	Deliverable 5
6	Architecture decisions & trade-offs	Priorities alignment
7	Requirements traceability matrix	Completeness
8	Roadmap & risks	Priorities alignment
—	Presentation	Deliverable 6

0.1 MedX in one paragraph

MedX is a healthcare platform with three pillars: **telehealth consultations** (book, meet a doctor by video, chat and share documents), **patient records management** (a secure, lifelong medical record available to the patient and to the professionals who treat them) and **prescription tracking** (doctors issue e-prescriptions that pharmacies receive, prepare and dispense, while the patient follows every step). Because MedX handles health data — the most sensitive category of personal data — the architecture is driven first by **data protection, record integrity and emergency availability**, then by the care workflows, and last by business-support features.

0.2 Actors

Actor	Description	Main channels	Typical actions
Patient	Adult user; can also act as **proxy** (parent of a minor, caregiver with consent)	Mobile app (iOS/Android), web app	Book, attend consultations, read records, follow prescriptions, pay
Doctor / practitioner	Verified health professional (license checked against the national registry)	Web portal (desktop), mobile app	Manage availability, run consultations, write notes, prescribe
Emergency clinician	Practitioner holding the `emergency_access` entitlement (ER, on-call, EMS)	Web portal, mobile app	Break-glass access to a patient's critical information
Pharmacy staff	Pharmacist, technician, pharmacy administrator	Pharmacy portal, or their pharmacy management system (PMS) through the partner API	Receive prescriptions, update status, answer patient inquiries
MedX operations	Support agent, billing agent, platform administrator, **privacy officer / DPO**, auditor	Back-office console (VPN + SSO + hardware MFA)	Support without clinical data, audits, break-glass reviews

Actor	Description	Main channels	Typical actions
External systems	Calendars, payment provider, SMS/e-mail/push providers, pharmacy systems, drug database, national health services, identity federation	APIs, webhooks	Integration

0.3 Scope

In scope — the eight business components of Appendix 1 plus the platform capabilities they need:

1. User Management (identity, authentication, authorization, preferences)
2. Teleconsultation Scheduling (booking, waitlists, urgent requests, external calendars)
3. Consultation (virtual rooms, video/audio, chat, file transfer)
4. Patient Records Management (medical history, notes, documents, **emergency access**)
5. Prescription Management
6. Pharmacy Interface
7. Notification
8. Billing & Payment

Platform capabilities: API & real-time gateways, policy engine, event bus, secure document store, audit trail, key management, observability, de-identified data platform.

Out of scope for v1 (kept compatible with the design): in-clinic practice management, medical imaging (PACS/DICOM viewers — only document uploads and links), insurance claim processing (third-party payer), AI-assisted diagnosis, connected medical devices. They are listed in the roadmap.

0.4 MedX priorities and how they drive the design

The brief is explicit: *protecting data in transit and at rest is paramount* and *health professionals must access patient information promptly in emergencies*. Each High-priority requirement is therefore turned into a **non-negotiable architecture principle**, not just a feature.

Priority	Requirement	Turned into…
High	H1 Data privacy (encryption at rest & in transit)	TLS 1.3 / mTLS everywhere, AES-256 at rest everywhere, **per-patient envelope encryption** of clinical data (principle P1)
High	H2 Patient records management	Dedicated **FHIR-based** Records service, versioned & audited (P3, P4)
High	H3 Emergency access	**Break-glass** capability on an isolated, Tier-0 read path with post-hoc review (P2)
High	H4 Authentication & authorization	Proven IdP (OIDC, MFA) + **RBAC + ABAC** policy engine, zero trust between services (P1)
High	H5 Data integrity & secure file storage	Append-only history, hash chaining, checksums, immutable backups, scanned & encrypted document store (P3)
Medium	M1–M5 Scheduling, consultations, prescriptions, masking, calendars	Core care workflows built on the High-priority foundation
Low	L1–L6 Pharmacy, preferences, notification tracking, anonymization, billing, refunds	Designed now, delivered last, **bought rather than built** when possible (PSP, messaging providers)

0.5 Architecture principles

ID	Principle	What it means in practice
P1	**Privacy & security by design and by default**	Encrypt everything, least privilege, deny by default, data minimisation (no health data in SMS, e-mails, calendars, logs or events).
P2	**Patient safety first**	The emergency access path must keep working when everything else is degraded: isolated deployment, minimal dependencies, highest availability tier.
P3	**Integrity over convenience**	Clinical data is never overwritten or physically deleted during its retention period: every change is a new, attributed, tamper-evident version.
P4	**Standards over custom**	HL7 FHIR R4 (+ International Patient Summary), OpenID Connect / OAuth 2.1, WebRTC, OpenAPI 3.1, CloudEvents. Easier interoperability, audits and hiring.
P5	**Loose coupling, strong ownership**	One service = one bounded context = one private datastore. Services integrate through versioned APIs and domain events.
P6	**Buy commodity, build differentiators**	Reuse an IdP, a payment provider, messaging providers, an open-source SFU. Build what makes MedX unique: records, emergency access, care workflows.
P7	**Scale out and automate**	Stateless services, horizontal autoscaling, infrastructure as code, GitOps, no manual changes in production.
P8	**Observable and auditable by default**	Every request traced, every access to health data audited in an immutable trail.

0.6 Assumptions

ID	Assumption	Impact on the design
A1	MedX launches in the **EU (France first)**; the US is a possible later market.	GDPR + French **HDS** certification (health-data hosting) are mandatory; the design is **HIPAA-ready** (BAAs, audit, encryption) for a later US cell.
A2	Hosting on a **public cloud with HDS certification** (reference implementation: AWS Paris region, DR in Frankfurt; Azure, Google Cloud or OVHcloud are equivalent options).	Managed services are used only when they are inside the provider's certified scope. All health data stays in the EEA.
A3	Legal roles (controller / processor) between MedX and practitioners are defined by the DPO.	The design supports both: per-practitioner access control, consent registry, full audit.
A4	Patients use a recent smartphone or browser supporting WebRTC; some have poor connectivity.	Adaptive bitrate, audio-only fallback, TURN relays, mobile-first UX.
A5	Pharmacies use heterogeneous management systems, some without APIs.	Web portal for everyone + adapter-based partner API for connected systems.
A6	Payments are processed by a PCI DSS Level 1 **payment service provider** (PSP).	MedX never stores card data → minimal PCI scope (SAQ A).
A7	Doctors use Google Workspace, Microsoft 365 or CalDAV calendars.	Connectors for the three families, free/busy only.
A8	Integration with national services (e-prescription, national health identifier, shared medical record) comes after v1.	Records and prescriptions are modelled in FHIR to make these integrations adapters, not redesigns.
A9	Growth targets below are planning figures, to be re-validated each quarter.	Capacity is designed with ×2 headroom and a clear scaling path beyond.

0.7 Capacity targets

Figures used for sizing in the scalability document.

Metric	Year 1 (launch)	Year 3 (target)	Design capacity (Year 3 × 2)
Registered patients	1 M	10 M	—
Healthcare professionals	5 k	50 k	—
Connected pharmacies	2 k	15 k	—
Daily active users (peak day)	100 k	1 M	2 M
Teleconsultations per day	20 k	200 k	400 k
Concurrent video sessions (peak hour)	1 k	10 k	20 k
Prescriptions per day	12 k	120 k	240 k
Notifications per day	200 k	2 M	4 M
API requests per second (peak)	500	5 k	10 k
Concurrent WebSocket connections (peak)	20 k	200 k	400 k
Medical documents (cumulative)	10 TB	150 TB	—
Structured clinical data (FHIR store)	0.5 TB	10 TB	—
Audit events per day	2 M	20 M	40 M

How the Year-3 figures were derived: 50 k professionals × 4 teleconsultations/day = 200 k consultations; 12 % of them in the peak hour × 20 min average session (waiting room included) ≈ 8 k concurrent sessions, rounded to 10 k; 60 % of consultations end with a prescription; ~6 notifications per consultation plus pharmacy and messaging events ≈ 2 M/day; ~75 M API calls/day with a ×5 peak factor ≈ 4.3 k req/s, rounded to 5 k.

0.8 Service levels and criticality tiers

Services are grouped into **criticality tiers**. The tier sets the availability objective, the recovery objectives and the order in which capacity is protected when the platform is under stress.

Tier	Scope	Availability	RPO	RTO (region loss)	RTO (zone loss)
Tier 0 — life-critical	Emergency access, authentication/token validation, records read path	**99.99 %**	0 in-region (synchronous multi-AZ), ≤ 1 min cross-region	**≤ 15 min**	automatic, < 1 min
Tier 1 — care delivery	Records write, consultation, scheduling, prescription, chat	99.95 %	0 in-region, ≤ 1 min cross-region	≤ 1 h	automatic, < 2 min
Tier 2 — support	Pharmacy interface, notification, billing, analytics	99.9 %	≤ 5 min	≤ 4 h	automatic, < 5 min

Capability	Tier	Latency objective (p95)
Emergency access — critical summary	0	**< 300 ms** API, < 1 s on screen
Login / token issuance	0	< 300 ms (excluding user MFA interaction)
Patient record read / write	1	< 300 ms / < 500 ms
Video consultation	1	join < 3 s; < 100 ms RTT to the nearest media node; mouth-to-ear < 400 ms
Chat message delivery	1	< 500 ms
Prescription signed → visible in pharmacy inbox	1	< 5 s
Slot search / booking	1	< 300 ms / < 800 ms (excluding 3-D Secure)

Capability	Tier	Latency objective (p95)
Notification dispatch (critical / standard)	2	< 10 s / < 60 s
Inventory availability check	2	< 1 s cached, < 3 s live
Payment authorisation	2	< 2 s (excluding 3-D Secure)

0.9 Glossary

Term	Meaning
PHI	Protected Health Information — any health data linked to an identifiable person ("données de santé" under GDPR art. 9).
HDS	*Hébergeur de Données de Santé* — French certification mandatory for hosting health data.
FHIR	HL7 Fast Healthcare Interoperability Resources — the standard data model and REST API for health data.
IPS	International Patient Summary (HL7 FHIR IG) — minimal clinical summary designed for unplanned care; basis of the MedX emergency summary.
Break-glass	Controlled override that grants immediate access in an emergency, with justification and mandatory review.
RBAC / ABAC	Role-Based / Attribute-Based Access Control.
PDP / PEP	Policy Decision Point (evaluates rules) / Policy Enforcement Point (enforces the decision in each service).
DEK / KEK	Data Encryption Key / Key Encryption Key (envelope encryption).
HSM / KMS	Hardware Security Module / Key Management Service.
SFU / TURN	Selective Forwarding Unit (video router) / relay server used when a direct media path is impossible.
WORM	Write Once, Read Many — storage that cannot be altered or deleted before its retention expires.
RPO / RTO	Recovery Point Objective (max data loss) / Recovery Time Objective (max downtime).
SLO	Service Level Objective.
INS / RPPS / FINESS	French national patient health identifier / practitioner registry number / health establishment (pharmacy) registry number.
PSP	Payment Service Provider (e.g., Stripe, Adyen).
Outbox pattern	Events are written in the same database transaction as the business change, then published — no lost or phantom events.

1. High-Level Architecture

Deliverable 1. Visual representation of the overall structure of MedX and the logic behind it. Assumptions and targets come from 0. Context.

1.1 Architecture at a glance

MedX — High-Level Architecture

Container view · microservices by bounded context · event-driven · zero trust · every link encrypted



MedX high-level architecture

Vector source: `diagrams/medx-high-level-architecture.svg`*. Circled numbers ①–⑧ are the components of Appendix 1; H / M / L badges are MedX priorities (Appendix 2).*

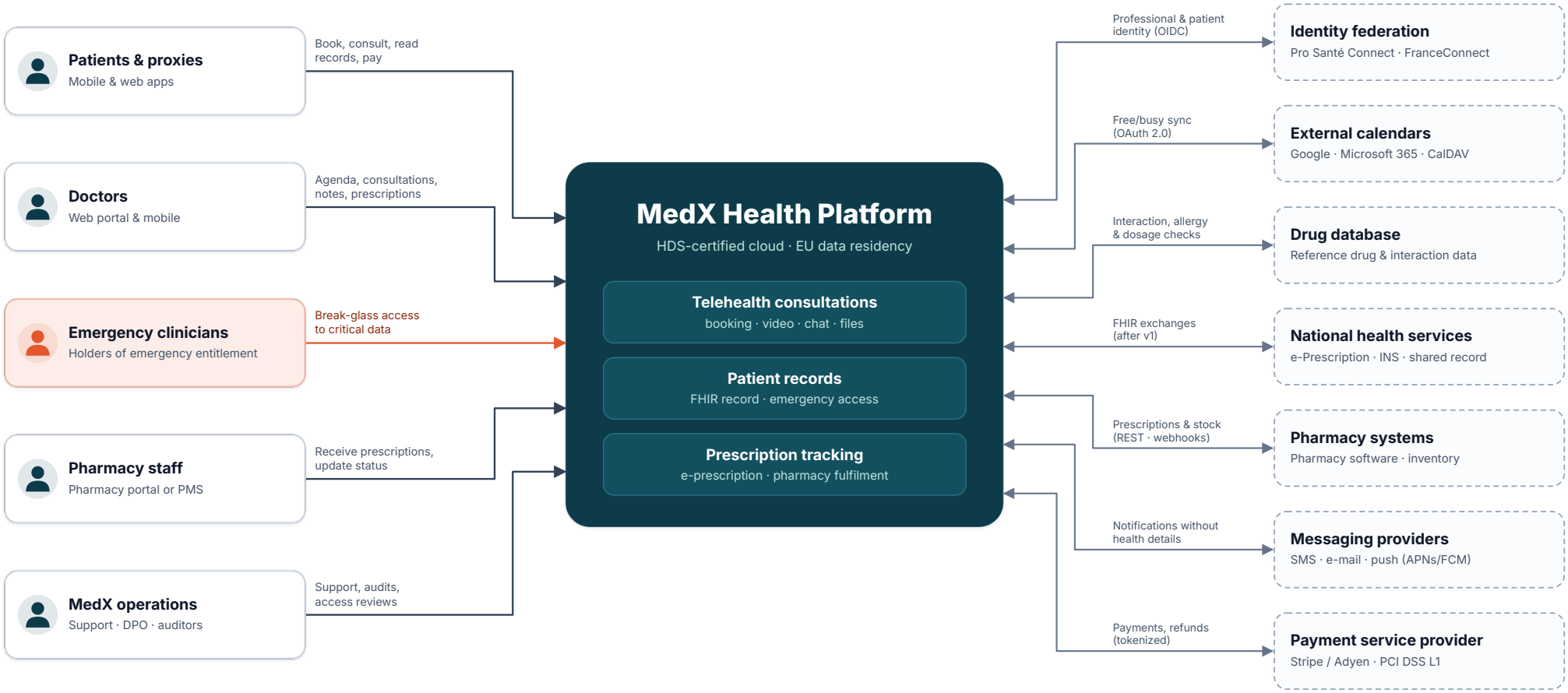
Seven ideas to remember

1. **Eight business microservices, one per component of Appendix 1**, grouped into **security zones**. The *clinical zone* concentrates health data (PHI) and receives the strictest controls.
2. **One hardened front door.** Only the edge (CDN/WAF, API gateway, real-time gateway, media plane) is reachable from the internet; every service and datastore lives in private subnets.
3. **Identity is centralised, enforcement is local.** A proven identity provider issues tokens; a policy engine (OPA) runs next to each service so every request is authorised in about a millisecond without a network hop.
4. **Events first, synchronous calls only when a user is waiting.** State changes are published as domain events on Kafka through a transactional outbox; this keeps services decoupled and failures contained.
5. **Each service owns its data** (database-per-service). PostgreSQL is the default; specialised stores are used only where a clear need exists (Redis, Cassandra, object storage, OpenSearch).
6. **Security-critical capabilities are built once, as platform services**: audit trail, secure document store, key management, observability & SIEM, de-identified data platform.
7. **Emergency access is a Tier-0 path**: an isolated deployment that reads pre-computed summaries from replicas in two regions and depends on nothing that is not strictly necessary.

1.2 System context (C4 level 1)

MedX — System Context

C4 level 1 · who uses MedX, what for, and which external systems it relies on



Solid boxes: people · dashed boxes: external systems outside the MedX trust boundary · every connection is encrypted (TLS 1.3 / mTLS)

MedX system context

MedX serves five groups of people and relies on seven families of external systems. Every external system sits **outside the trust boundary**: it only receives the data it needs (for example, calendars get free/busy slots and a generic "MedX teleconsultation" title — never a patient name or a specialty), through adapters owned by the consuming service.

1.3 Architectural style and rationale

Option	Strengths	Weaknesses for MedX	Verdict
Monolith	Simple to build and deploy	One blast radius for PHI and payments; video, records and billing cannot scale independently; one bug can take down emergency access	Rejected
Modular monolith	Clear modules, simple operations	Still one runtime and one database: no isolation of the Tier-0 path, compliance scopes (PHI, PCI) cannot be separated	Good start-up option, rejected at MedX's target scale
Microservices by bounded context (8 business services + platform)	Isolation of sensitive data and of failures, independent scaling (video ≠ records ≠ billing), independent release cadence, clear ownership	Operational complexity, distributed data consistency	**Chosen** — complexity mitigated below
Serverless functions everywhere	No servers to operate, pay-per-use	Cold starts on the emergency path, hard long-lived WebSocket/WebRTC sessions, vendor lock-in, harder HDS scoping	Used only for small event handlers if useful

Why the chosen style fits the priorities

- *Data privacy (H1) and compliance*: PHI is confined to the clinical zone; billing sits in a separate payment zone so its PCI scope stays minimal; the notification service never stores health data.
- *Emergency access (H3)*: the break-glass path is deployed separately (bulkhead) and cannot be starved by a spike in bookings or video traffic.
- *Scalability*: the media plane scales on bandwidth, the records service on read throughput, the notification service on queue depth — each with its own autoscaling metric.

How the complexity is kept under control

- The number of services is deliberately **limited to the business components** (no nano-services).
- A **golden-path service template** gives every service the same skeleton: OpenAPI contract, authorization hook, outbox, health checks, OpenTelemetry, security headers, CI pipeline.
- A **service mesh** provides mTLS, retries, timeouts and circuit breaking uniformly.
- A **platform team** owns Kubernetes, the event bus, observability, and the security building blocks.

1.4 Layers and components

Layer	Components	Responsibility
Channels	Patient mobile app (iOS/Android), patient web app, doctor web portal, pharmacy portal, back-office console	User interfaces. No business logic, minimal local storage of health data (encrypted, wiped on logout).
Edge	CDN + WAF + DDoS protection · **API Gateway** · **Real-time Gateway** (WebSocket) · **Media plane** (WebRTC SFU + TURN/STUN)	Single entry point: TLS termination, token validation, rate limiting, request validation, routing, webhook ingress, real-time fan-out, media relay.

Layer	Components	Responsibility
Identity & access	① **User Management** (+ Keycloak IdP) · **Policy Decision Point** (OPA)	Who you are (authentication, MFA, federation) and what you may do (RBAC + ABAC, consent, care relationships, masking obligations).
Clinical zone	② Scheduling · ③ Consultation · ④ Patient Records + **Emergency Access** · ⑤ Prescription · ⑥ Pharmacy Interface	Care workflows and health data.
Engagement zone	⑦ Notification	Multi-channel messages without health details, preferences, delivery/read tracking.
Payment zone	⑧ Billing & Payment	Fees, pre-authorisation and capture, invoices, refunds, ledger — card data stays at the PSP.
Event backbone	Apache Kafka + schema registry + outbox relay (CDC)	Asynchronous integration, audit feed, projections.
Platform services	Audit trail · Secure document store · Keys & secrets (KMS/HSM, Vault) · Observability & SIEM · De-identified data platform	Shared, security-critical capabilities.
Data layer	PostgreSQL (one database per service) · Redis · Cassandra · Object storage · OpenSearch · Backup vault	Persistence — see 3. Data model & storage.

Detailed responsibilities, APIs, events and flows of each component are in 2. Detailed component designs.

1.5 Security zones and trust boundaries

Zone	Contains	Data allowed	Main controls
Edge (public)	CDN/WAF, gateways, media nodes	Encrypted traffic in transit only; no datastore	WAF rules, DDoS protection, TLS 1.3, token validation, rate limits
Identity & access	User Management, IdP, PDP	Identity, credentials (hashed), roles, preferences	MFA, credential hashing, admin JIT access, separate database
Clinical (PHI)	Scheduling, Consultation, Records, Emergency Access, Prescription, Pharmacy Interface, Document Store	Health data, per-patient encrypted	Dedicated node pool, strict network policies, ABAC on every request, field-level encryption, full audit
Engagement	Notification	Contact details, preferences, message metadata — **no PHI**	Templates reviewed for PHI, provider DPAs, opt-in/opt-out
Payment	Billing & Payment	Amounts, invoices, PSP tokens — **no card numbers, no diagnosis**	Separate namespace & network policy, PSP hosted fields, webhook signatures
Platform	Audit, KMS/Vault, observability, data platform	Audit records (restricted), keys (never exported), logs scrubbed of PHI, de-identified datasets	Separate security account for audit/logs, WORM storage, separation of duties
Non-production	Dev, staging, demo	**Synthetic or anonymised data only**	Separate accounts, no network path to production

Flow rules across boundaries

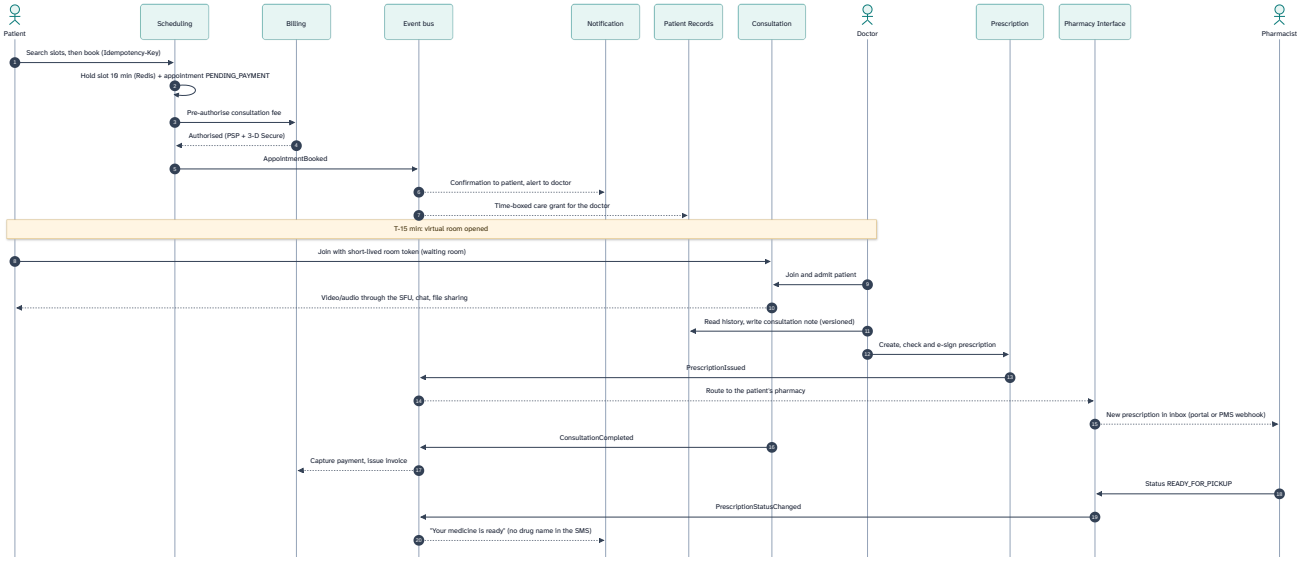
- Health data leaves the clinical zone only (a) towards an authorised user through the gateway, (b) towards a pharmacy through the Pharmacy Interface, minimised to what dispensing requires, or (c) towards the de-identified data platform **after** anonymisation.
- Events on the bus carry **identifiers and status, not clinical content** ("claim-check" pattern: consumers fetch details from the owning service under their own authorisation).
- Nothing in the clinical zone accepts inbound connections from the internet except through the gateways.

1.6 Communication patterns

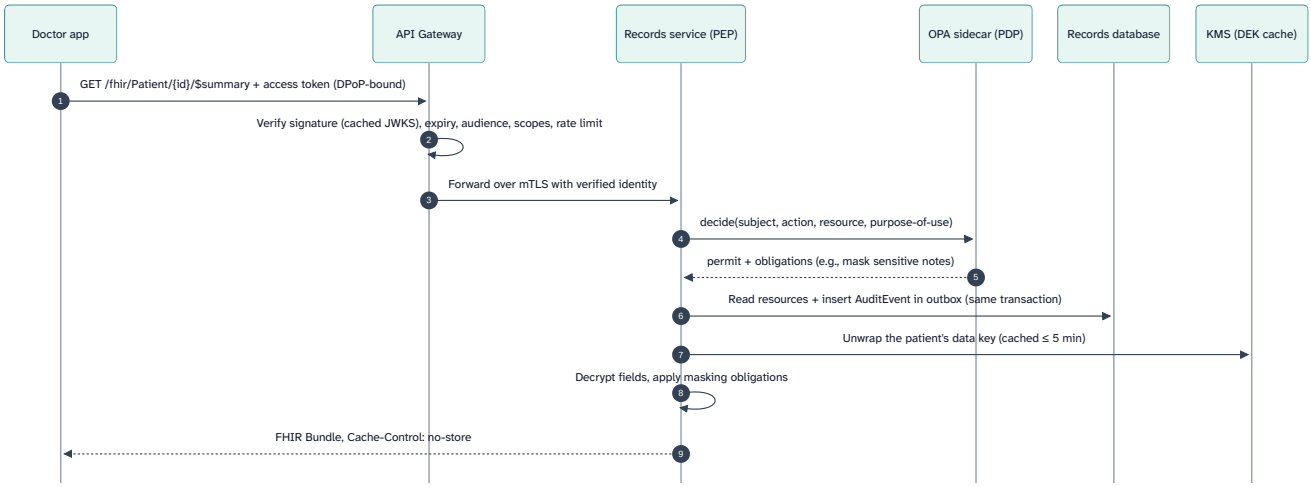
Pattern	Used for	Protocol & format	Rules
Synchronous request/response	User-facing reads and commands; checks that need an immediate answer (e.g., allergy check while prescribing)	HTTPS/REST, JSON, OpenAPI 3.1; **FHIR R4 REST** for clinical data; internal calls over mTLS	Timeouts on every call, retries only for idempotent operations (with jitter), circuit breakers, at most **two synchronous hops** per user request, `Idempotency-Key` on every create
Asynchronous domain events	Propagating state changes (booking, prescription, payment…), notifications, projections, audit	Kafka topics, CloudEvents envelope, versioned schemas (schema registry, backward compatible)	Published through the **transactional outbox** (no dual writes); at-least-once delivery + **idempotent consumers**; ordering by aggregate key (e.g., `appointmentId`)
Real-time push	Chat, live statuses (prescription, queue position), in-app notifications	WebSocket (WSS) via the Real-time Gateway, backed by pub/sub	Authenticated at connection time, per-user channels, re-authorised on token refresh
Media	Video and audio	WebRTC: DTLS-SRTP (+ E2EE with SFrame when supported), ICE/STUN/TURN	Media never transits through business services; rooms and tokens controlled by the Consultation service
External integration	Calendars, PSP, messaging, pharmacy systems, drug database, national services	REST/FHIR over TLS or mTLS, OAuth 2.0; inbound webhooks through the API gateway	One **adapter (anti-corruption layer)** per external system, owned by the consuming service; signed webhooks; retries with backoff; dead-letter queues

1.7 Key end-to-end flows

1.7.1 The care journey: booking → consultation → prescription → pharmacy



1.7.2 How every request on health data is authorised



Two properties matter here: **no access without an audit record** (the audit event is written in the same database transaction as the read or write), and **no network hop for the authorisation decision** (OPA runs as a sidecar with policies distributed as signed bundles).

1.7.3 Emergency access (break-glass)

Emergency Access (Break-Glass)

High-priority requirement H3 · immediate access to critical data for authorized professionals, with full accountability afterwards

1 Identify the patient

Exact-match search only:
INS / MedX ID, or name + birth date + sex
Minimal demographics to confirm identity
No browsing, no wildcards

2 Declare emergency

Reason code (unconscious, life-threatening, unable to consent…)
Free-text justification
Accountability statement
“I will be audited”

3 Step-up MFA

Passkey or smart card (e-CPS) in the last 15 min
Policy engine checks the emergency_access entitlement and the patient's opt-out

4 Time-boxed grant

4 hours · read-only
This patient only
Grant + audit event written in the same transaction
Renewable with a new reason

5 Summary in < 1 s

International Patient Summary: allergies, active problems, medications, blood group, implants, advance directives, contacts
Full record: same grant

Clinician: ≈ 30 seconds from search to data (human steps) · System: p95 < 300 ms per API call · Availability 99.99 %

GUARDRAILS — ACCOUNTABILITY AFTER THE FACT (access is never blocked, always answered for)

Immutable audit

AuditEvent, purpose = emergency treatment, kept in the WORM trail

Real-time alert

Privacy office alerted through the SIEM the moment access starts

Patient informed

In-app + e-mail notice within 24 h: who, when, what was viewed

Mandatory review

Privacy officer + medical lead review every case within 72 h

Misuse detection

Behaviour analytics flag outliers; sanctions and account suspension

RESILIENCE — WHY THE TIER-0 PATH KEEPS WORKING WHEN OTHER SERVICES ARE DEGRADED

Isolated deployment

bulkhead with reserved capacity

Pre-computed summary

projection, no joins at read time

Multi-AZ replicas

+ DR region serving reads (active)

Local token validation

cached IdP signing keys

Local policy decisions

OPA sidecar, cached bundles

Audit via outbox

no dependency on the audit pipeline

Who can break the glass?

Verified practitioners holding the emergency_access entitlement (emergency medicine, on-call, EMS), granted by the MedX medical lead and reviewed quarterly. Patients may opt out where the law allows it.

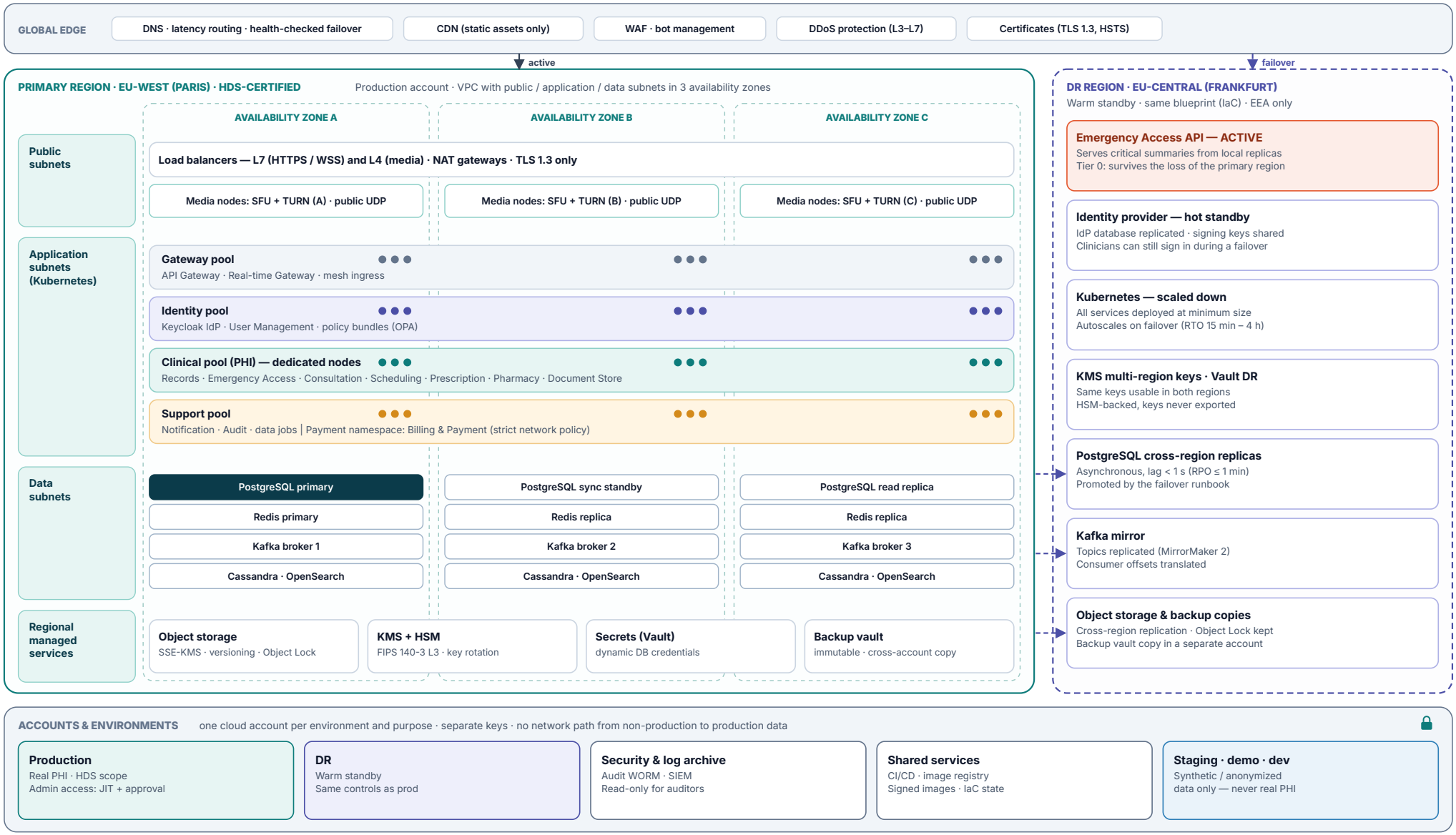
Emergency access flow

The detailed design (entitlement, grant model, critical summary content, review process) is in 2.4 Patient Records & Emergency Access.

1.8 Deployment view

MedX — Deployment & Security Zones

Reference implementation on an HDS-certified public cloud · multi-AZ primary region · warm-standby DR region · isolated accounts



MedX deployment view

- **Primary region** in France (HDS-certified), spread over **three availability zones**. Every tier is redundant across zones: load balancers, Kubernetes node pools, database primary + **synchronous standby** + read replica, Redis replicas, three Kafka brokers, Cassandra and OpenSearch nodes.
- **Kubernetes node pools** follow the security zones: gateway, identity, **clinical (dedicated nodes)** and support pools, plus a **payment namespace** isolated by network policy. Media nodes run in public subnets because WebRTC needs public UDP ports; they hold no data.
- **DR region** (second EU region, EEA only) runs as a **warm standby** built from the same infrastructure-as-code: database replicas (lag < 1 s), Kafka mirror, replicated object storage, multi-region KMS keys. The **Emergency Access API and the identity provider are active in both regions**, so the Tier-0 path survives the loss of the primary region.
- **One cloud account per environment and purpose**: production, DR, security & log archive (audit WORM, SIEM), shared services (CI/CD, signed images), non-production (synthetic data only).

1.9 Technology stack (reference implementation)

The design is cloud-agnostic; the choices below are the reference implementation used for sizing and costing.

Concern	Reference choice	Why	Alternatives
Hosting	HDS-certified public cloud — primary Paris, DR Frankfurt	Certified scope, managed services, EU data residency	Azure France, Google Cloud Paris, OVHcloud
Compute	Managed Kubernetes + Istio service mesh	Standard autoscaling, uniform mTLS, traffic policies	Linkerd; Nomad
Business services	Java/Kotlin on a current LTS JDK, Spring Boot	Mature ecosystem, strong healthcare libraries (HAPI FHIR), large talent pool	Go, .NET
Clinical data	**HAPI FHIR** server on PostgreSQL	Open-source, FHIR R4 validation, versioning, search	Managed FHIR services (Azure Health Data Services, Google Cloud Healthcare API, AWS HealthLake)
Identity	**Keycloak** (OIDC, MFA, WebAuthn/passkeys, identity brokering)	Self-hostable inside the HDS scope, standards-based, no per-user fee	Commercial IdPs if their hosting is HDS-compliant
Authorization	**Open Policy Agent** (policies as code, sidecar)	Low-latency local decisions, testable and versioned policies	AWS Cedar, OpenFGA
Video	**LiveKit** SFU (open-source) + TURN servers	EU self-hosting, simulcast, E2EE support, horizontal scaling	Jitsi, mediasoup; CPaaS (Vonage, Daily) if time-to-market dominates
Events	**Apache Kafka** (managed) + schema registry + Debezium outbox relay	Durable, replayable log, per-key ordering, high throughput	RabbitMQ, NATS JetStream, Pulsar
Datastores	PostgreSQL (managed, Multi-AZ), Redis/Valkey, Cassandra, OpenSearch, S3-compatible object storage	Each justified per component — see §3	—
Keys & secrets	Cloud KMS backed by HSM (FIPS 140-3) + HashiCorp Vault	Hardware-protected keys, rotation, dynamic credentials	Cloud-native secrets manager
Observability	OpenTelemetry → Prometheus / Grafana / Loki / Tempo; SIEM	Open standards, one correlation ID end-to-end	Datadog, Elastic (hosting permitting)
Delivery	CI pipelines, Argo CD (GitOps), Terraform, image signing (cosign), scanning (Trivy)	Every change reviewed, reproducible and traceable	Flux, Pulumi
Front-ends	React + TypeScript (web), React Native (mobile)	Shared skills and components across patient, doctor and pharmacy apps	Native Swift/Kotlin

1.10 How the architecture serves MedX priorities

Priority	Architectural answer	Details
H1 Data privacy	TLS 1.3 / mTLS on every hop; AES-256 storage encryption + per-patient envelope encryption; PHI confined to the clinical zone	§5.5–5.6
H2 Patient records	Dedicated FHIR-based Records service, append-only history, consent registry, full audit	§2.4
H3 Emergency access	Break-glass on an isolated, two-region, Tier-0 path; post-hoc review	§2.4
H4 AuthN / AuthZ	Keycloak (OIDC, MFA, passkeys) + OPA (RBAC + ABAC) evaluated beside each service	§2.1
H5 Data integrity & secure files	ACID stores, versioning, hash chains, checksums, Object Lock, immutable backups, scanned & encrypted document store	§3.8
Medium M1–M5	Scheduling, consultation, prescription services; masking obligations in the PDP; calendar adapters	§2.2, §2.3, §2.5
Low L1–L6	Pharmacy, notification, billing services; anonymisation pipeline — designed now, delivered last	§8 Roadmap

The complete requirement-by-requirement mapping is in the traceability matrix.

2. Detailed Component Designs

Deliverable 2. Functionalities, interfaces and interactions of every MedX component. Architecture overview: 1. High-level architecture.

2.0 How to read this section

Each of the eight business components of Appendix 1 has its own document with the same structure: **mission & priority → responsibilities (mapped to Appendix 1) → internal design → key flows → API → events → data owned → security → scalability & resilience → design decisions & trade-offs.**

#	Component	Priority (Appendix 2)	Zone	Tier	Document
①	User Management (identity & access)	**HIGH** (H4) · Low (L2 preferences)	Identity & access	0	01-user-management.md
②	Teleconsultation Scheduling	Medium (M1, M5)	Clinical	1	02-scheduling.md
③	Consultation (video, chat, files)	Medium (M2)	Clinical + media plane	1	03-consultation.md
④	Patient Records + Emergency Access	**HIGH** (H2, H3, H5) · Medium (M4)	Clinical	0 / 1	04-patient-records.md
⑤	Prescription Management	Medium (M3)	Clinical	1	05-prescription.md
⑥	Pharmacy Interface	Low (L1)	Clinical (minimised data)	2	06-pharmacy-interface.md
⑦	Notification	Low (L2, L3)	Engagement	2	07-notification.md
⑧	Billing & Payment	Low (L5, L6)	Payment	2	08-billing-payment.md
—	Platform components (gateways, policy engine, event bus, document store, audit, keys, observability, data platform)	**HIGH** (H1, H4, H5) · Low (L4)	Edge / platform	0–2	09-platform-components.md

2.1 Common service blueprint

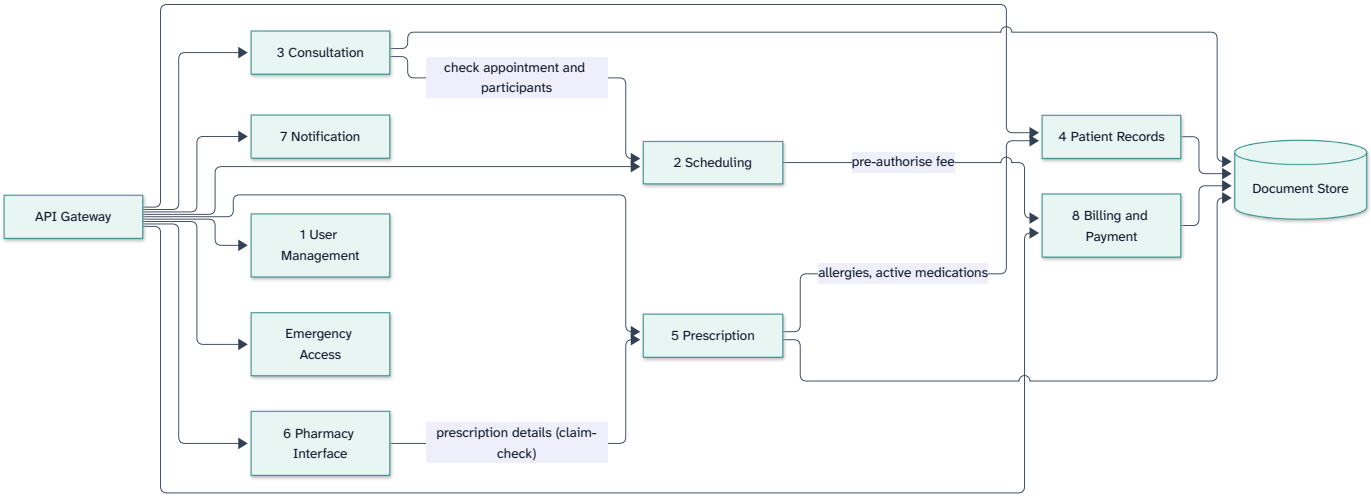
Every service is generated from the same **golden-path template**, so security and operability do not depend on each team's memory.

Concern	Standard applied to every service
API contract	OpenAPI 3.1 (FHIR CapabilityStatement for the Records service), URI versioning (`/v1`), additive changes only within a version, cursor pagination, errors as `application/problem+json` (RFC 9457), `Idempotency-Key` required on every `POST` that creates something.
Authentication	JWT validated again in the service (defence in depth, even behind the gateway); service-to-service identity through mesh mTLS (SPIFFE IDs).
Authorization	A Policy Enforcement Point (PEP) in the service calls the local OPA sidecar with `subject · action · resource · purpose` ; deny by default; obligations (masking) applied before the response leaves.
Data	Private database/schema and credentials (issued by Vault, 1 h TTL); reviewed migrations (Flyway); **UUIDv7** identifiers (time-ordered, never sequential integers exposed to clients); `outbox` table for events.
Events	CloudEvents envelope, schema in the registry, published via the outbox relay; consumers idempotent (processed-event table), dead-letter topic after 5 attempts.

Concern	Standard applied to every service
Resilience	≥ 3 replicas spread over 3 AZs (Tier 0/1), readiness/liveness probes, graceful shutdown, pod disruption budgets, timeouts + retries with jitter + circuit breakers (mesh), bulkheads for critical paths.
Observability	OpenTelemetry traces (W3C `traceparent` also propagated in Kafka headers), RED metrics, structured JSON logs **without PHI** (allow-listed fields, tokenised identifiers), SLO dashboard and alerts.
Delivery	CI: unit, contract (consumer-driven) and integration tests, SAST, dependency and container scanning, signed images; CD: GitOps, canary release through the mesh with automatic rollback.

2.2 Synchronous dependencies

Synchronous calls are limited to cases where a user is waiting for an answer. The graph is acyclic and never deeper than two hops; everything else goes through events.



Caller → callee	Why synchronous	Failure handling
Scheduling → Billing	The patient is waiting for the booking confirmation that depends on the payment authorisation	Timeout 5 s, slot hold kept 10 min so the patient can retry; no retry on non-idempotent calls without the idempotency key
Consultation → Scheduling	Join tokens are issued only to the appointment's participants within its time window	Local replica of today's appointments (built from events) used if Scheduling is unavailable
Prescription → Records	Allergy and interaction checks need the current allergy and medication lists	On failure the doctor sees "safety check unavailable" and must explicitly acknowledge before signing
Pharmacy Interface → Prescription	Pharmacies fetch the full prescription only when they open it (events carry identifiers only)	Retries with backoff; the pharmacy portal shows the item as "loading"
Services → Document Store	Upload/download of files	Uploads resumable; downloads retried

2.3 Event catalogue

Topic naming: `<domain>.<aggregate>.v<major>`. Partition key = aggregate identifier (keeps per-aggregate ordering). Events carry **identifiers, statuses and timestamps — never clinical content**; consumers fetch details from the owning service under their own authorisation (claim-check).

Topic	Events	Producer	Main consumers
`identity.user.v1`	UserRegistered, ProfileUpdated, PractitionerVerified, PractitionerSuspended, UserDeactivated, RoleAssigned, DelegationGranted/Revoked	① User Management	② (enable/disable agendas), ④ (create patient record & keys, revoke grants), ⑦, ⑧
`identity.preferences.v1`	PreferencesUpdated	①	⑦
`scheduling.appointment.v1`	AppointmentBooked, AppointmentRescheduled, AppointmentCancelled, AppointmentNoShow, AppointmentReminderDue, WaitlistSlotOffered, UrgentRequestCreated/Assigned	②	③ (plan rooms), ④ (care grants), ⑦, ⑧ (void/refund)
`consultation.session.v1`	ConsultationOpened, ConsultationStarted, ConsultationCompleted, ConsultationFailed, ChatTranscriptReady	③	② (complete/no-show), ④ (encounter, transcript archive), ⑧ (capture or refund), ⑦
`records.record.v1`	RecordUpdated, DocumentAdded, ResultReleased, ConsentChanged	④	④ projections (critical summary, timeline), ⑦ ("new document available")
`records.emergency.v1`	BreakGlassGranted, BreakGlassReviewDue, BreakGlassReviewed	④	Audit/SIEM (real-time alert), ⑦ (privacy office, patient notice)
`prescription.v1`	PrescriptionIssued, PrescriptionModified, PrescriptionRevoked, PrescriptionExpired, PharmacyAssigned, FulfilmentStatusChanged	⑤	⑥, ④ (medication list), ⑦, ⑧ (medication orders)
`pharmacy.fulfilment.v1`	FulfilmentAcknowledged, FulfilmentStatusUpdated, UnableToFulfil, InquiryMessagePosted	⑥	⑤ (status of record), ⑦
`notification.delivery.v1`	NotificationSent, NotificationDelivered, NotificationRead, NotificationFailed, ContactPointInvalid	⑦	① (invalid contact), analytics
`billing.payment.v1`	PaymentAuthorized, PaymentFailed, PaymentCaptured, PaymentVoided, RefundSucceeded, RefundFailed, InvoiceIssued	⑧	② (booking saga), ⑦ (receipts), ⑥ (paid orders)
`audit.events.v1`	AuditEvent (one per access or change to personal data)	every service, via its outbox	Audit Trail, SIEM

Example event (no drug name, no diagnosis — only what a consumer needs to decide whether to fetch more):

```
{
  "specversion": "1.0",
  "id": "01929e8c-6f1a-7c2b-9d3e-5a4b3c2d1e0f",
  "source": "medx/prescription-service",
  "type": "medx.prescription.issued.v1",
  "subject": "prescription/01929e8c-51aa-7d10-8f00-1b2c3d4e5f60",
  "time": "2026-03-02T10:15:30Z",
  "datacontenttype": "application/json",
  "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
  "data": {
    "prescriptionId": "01929e8c-51aa-7d10-8f00-1b2c3d4e5f60",
    "version": 1,
    "patientId": "01927f3a-0c55-7e21-a9f4-6d7e8f9a0b1c",
    "prescriberId": "01920b11-7a2e-7c3d-8e4f-5a6b7c8d9e0f",
    "pharmacyId": "01925d22-3b4c-7d5e-8f6a-7b8c9d0e1f2a",
    "validUntil": "2026-06-02"
  }
}
```


2.4 Component interaction summary

Component	Calls (sync)	Publishes	Consumes
① User Management	IdP (embedded), identity federation	`identity.*`	`notification.delivery.v1` (invalid contacts)
② Scheduling	Billing, calendar providers	`scheduling.appointment.v1`	`identity.user.v1`, `consultation.session.v1`, `billing.payment.v1`
③ Consultation	Scheduling, Document Store, SFU API	`consultation.session.v1`	`scheduling.appointment.v1`
④ Patient Records	Document Store, KMS	`records.*`	`identity.user.v1`, `scheduling.appointment.v1`, `consultation.session.v1`, `prescription.v1`
⑤ Prescription	Records, drug database, Document Store, HSM signing	`prescription.v1`	`pharmacy.fulfilment.v1`, `identity.user.v1`
⑥ Pharmacy Interface	Prescription, pharmacy systems	`pharmacy.fulfilment.v1`	`prescription.v1`
⑦ Notification	Messaging providers	`notification.delivery.v1`	almost every topic
⑧ Billing & Payment	PSP, Document Store	`billing.payment.v1`	`scheduling.appointment.v1`, `consultation.session.v1`, `prescription.v1`

2.1 ① User Management Service — Identity & Access

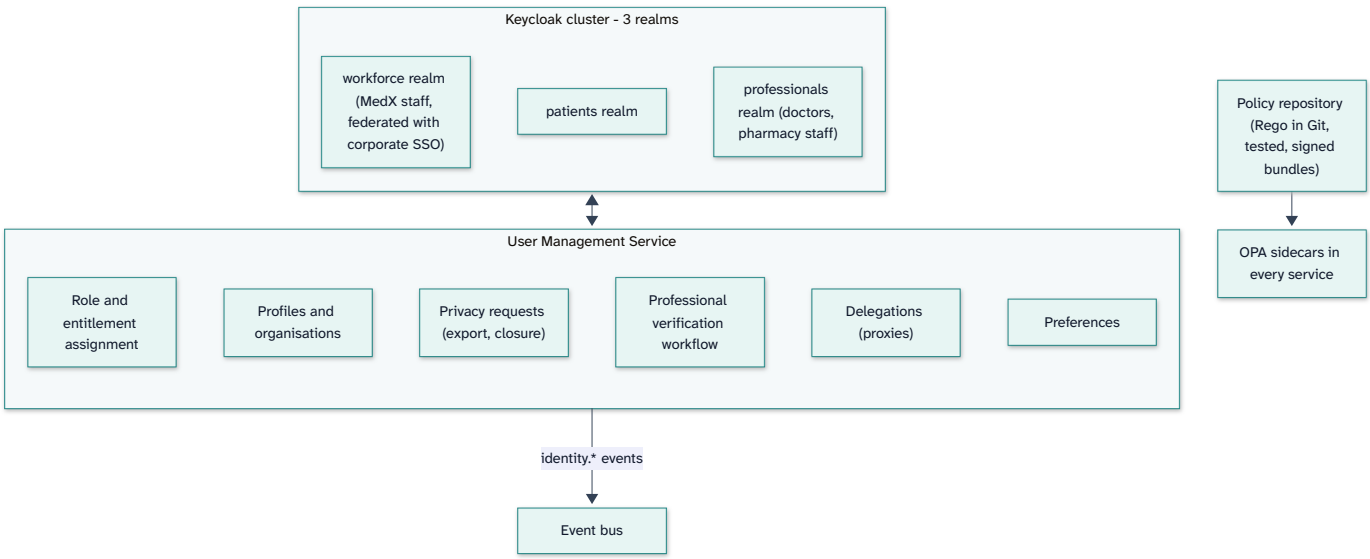
Priority	Zone	Criticality	Owns
HIGH — H4 authentication & authorization · Low — L2 user preferences	Identity & access	Tier 0 (login, token validation)	Identities, profiles, roles, organisations, delegations, preferences

Mission — know who every user is, prove it strongly, and give every other service a trustworthy basis to decide what that user may do.

Responsibilities

Appendix 1 feature	How MedX delivers it
Handle user registrations and profile updates	**Patients** self-register (e-mail + phone verification). Access to their medical record requires a verified identity (FranceConnect or ID-document check). **Doctors** go through professional onboarding: licence number checked against the national registry (RPPS / *Annuaire Santé*), identity document, manual approval. **Pharmacies** register as organisations (FINESS number checked); the pharmacy administrator invites staff.
Authenticate users	**Keycloak** identity provider: OpenID Connect Authorization Code + PKCE, mandatory **MFA for all professionals** (passkeys/WebAuthn, TOTP, or the professional smart card through Pro Santé Connect), risk-based MFA for patients, step-up for sensitive actions.
Authorize access based on user roles	Coarse **RBAC** roles carried in tokens + fine-grained **ABAC** decisions by the policy engine (OPA) beside each service: care relationship, consent, purpose of use, data sensitivity, organisation.
Store and manage user preferences	Preferences API (language, time zone, accessibility, notification channels per category, quiet hours, frequency caps). Changes are published as `PreferencesUpdated` so the Notification service applies them without calling back.

Internal design



- **Three realms** isolate populations with different risks: patient policies favour usability (passkeys, biometrics on mobile), professional policies are strict (MFA always, 15-minute idle timeout, 12-hour absolute session), workforce identities come from the corporate IdP with hardware security keys.
- **Policies are code**: Rego files in Git, unit-tested in CI (`opa test`), packaged as **signed bundles** pulled by the OPA sidecars. A policy change is reviewed like any code change and is fully traceable.
- The service keeps **attributes** (roles, organisation memberships, entitlements such as `emergency_access`) and exposes them in tokens; facts that change per request (care grants, consents) are looked up by the calling

service and passed to OPA as input.

Roles and permissions (RBAC layer)

Capability	Patient	Proxy	Doctor	Doctor + `emergency_access`	Pharmacist / technician	Support agent	Privacy officer	Platform admin
Manage own profile & preferences	✅	✅ (delegated)	✅	✅	✅	—	—	—
Book / cancel consultations	own	dependant's	—	—	—	assist, no PHI	—	—
Manage availability & agenda	—	—	own	own	—	—	—	—
Read medical record	own (with release rules)	scoped	**with care grant**	+ **break-glass**	❌	❌	audit metadata only	❌
Write clinical notes	upload own documents	—	with care grant	emergency note	❌	❌	❌	❌
Create / modify / revoke prescriptions	—	—	own patients	—	❌	❌	❌	❌
View prescriptions	own	dependant's	issued by self	via break-glass	**assigned to their pharmacy only** (minimised)	❌	❌	❌
Update fulfilment status	—	—	—	—	✅	❌	❌	❌
Issue refunds	request	request	—	—	—	≤ threshold (4-eyes above)	—	❌
View audit trail	own access history	—	—	—	—	❌	✅	❌
Operate infrastructure	—	—	—	—	—	—	—	JIT, time-boxed, **no PHI**

ABAC layer (illustrative policy)

RBAC answers "can a doctor read records?"; ABAC answers "can **this** doctor read **this** patient's record **now**, for **this** purpose?".

```
package medx.records

import rego.v1

default allow := false

# Treating doctor: verified, MFA-authenticated, holding an unexpired care grant for this patient
allow if {
    input.subject.role == "doctor"
    input.subject.verified
    input.subject.aal >= 2
    input.facts.care_grant.expires_at > input.now
    not sensitive_without_consent
}

# Break-glass: read-only, time-boxed, and never if the patient has lawfully opted out
allow if {
    "emergency_access" in input.subject.entitlements
    input.purpose_of_use == "ETREAT"
    input.action == "read"
    input.facts.emergency_grant.expires_at > input.now
    not input.facts.patient_opted_out_of_break_glass
}

sensitive_without_consent if {
    input.resource.security_label == "restricted"
    not input.facts.explicit_consent
}

# Obligations returned with the decision and applied by the service before responding
obligations contains {"mask": "psychotherapy-notes"} if input.purpose_of_use == "ETREAT"
```

Authentication flows

Audience	Flow	Factors	Session & tokens
Patient mobile app	OIDC Authorization Code + PKCE through the system browser	Password or passkey; risk-based second factor (new device, record download); biometric unlock of the stored refresh token	Access token 10 min, **DPoP-bound** (RFC 9449); refresh token rotated on every use, 30-day sliding window, device-bound
Patient web app	Same, confidential client via a thin backend-for-frontend holding tokens server-side	Same	HttpOnly, Secure, SameSite cookies; no tokens in browser storage
Doctor / pharmacy portals	OIDC Authorization Code + PKCE	**Mandatory MFA**: passkey/WebAuthn, TOTP or professional smart card (e-CPS via Pro Santé Connect); SMS OTP not allowed (SIM-swap risk)	Access token 5 min, idle timeout 15 min, absolute 12 h; step-up (`max_age` / `acr`) before signing a prescription or breaking the glass
Pharmacy systems (PMS)	OAuth 2.0 client credentials	**mTLS client certificate**, certificate-bound tokens (RFC 8705)	Token 5 min, per-pharmacy scopes and rate limits
MedX workforce	SSO with corporate IdP	FIDO2 hardware key	Privileged roles only through just-in-time elevation (approval, time-boxed, session recorded)

Token content — access tokens are JWTs signed with ES256 (keys rotated, published on the JWKS endpoint) and contain an opaque subject UUID, realm, roles, organisation, entitlements, authentication assurance (`acr` / `amr`) and

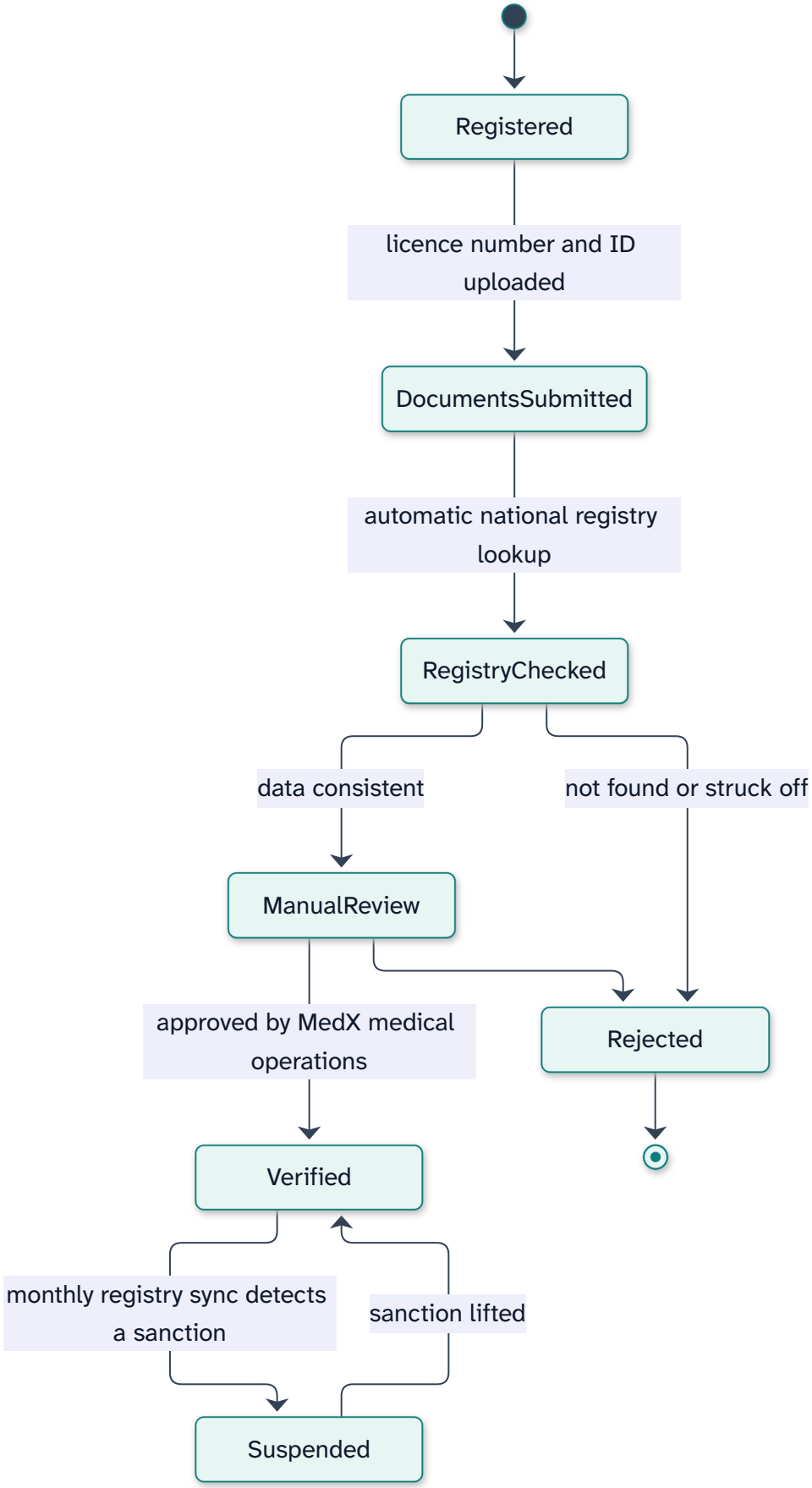
audience. They **never contain personal data** (no name, no e-mail, no birth date).

Revocation — logout, password reset and account suspension revoke the refresh-token family immediately; the short access-token lifetime bounds the remaining exposure, and a denylist in Redis (checked by the gateway) cuts suspended accounts within seconds.

Account protection — passwords hashed with Argon2id (PBKDF2-SHA512 in FIPS mode), minimum 12 characters, breached-password check, no forced periodic rotation (NIST SP 800-63B); progressive rate limiting and lockout; bot detection on registration and login; alerts for logins from new devices; account recovery that never downgrades assurance (a professional who loses a factor goes through identity re-verification).

Key flows

Professional onboarding



Proxy access (delegation) — a parent is linked to a minor after document verification (the link expires at majority; care the minor asked to keep confidential is hidden from the parent by policy); an adult patient can delegate scoped rights (book, view prescriptions, view records) to a caregiver and revoke them at any time.

API (selection)

Method	Endpoint	Purpose
`POST`	`/v1/users`	Patient registration
`POST`	`/v1/practitioner-registrations`	Start professional onboarding
`GET` / `PATCH`	`/v1/users/me`	Read / update own profile
`GET` / `PUT`	`/v1/users/me/preferences`	Read / replace preferences (versioned, `If-Match`)
`POST` / `DELETE`	`/v1/users/me/delegations`	Grant / revoke proxy access
`POST`	`/v1/organizations/{orgId}/members`	Pharmacy administrator invites staff
`POST`	`/v1/admin/practitioner-registrations/{id}/decision`	Approve / reject (back-office, 4-eyes)
`POST`	`/v1/users/me/privacy-requests`	GDPR access/portability request, account closure
—	`/realms/{realm}/protocol/openid-connect/*`	Standard OIDC endpoints (authorize, token, JWKS, logout)

Data owned

PostgreSQL (`usermgmt` database) + Keycloak's own PostgreSQL database — details in §3.4. Contact data (phone, address, national identifier) is encrypted at field level; e-mail is stored encrypted **with a blind index** (keyed HMAC) so it can still be looked up for login without being readable.

Scalability & resilience

- Token **validation** is local everywhere (gateway and services cache the JWKS), so the IdP only handles logins and refreshes: ~3 M per day at Year 3, about 35/s on average and 300/s at peak — modest, but password hashing is CPU-intensive, so login endpoints are rate-limited and Keycloak nodes sized on hashing cost.
- Keycloak runs as a 3+ node cluster across AZs with a distributed session cache, **active in both regions** (Tier 0): clinicians can still sign in during a regional failover.
- The User Management service itself is stateless and scales horizontally; profile reads are cached for 60 s.

Design decisions & trade-offs

Decision	Alternative	Why this choice
Reuse a proven IdP (Keycloak)	Build authentication in-house	Authentication is where custom code fails most dangerously; Keycloak brings OIDC, MFA, WebAuthn, brokering and audits. Cost: running and patching a stateful cluster.
Short-lived JWTs validated locally	Opaque tokens + introspection on every call	Removes the IdP from the hot path (latency, resilience of the emergency path). Cost: revocation is not instant → 5–10 min lifetimes + denylist for high-risk events.
RBAC + ABAC with OPA	RBAC only	RBAC alone cannot express care relationships, consent or break-glass. OPA keeps policies testable and versioned.
Separate realms per population	One realm	Different assurance levels and session rules; a misconfiguration on the patient side cannot weaken professional accounts.

2.2 ② Teleconsultation Scheduling Service

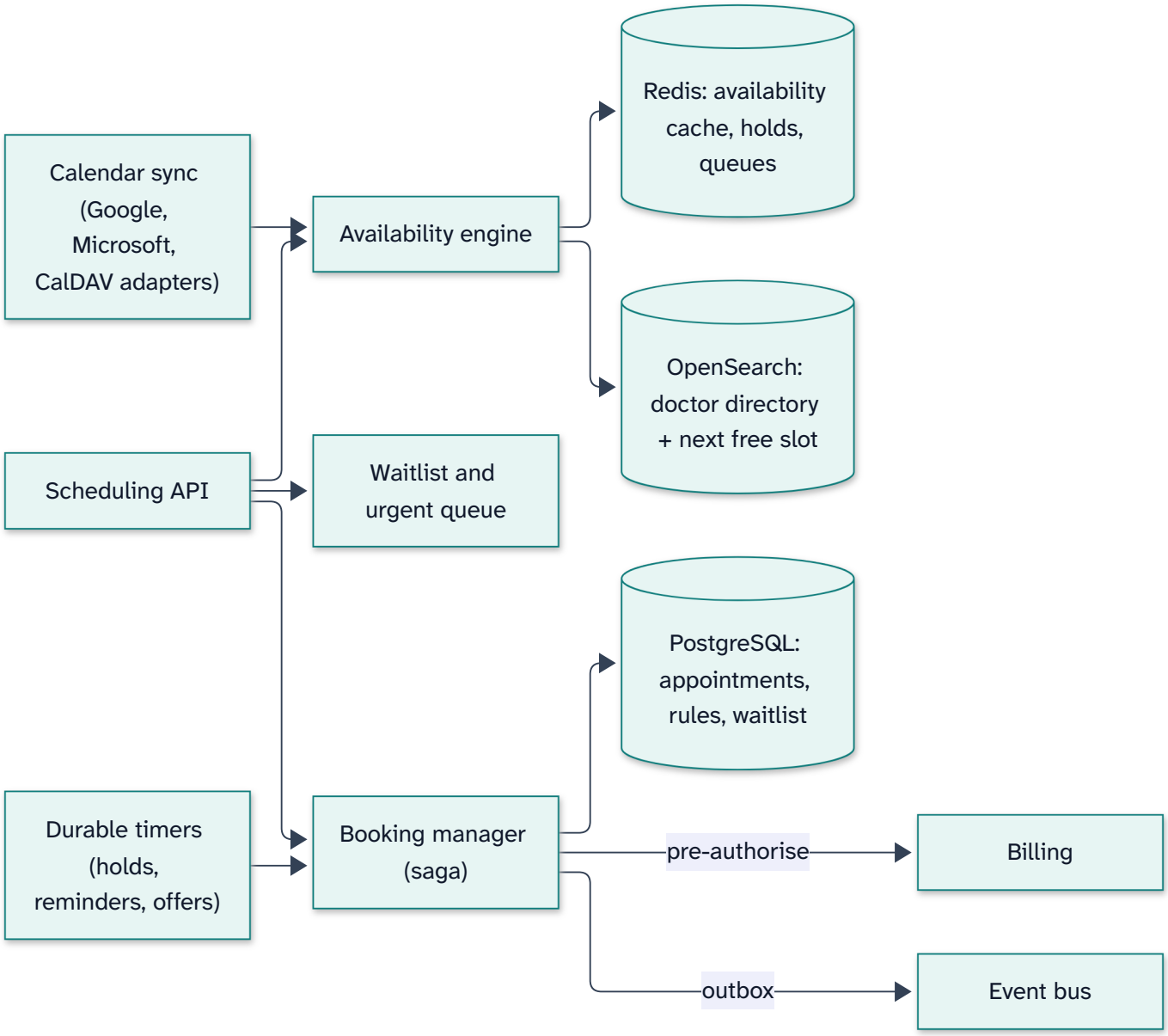
Priority	Zone	Criticality	Owns
Medium — M1 scheduling · M5 external calendars	Clinical (an appointment with a specialist is health data)	Tier 1	Availability rules, appointments, waitlists, urgent requests, calendar connections

Mission — let patients find and book the right consultation in a few taps, never double-book a doctor, keep doctors informed in real time, and give urgent cases a faster lane.

Responsibilities

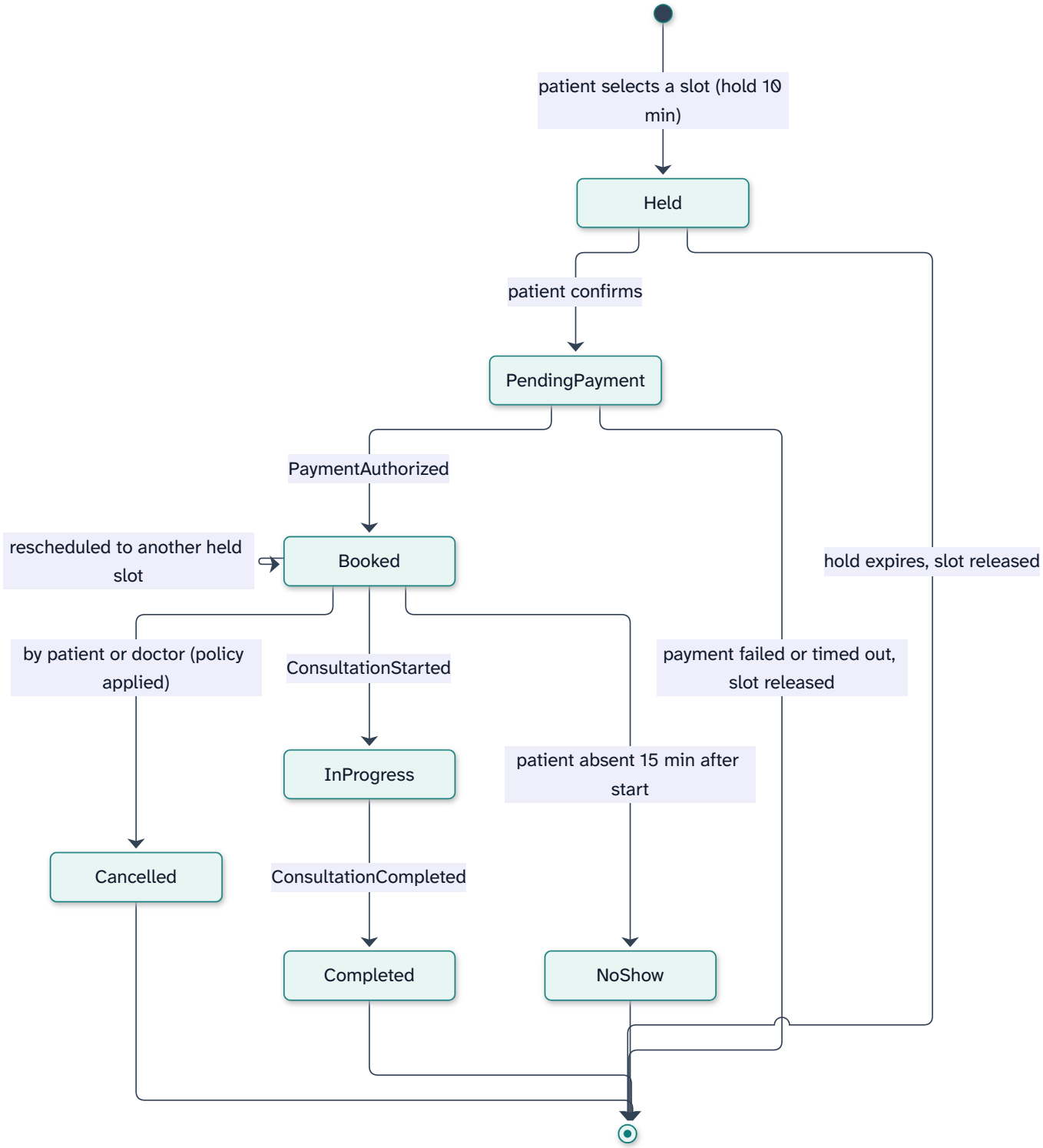
Appendix 1 feature	How MedX delivers it
Book, reschedule, cancel consultations	Slot search → short **hold** → confirmation with payment pre-authorisation (saga with Billing). Rescheduling moves the same appointment to a new held slot; cancellation applies the cancellation policy (free until 24 h before).
Notify doctors of new bookings or changes	`AppointmentBooked/Rescheduled/Cancelled` events → Notification (push/e-mail per preferences) **and** live update of the doctor's agenda through the Real-time Gateway.
Integrate with doctors' calendars	Connectors for Google Calendar, Microsoft 365 (Graph) and CalDAV: external busy times block MedX slots; MedX appointments are written back **without any patient data**.
Waitlists and priority scheduling for urgent cases	Waitlist per doctor or specialty with automatic offers when a slot is released; triage questionnaire → urgency level; **on-demand urgent queue** served by on-call doctors; reserved urgent capacity.

Internal design

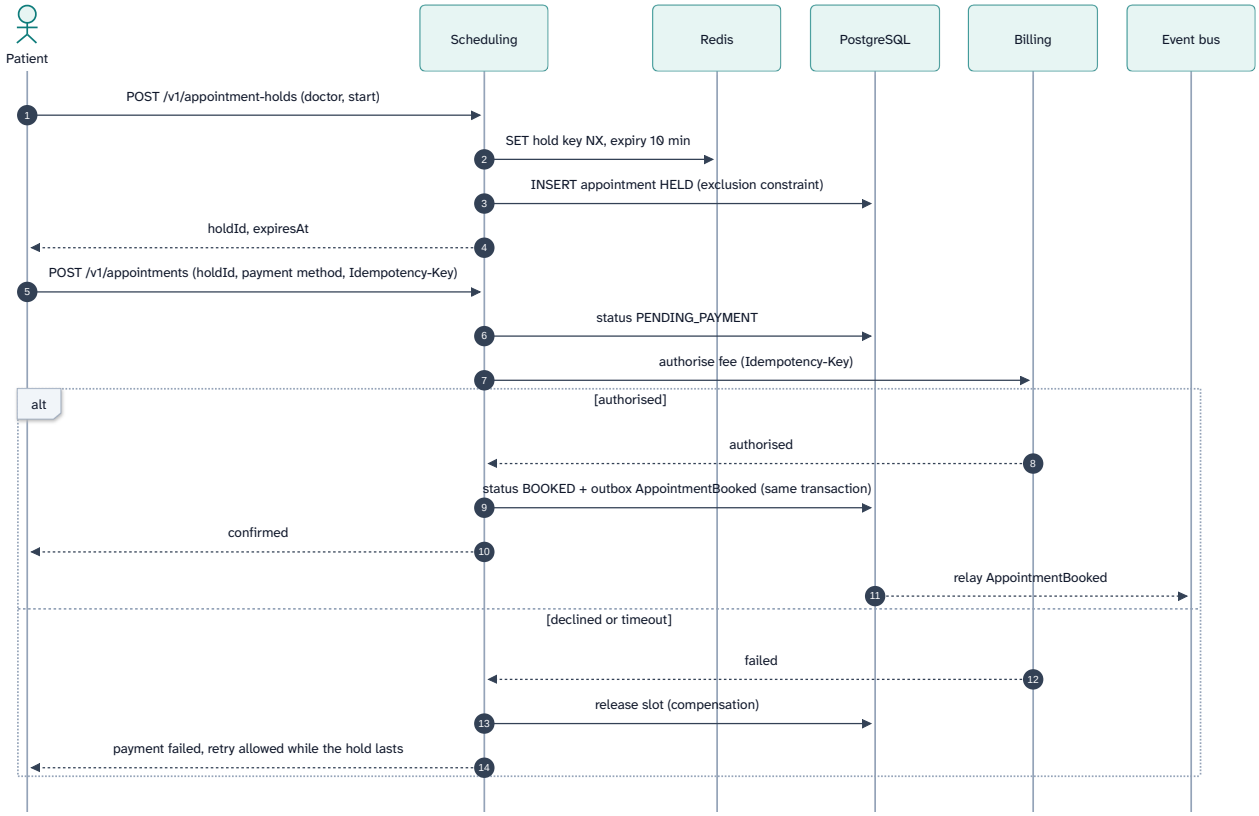


Module	Role
Availability engine	Computes free slots = recurring rules (RRULE, per consultation type) – exceptions (leave) – external busy blocks – booked/held slots – reserved urgent capacity. Pre-computes the next 60 days per doctor in Redis; recomputes incrementally on every relevant event.
Booking manager	Holds, confirmations, rescheduling, cancellations; orchestrates the payment saga; owns the appointment state machine.
Waitlist & urgent queue	Priority queues, slot offers with exclusive time-boxed holds, matching of urgent requests to on-call doctors.
Calendar sync	OAuth connections, push-notification webhooks, incremental delta sync, outbound event writing, subscription renewal, nightly reconciliation.
Durable timers	Database-backed scheduler (survives restarts) for hold expiry, reminders (T-24 h, T-1 h, T-10 min), waitlist offer expiry.
Directory projection	Denormalised doctor profiles (specialty, languages, fees, next available slot) in OpenSearch for fast search.

Appointment lifecycle



Booking saga and double-booking prevention



Two independent safeguards prevent double booking:

1. **Fast path** — an atomic Redis hold (`SET … NX PX 600000`) rejects concurrent attempts on the same slot in under a millisecond.
2. **Source of truth** — a PostgreSQL **exclusion constraint** makes overlapping active appointments for a doctor physically impossible, even if Redis is unavailable or a bug bypasses the hold:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE appointment
  ADD CONSTRAINT appointment_no_overlap
  EXCLUDE USING gist (
    practitioner_id WITH =,
    tstzrange(starts_at, ends_at, '[)') WITH &&
  )
  WHERE (status IN ('HELD', 'PENDING_PAYMENT', 'BOOKED', 'IN_PROGRESS'));
```

Waitlist and urgent cases

- **Waitlist** — a patient can wait for an earlier slot with a doctor or any doctor of a specialty. When a slot is released, the matcher offers it to the best candidate, ordered by **urgency level then waiting time**, with an exclusive 15-minute hold; if declined or expired, the next candidate gets it.
- **Triage** — a short, medically validated questionnaire (rule-based, conservative) classifies the request: `EMERGENCY` (red flags → **booking stopped, the app displays the emergency number 15/112**), `URGENT` (same day), `SOON` (≤ 48 h), `ROUTINE`. Telehealth must never delay emergency care.

- **Urgent on-demand queue** — `URGENT` requests enter a queue served by on-call doctors (matched on specialty, language, licence). A doctor has 2 minutes to accept before the request moves on; a supervisor dashboard tracks the waiting-time objective (< 30 min) and escalates.
- **Reserved capacity** — doctors may reserve a share of their slots for urgent cases; unused reserved slots are released to everyone 2 hours in advance.

External calendar integration

Aspect	Design
Providers	Google Calendar API, Microsoft Graph (Outlook / Microsoft 365), CalDAV (Apple iCloud and others)
Consent & scopes	Doctor connects through OAuth 2.0 with **minimum scopes**: read free/busy, write only MedX events (a dedicated "MedX" calendar where the provider supports it). CalDAV uses an app-specific password stored in Vault.
Inbound (busy times)	Provider push notifications (Google watch channels, Graph change subscriptions) → webhook through the gateway → incremental fetch (sync token / delta link) → `external_busy_block` updated → availability recomputed within seconds. Fallback polling every 10 min; nightly full reconciliation; automatic renewal of expiring subscriptions.
Outbound (MedX bookings)	Each booking creates an event titled **"MedX teleconsultation"** with a link to the portal — **no patient name, no reason, no specialty**. Reschedules and cancellations update or delete it (mapping `appointment_id ↔ external_event_id`, idempotent).
Data minimisation	Only start/end of busy periods are stored — never titles, attendees or descriptions of the doctor's private events.
Conflicts	MedX appointments are the source of truth. If the doctor adds an overlapping private event, MedX alerts the doctor; it never cancels a patient automatically.
Failures	Per-provider rate-limit handling with exponential backoff and circuit breaker; if a provider is down, the last known busy blocks are used and the doctor sees a "calendar sync delayed" banner.
Secrets	OAuth refresh tokens encrypted with Vault Transit, revoked when the doctor disconnects.

API (selection)

Method	Endpoint	Purpose
`GET`	`/v1/practitioners?specialty=&language=&date=`	Directory search (OpenSearch)
`GET`	`/v1/practitioners/{id}/slots?from=&to=&type=`	Free slots (Redis cache)
`POST`	`/v1/appointment-holds`	Hold a slot for 10 min
`POST`	`/v1/appointments`	Confirm booking (`Idempotency-Key`, `holdId`, payment method)
`POST`	`/v1/appointments/{id}/reschedule`	Move to another held slot
`POST`	`/v1/appointments/{id}/cancel`	Cancel with reason (policy → refund/void)
`POST`	`/v1/waitlist-entries`	Join a waitlist
`POST`	`/v1/urgent-requests`	Triage answers → urgent queue or emergency advice
`PUT`	`/v1/practitioners/me/availability-rules`	Doctor's recurring availability
`POST`	`/v1/practitioners/me/calendar-connections`	Start OAuth connection to an external calendar
`POST`	`/v1/webhooks/calendars/{provider}`	Provider push notifications (validated channel tokens)

Events

- **Publishes** `scheduling.appointment.v1`: AppointmentBooked, AppointmentRescheduled, AppointmentCancelled, AppointmentNoShow, AppointmentReminderDue, WaitlistSlotOffered,

UrgentRequestCreated, UrgentRequestAssigned.

- **Consumes** `billing.payment.v1` (PaymentAuthorized/Failed), `consultation.session.v1` (Started/Completed → status), `identity.user.v1` (PractitionerVerified enables the agenda; PractitionerSuspended freezes it and triggers patient rebooking).

Data owned

PostgreSQL: `availability_rule`, `availability_exception`, `appointment` (+ append-only `appointment_history`), `waitlist_entry`, `urgent_request`, `calendar_connection`, `external_busy_block`, `timer`, `outbox`. Redis: holds, per-doctor-day availability, waitlist queues. OpenSearch: doctor directory (public profile data only). Details in §3.4.

Security & privacy

- Patients see only their own appointments; doctors only their own agenda; support agents see masked data.
- Appointment reason and triage answers are health data: encrypted at field level, visible only to the patient and the booked doctor.
- Directory search is public (profiles only) and protected by rate limiting and bot detection against scraping.

Scalability & resilience

- **Reads dominate** (search and availability): served from OpenSearch and the Redis availability cache, so the database is not hit by browsing; cache entries are recomputed on events, not on reads.
- **Writes are modest** (200 k bookings/day at Year 3 ≈ 2–3/s, peaks ≈ 30/s) and contention is local to one doctor.
- Peaks (Monday 8–10 am, epidemics) are absorbed by autoscaling on request rate; if the payment provider is slow, holds keep the slot for the patient.

Design decisions & trade-offs

Decision	Alternative	Why this choice
Pre-computed availability	Compute on every request	Fast, predictable search under peak load. Cost: invalidation logic, bounded staleness (seconds).
Redis hold **and** DB exclusion constraint	Only one of them	Redis gives speed; the constraint gives a guarantee. Belt and braces for a problem users notice immediately.
Payment pre-authorisation at booking, capture after the consultation	Pay at the end	Reduces no-shows and failed payments; the patient pays only for a consultation that happened. Cost: authorisations expire after ~7 days → for distant appointments the card is saved (SCA) and authorised 24 h before.
Free/busy only, no PHI in external calendars	Full two-way sync	External calendars are outside MedX's trust boundary and HDS scope.

2.3 ③ Consultation Service — Video, Audio, Chat & Files

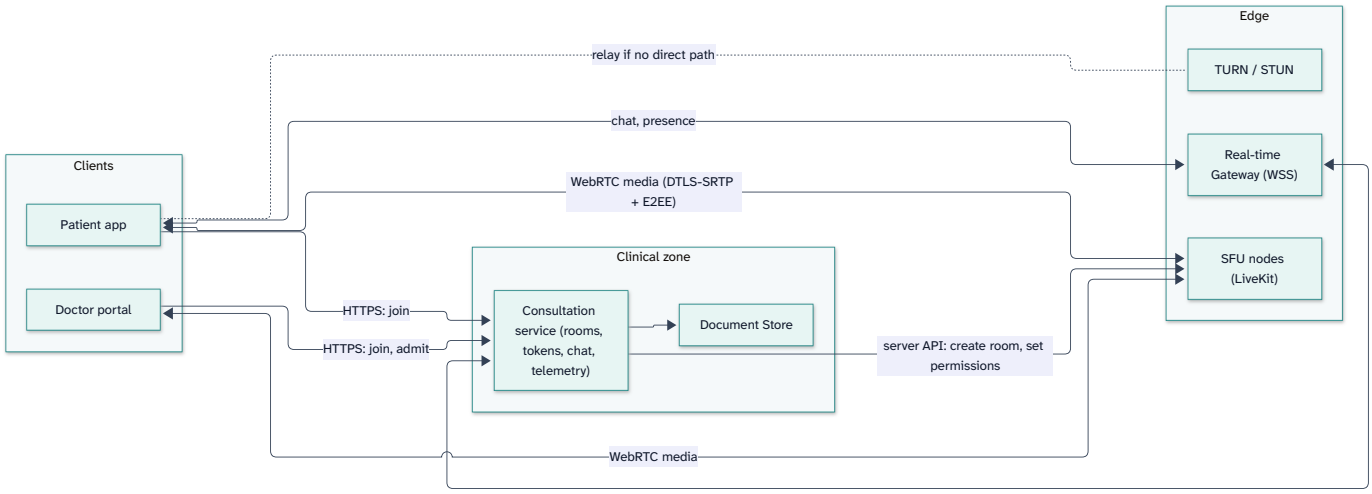
Priority	Zone	Criticality	Owns
Medium — M2 consultation services	Control plane in the clinical zone · media plane at the edge	Tier 1	Virtual rooms, participants, session events, chat history (hot), call-quality telemetry

Mission — give patient and doctor a reliable, private virtual room: good video on a bad network, a chat that never loses a message, and a safe way to exchange medical documents.

Responsibilities

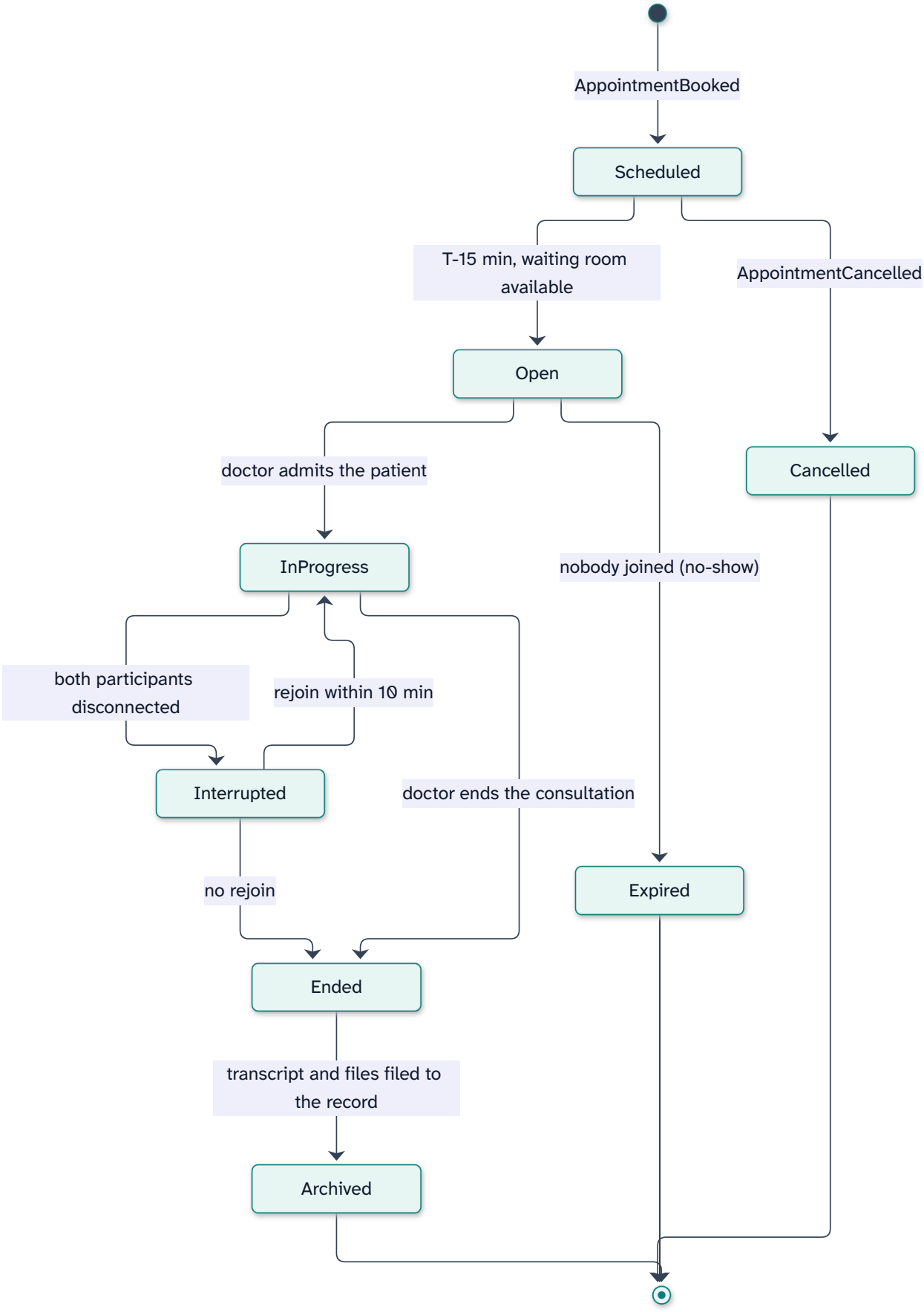
Appendix 1 feature	How MedX delivers it
Create virtual rooms	One room per appointment, planned when the appointment is booked and opened 15 min before; **waiting room** until the doctor admits the patient; unguessable 128-bit room identifiers.
Handle video and audio streaming	**WebRTC** through a self-hosted **SFU** (LiveKit) with TURN/STUN relays; simulcast and adaptive bitrate; automatic audio-only fallback; reconnection with ICE restart.
Enable text chat during consultations	Chat over the Real-time Gateway (WSS), persisted and ordered per room, with delivery/read receipts; transcript archived into the patient record at the end.
Allow file transfers for medical documents	Uploads through the **Secure Document Store** (quarantine, antivirus, type checks, encryption); files shared in the chat; doctor can file them into the record.

Control plane vs media plane

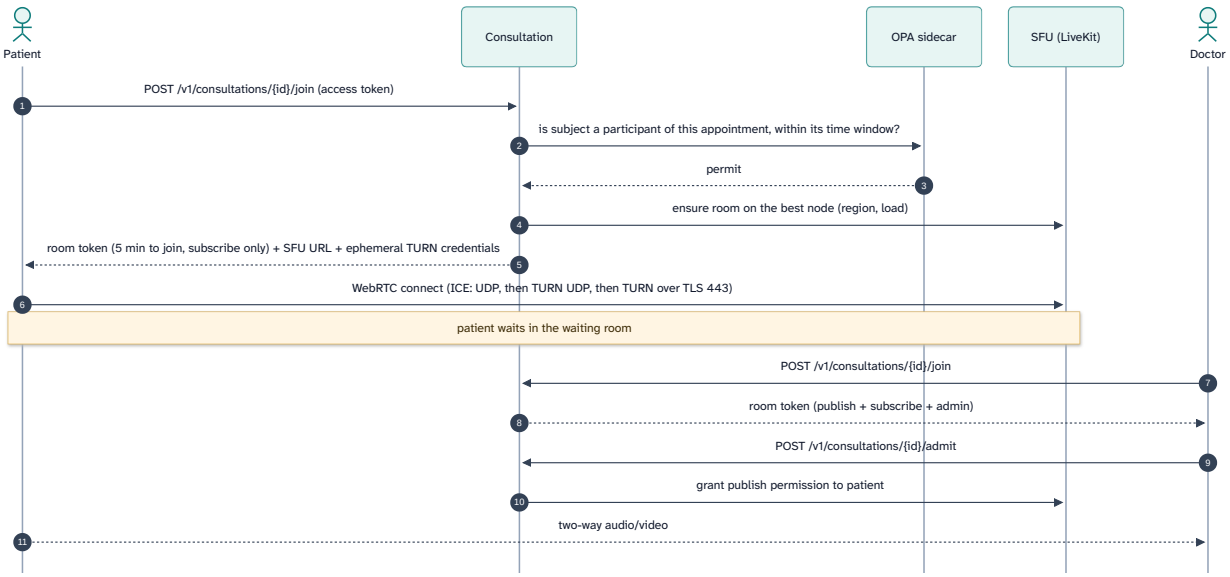


- The **control plane** (Consultation service) decides *who* may enter *which* room *when*; it never touches media.
- The **media plane** (SFU + TURN) only forwards encrypted packets; it stores nothing and holds no health data at rest. Media nodes live in public subnets because WebRTC needs public UDP ports.

Room lifecycle



Joining a consultation



Tokens are **short-lived, room-scoped and participant-scoped**; the participant identity inside the SFU is a pseudonymous ID, never the patient's name. Additional participants (interpreter, caregiver) can be invited only with the patient's consent and receive their own scoped token.

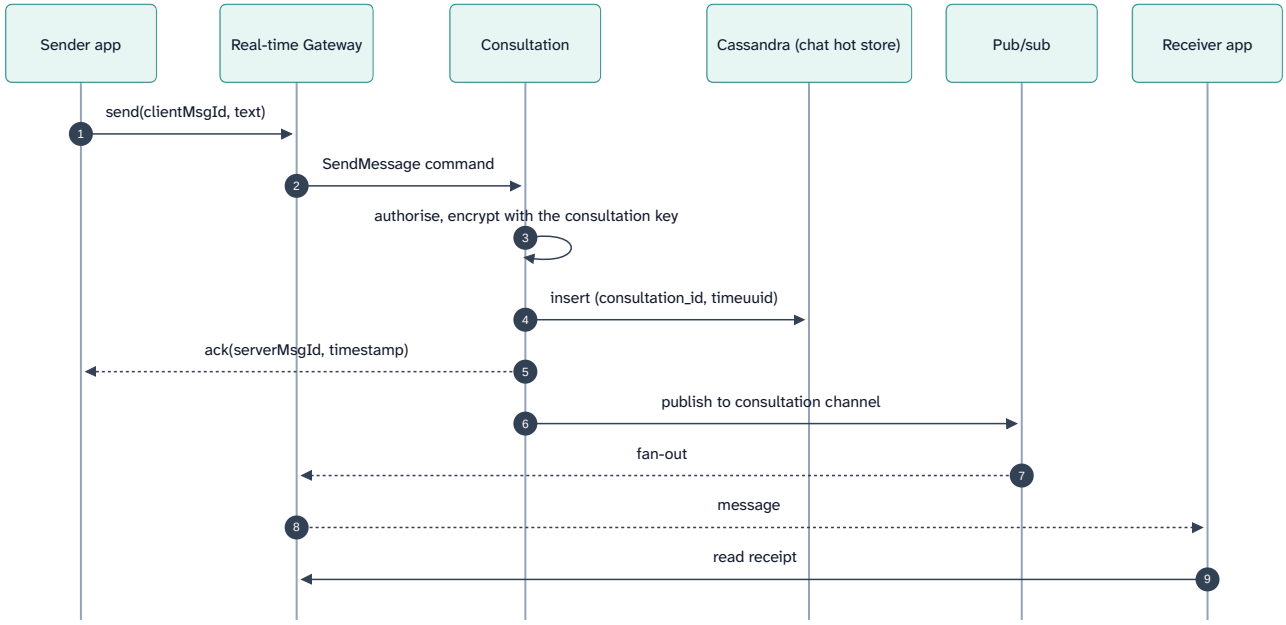
Media quality on real-world networks

Technique	Effect
Simulcast (3 layers) + dynacast	Each receiver gets the layer its bandwidth supports; unused layers are paused to save uplink.
Opus audio with in-band FEC and DTX	Speech stays intelligible up to ~20 % packet loss; audio is prioritised over video.
Congestion control (transport-wide CC)	Bitrate adapts continuously; below ~150 kbit/s the call switches to **audio-only** automatically.
TURN over TLS on port 443	Works on restrictive hospital and corporate networks.
ICE restart + reconnect	Moving from Wi-Fi to 4G does not end the consultation.
Pre-call test	Camera, microphone and bandwidth checked before the patient enters the waiting room.
Quality telemetry	WebRTC statistics (RTT, jitter, loss, freezes) sampled every 10 s — no media, no PHI — feed dashboards and alerts per region and per TURN node.

Privacy of the media

- **Hop-by-hop encryption is mandatory**: WebRTC always uses DTLS-SRTP.
- **End-to-end encryption (E2EE)** with insertable streams (SFrame) is **on by default for 1-to-1 consultations** when both devices support it: keys are negotiated between participants' devices and never stored by MedX, so neither MedX nor the SFU operator can decrypt audio or video.
- **No recording** of consultations (privacy by design). If recording is ever introduced (e.g., for training), it would require explicit consent, disable E2EE for that session and be stored as a document of the record.

Chat



- **Exactly-once effect**: at-least-once delivery + deduplication on `clientMsgId`; ordering by server time-UUID.
- **Retention**: the hot store keeps messages 30 days (TTL) for convenience; when the consultation ends, the full transcript is **archived into the patient record** (`ChatTranscriptReady` → Records creates a document), which is the legal, long-term copy.
- Typing indicators and presence are ephemeral (never stored).

File sharing

1. The app asks the Document Store for an upload slot → pre-signed URL to a **quarantine** bucket (5 min validity, size ≤ 50 MB, allowed types: PDF, JPEG, PNG, HEIC).
2. The file is scanned (antivirus, real type from magic bytes, PDF active content removed, image metadata such as GPS location stripped), then encrypted and moved to the documents bucket with its SHA-256 checksum.
3. A "file shared" message referencing the document ID appears in the chat; downloads go through the Document Store with the same authorisation as the consultation and are audited.
4. Files sent by the patient are attached automatically to the encounter as *patient-provided documents*; the doctor decides which files to file permanently into the record.

API (selection)

Method	Endpoint	Purpose
`POST`	`/v1/consultations/{id}/join`	Room token + SFU URL + ephemeral TURN credentials
`POST`	`/v1/consultations/{id}/admit`	Doctor admits the patient from the waiting room
`POST`	`/v1/consultations/{id}/participants`	Invite interpreter/caregiver (patient consent required)
`POST`	`/v1/consultations/{id}/end`	Doctor ends the consultation
`GET`	`/v1/consultations/{id}/messages?cursor=`	Chat history (paginated)
`POST`	`/v1/consultations/{id}/files`	Request an upload slot (delegated to the Document Store)
`POST`	`/v1/telemetry/webrtc`	Call-quality statistics

Method	Endpoint	Purpose
`WSS`	`/v1/realtime` → channel `consultation:{id}`	Chat, receipts, presence

Events

- **Publishes** `consultation.session.v1` : ConsultationOpened, ConsultationStarted, ConsultationCompleted (duration, participants), ConsultationFailed (nobody could connect → automatic refund), ChatTranscriptReady.
- **Consumes** `scheduling.appointment.v1` (plan, move or cancel rooms).

Data owned

PostgreSQL (`consultation` , `participant` , `session_event` , `quality_summary`), Cassandra (`chat_message` , 30-day TTL, encrypted content), Redis (room → SFU node routing, presence). Files belong to the Document Store.

Scalability & resilience

- **Sizing (Year 3)**: 10 k concurrent sessions → 20 k participants; ~1.2 Mbit/s average per received stream → ~24 Gbit/s of SFU egress. At ~200 participants per 8-vCPU node (to be confirmed by load tests), about **100 media nodes at peak**, autoscaled up to 200 for the ×2 headroom, spread over three AZs.
- **Placement**: each 1-to-1 room lives on one node in the region closest to its participants; nodes are **drained, never killed** on scale-in (no new rooms, existing calls finish).
- **TURN** capacity sized for ~20 % of sessions relayed (restrictive networks).
- **Failure of a media node**: clients reconnect automatically to another node within seconds (ICE restart); the room state is re-created from the control plane.
- **Degraded mode**: if video is impossible, audio-only; if media fails entirely, chat keeps working and the doctor can call the patient by phone (the event is logged).

Design decisions & trade-offs

Decision	Alternative	Why this choice
Self-hosted open-source SFU (LiveKit)	Video CPaaS (Vonage, Daily…)	Media stays on HDS-certified EU infrastructure; lower cost at scale; E2EE support. Cost: operating media servers (UDP, autoscaling, TURN). A CPaaS remains a fallback option for time-to-market.
SFU	Peer-to-peer mesh / MCU	P2P gives no control on quality, relaying or multi-party (interpreter); an MCU decodes and mixes streams (CPU-heavy, breaks E2EE). The SFU forwards packets: scalable and compatible with E2EE.
E2EE on by default for 1-to-1	Hop-by-hop encryption only	Maximum confidentiality for the most sensitive conversations. Cost: no server-side recording or transcription, a little more device CPU.
Chat hot store + archive in the record	Chat written directly in the record	Low-latency messaging without loading the records store; the record still receives the complete, signed transcript.

2.4 ④ Patient Records Management Service & Emergency Access

Priority	Zone	Criticality	Owns
HIGH — H2 records management · H3 emergency access · H5 integrity · Medium — M4 masking & redaction	Clinical	**Tier 0** (read path, emergency access) · Tier 1 (writes)	The medical record (FHIR resources and history), consents, care grants, emergency grants, critical summaries, patient data keys

Mission — keep each patient's lifelong medical record **complete, correct and tamper-evident**, show it only to the right people for the right reason, and make its critical part available **in seconds** in an emergency.

Responsibilities

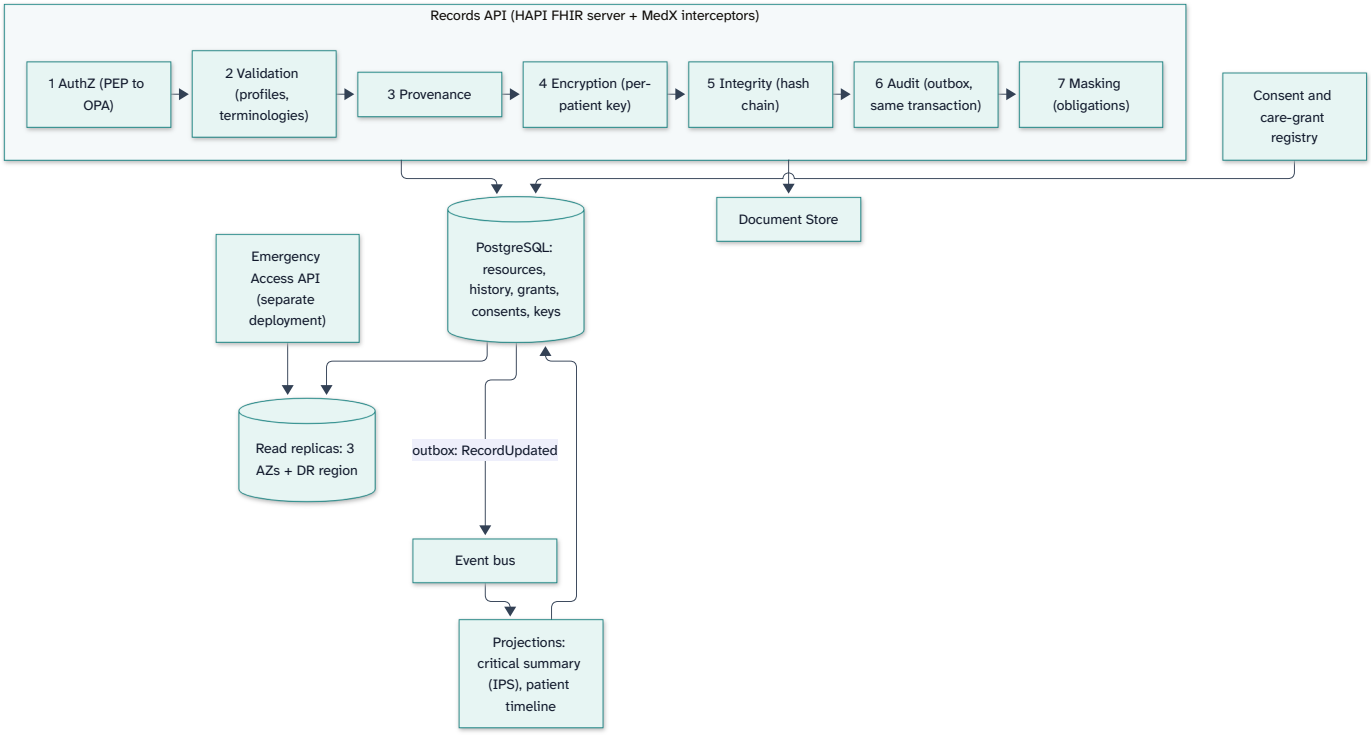
Appendix 1 feature	How MedX delivers it
Store and manage medical histories, consultation notes and other records	**HL7 FHIR R4** resources (conditions, allergies, medications, immunisations, procedures, observations, reports, encounters, clinical notes, documents) stored by a FHIR server (HAPI FHIR on PostgreSQL); binaries in the Secure Document Store.
Allow doctors to add/edit records post-consultation	Validated writes with optimistic concurrency; notes are drafts until **signed**; after signature, changes are **addenda** (new version with a reason) — the original is never lost. Wrong-patient entries are marked *entered-in-error*, not deleted.
Give patients access to their records (with restrictions)	Patient portal API with patient-scoped tokens; release rules (e.g., results released after doctor review where the law allows), third-party information redacted, proxy scopes, adolescent confidentiality.
Ensure data integrity and regulatory compliance	Append-only history, per-patient hash chain anchored daily in WORM storage, provenance on every write, profile validation, full audit of reads and writes, consent registry, retention rules, per-patient encryption and crypto-shredding.
Emergency access (Appendix 2, H3)	Dedicated **Emergency Access** module deployed separately, serving a pre-computed **International Patient Summary** under a time-boxed break-glass grant — see below.

Clinical data model (FHIR R4)

MedX concept	FHIR resource(s)	Notes
Patient identity	`Patient`	MedX ID + national health identifier (INS) stored encrypted
Medical history	`Condition`, `AllergyIntolerance`, `Procedure`, `Immunization`, `FamilyMemberHistory`	Coded with SNOMED CT / ICD-10
Medications	`MedicationStatement`, `MedicationRequest` (from Prescription)	Coded with ATC and national drug codes
Measurements & results	`Observation`, `DiagnosticReport`	LOINC codes; release rules apply
Consultation	`Encounter` + `Composition` (clinical note)	Composition signed by the author
Documents	`DocumentReference` → binary in the Document Store	SHA-256 hash in `attachment.hash`
Who did what	`Provenance` (every write) · `AuditEvent` (every access, in the Audit Trail)	
Patient choices	`Consent`	Sharing, research opt-in, break-glass opt-out where allowed
Emergency summary	`Composition` following the **IPS** profile	Pre-computed projection

Profiles: International Patient Summary (IPS) and national profiles (e.g., FR Core) — validation rejects malformed or uncoded data at write time.

Internal design



Every request crosses the same **interceptor chain**, so no endpoint can forget a control: authorisation → validation → provenance → encryption → integrity → audit → masking.

Write path, versioning and integrity

1. **Optimistic concurrency** — updates must send `If-Match: W/"<version>"`; a concurrent change returns `412` and the doctor merges — no silent lost update.
2. **Append-only history** — every change inserts a new row in `resource_history`; the database role used by the service has **no `UPDATE` / `DELETE` privilege** on history tables (enforced by grants and triggers).
3. **Signature of notes** — `Composition.status` goes `preliminary → final` when the author signs; afterwards only addenda are possible (`amended`, with reason and attestation).
4. **Hash chain per patient** — each version stores `h(n) = SHA-256( h(n-1) ‖ JCS(resource version) ‖ author ‖ timestamp )` (JCS = RFC 8785 canonical JSON). A nightly job recomputes chains; any break raises a security incident.
5. **Daily anchoring** — the Merkle root of all chain heads of the day is signed with an HSM key, timestamped (RFC 3161 qualified timestamp) and written to **WORM** storage: even a database administrator could not rewrite history without detection.
6. **Provenance** — who, when, from which encounter, on whose behalf, and why.
7. **Documents** — binaries carry a SHA-256 checksum computed at upload and verified at every download.

Access control

Access path	Who	Conditions	Scope
Self	Patient	Verified identity	Own record, subject to release rules
Proxy	Parent / caregiver	Delegation from User Management	Delegated scope; adolescent-confidential items hidden
Care grant	Doctor	Created automatically at booking with the patient's consent; valid until 30 days after the consultation; renewable; "treating doctor" designation for long-term follow-up	Full record except restricted categories without explicit consent
Patient share	Any verified doctor	Patient shares the record (e.g., with a specialist) for a chosen period	As chosen by the patient
Break-glass	Doctor holding `emergency_access`	Emergency declared, step-up MFA, not opted out	Read-only, 4 hours, this patient, restricted notes excluded
Pharmacist	—	**No access to the record** — only to prescriptions assigned to the pharmacy, through the Prescription service	—
MedX staff	Support, admins	**No access to clinical data**; privacy officers see audit metadata only	—

Sensitivity labels — resources in sensitive categories (mental health, sexual and reproductive health, HIV, genetics, addiction) are labelled `restricted` and require explicit patient consent to be shared beyond their author.

Patient view restrictions — (1) *release rules*: a new result can be held until the doctor has reviewed it (configurable per jurisdiction; immediate release where the law requires it, e.g., US information-blocking rules); (2) information about third parties written in a note can be redacted from the patient view; (3) proxies see only their delegated scope.

Data masking & redaction (M4)

The policy engine returns **obligations** with each decision; a shared masking library applies them to the FHIR response before it leaves the service.

Situation	Technique	Example
Support agent looks up an account	Partial masking	`+33 6 •• •• •• 42`, `j•••@g•••.com`, no clinical data at all
Pharmacist opens a prescription	Field allow-list	Name, birth date, prescribed items, prescriber — no diagnosis, no notes
Break-glass access	Category exclusion	Psychotherapy notes excluded; medications and allergies always shown
Patient portal	Release rules + redaction	Held results hidden; third-party information redacted
Export / sharing a document	Tagged-section redaction; PDF redaction that removes the text layer (not a black box drawn over text)	Shared discharge letter without family history
Logs, traces, analytics	Never logged; tokenised identifiers; de-identification	See §5.7

Emergency access (break-glass)

Emergency Access (Break-Glass)

High-priority requirement H3 · immediate access to critical data for authorized professionals, with full accountability afterwards

1 Identify the patient

Exact-match search only:
INS / MedX ID, or name + birth date + sex
Minimal demographics to confirm identity
No browsing, no wildcards

2 Declare emergency

Reason code (unconscious, life-threatening, unable to consent…)
Free-text justification
Accountability statement "I will be audited"

3 Step-up MFA

Passkey or smart card (e-CPS) in the last 15 min
Policy engine checks the emergency_access entitlement and the patient's opt-out

4 Time-boxed grant

4 hours · read-only
This patient only
Grant + audit event written in the same transaction
Renewable with a new reason

5 Summary in < 1 s

International Patient Summary: allergies, active problems, medications, blood group, implants, advance directives, contacts
Full record: same grant

Clinician: ≈ 30 seconds from search to data (human steps) · System: p95 < 300 ms per API call · Availability 99.99 %

GUARDRAILS — ACCOUNTABILITY AFTER THE FACT (access is never blocked, always answered for)

Immutable audit

AuditEvent, purpose = emergency treatment, kept in the WORM trail

Real-time alert

Privacy office alerted through the SIEM the moment access starts

Patient informed

In-app + e-mail notice within 24 h: who, when, what was viewed

Mandatory review

Privacy officer + medical lead review every case within 72 h

Misuse detection

Behaviour analytics flag outliers; sanctions and account suspension

RESILIENCE — WHY THE TIER-0 PATH KEEPS WORKING WHEN OTHER SERVICES ARE DEGRADED

Isolated deployment

bulkhead with reserved capacity

Pre-computed summary

projection, no joins at read time

Multi-AZ replicas

+ DR region serving reads (active)

Local token validation

cached IdP signing keys

Local policy decisions

OPA sidecar, cached bundles

Audit via outbox

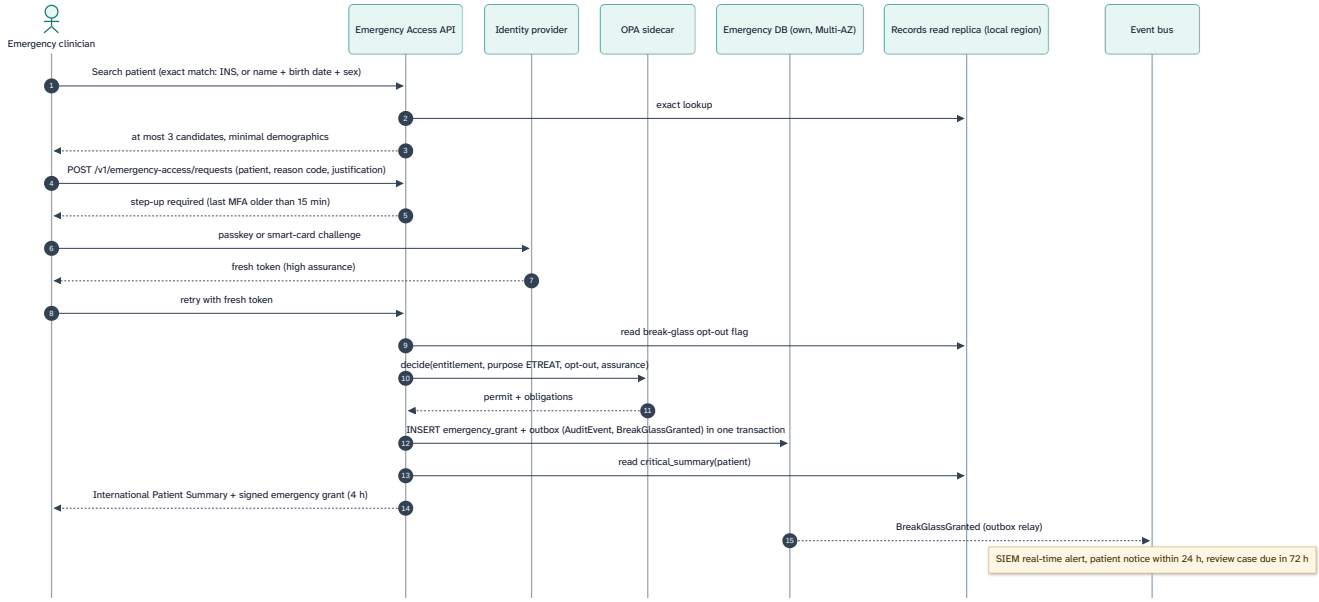
no dependency on the audit pipeline

Who can break the glass?

Verified practitioners holding the emergency_access entitlement (emergency medicine, on-call, EMS), granted by the MedX medical lead and reviewed quarterly. Patients may opt out where the law allows it.

Emergency access flow

Principle — in an emergency, *access is never blocked by process; it is always answered for afterwards*. Pre-approval workflows are too slow when a patient is unconscious; MedX grants immediate, narrow, time-boxed access and makes misuse highly visible and sanctioned.



Aspect	Design
Who	Verified practitioners holding the `emergency_access` entitlement (emergency medicine, on-call, EMS), granted by the MedX medical lead and re-certified quarterly.
Identification	Exact-match search only (national health identifier, MedX ID, or name + birth date + sex) returning at most three candidates with minimal demographics — no browsing, no wildcard, rate-limited.
Declaration	Coded reason (unconscious patient, life-threatening situation, patient unable to communicate…) + free-text justification + accountability statement.
Authentication	Step-up MFA if the last strong authentication is older than 15 minutes (passkey or professional smart card).
Grant	Read-only, this patient only, **4 hours**, renewable with a new justification; issued as a **signed emergency grant** (JWT bound to the clinician's key) that the regular Records API also accepts for the full record.
Data returned	**International Patient Summary**: allergies & intolerances, active problems, current medications, immunisations, implants & devices, past procedures, blood group, pregnancy status, advance directives, emergency contacts. The full record is available under the same grant, restricted notes excluded.
Patient choice	Patients can opt out of break-glass where the law allows it (warned about the risk); where it does not, the flag is ignored.
Accountability	AuditEvent with `purposeOfUse = ETREAT` written **in the same transaction** as the grant (no access without audit); real-time SIEM alert to the privacy office; patient notified (in-app + e-mail) within 24 h with who/when/what; **mandatory review within 72 h** by the privacy officer and a medical lead; unjustified use → suspension and referral to the professional body; behaviour analytics flag outliers.

Why it keeps working when the rest is degraded (Tier 0)

Failure	Effect on emergency access
Records primary database down	None — summaries are read from replicas; grants are written in the Emergency Access service's own database.
Primary region lost	None beyond DNS failover — the Emergency Access API is **active in the DR region** with its own grant database and local replicas (lag < 1 s).
Event bus / audit pipeline down	None — audit events wait in the outbox and are delivered later; nothing is lost.

Failure	Effect on emergency access
Policy bundle server down	None — OPA sidecars keep the last signed bundle.
Identity provider slow	Clinicians with a recent strong authentication proceed; tokens are validated locally with cached keys. The IdP itself is active in both regions.
Traffic spike on the rest of MedX	None — separate deployment with reserved capacity (bulkhead) and its own rate limits.

Performance — the critical summary is a pre-computed projection refreshed within seconds of any change (event-driven), stored as one encrypted row per patient: one indexed read from the nearest replica, one cached key unwrap, one decryption → **p95 < 300 ms**.

API (selection)

Method	Endpoint	Purpose
`GET`	`/fhir/Patient/{id}/$summary`	IPS summary (patient, care team)
`GET`	`/fhir/{type}?patient={id}&…`	FHIR search
`POST` / `PUT`	`/fhir/{type}` · `/fhir/{type}/{id}`	Create / update (`If-Match` required)
`GET`	`/fhir/{type}/{id}/_history`	Version history
`POST`	`/fhir/Composition/{id}/$sign`	Sign a clinical note
`GET`	`/fhir/Patient/{id}/$everything`	Full export (right of access / portability) — FHIR bundle or PDF
`POST`	`/v1/emergency-access/requests`	Break-glass request → grant + critical summary
`GET`	`/v1/patients/me/access-log`	"Who accessed my record" (from the audit trail)
`PUT`	`/v1/patients/me/consents/{type}`	Sharing, research, break-glass opt-out

Events

- **Publishes** `records.record.v1` (RecordUpdated, DocumentAdded, ResultReleased, ConsentChanged) and `records.emergency.v1` (BreakGlassGranted, BreakGlassReviewDue, BreakGlassReviewed).
- **Consumes** `identity.user.v1` (create patient record and data keys; revoke grants of suspended practitioners), `scheduling.appointment.v1` (create/revoke care grants), `consultation.session.v1` (Encounter, chat transcript archive), `prescription.v1` (medication list and summary refresh).

Data owned

PostgreSQL `records` database: `resource` (current versions, JSONB, sensitive elements encrypted), `resource_history` (append-only, hash chain), search-parameter indexes, `care_grant`, `consent`, `critical_summary`, `patient_key` (wrapped data keys), `outbox`. Emergency Access has its own small database (`emergency_grant`, `review_case`, `outbox`). Binaries in the Document Store. Details in §3.4.

Scalability & resilience

- **Read-heavy** (≈ 5–10 reads per write; ≈ 600 reads/s at Year-3 peak): a synchronous standby and two read replicas across the three AZs, plus replicas in the DR region; read-your-writes guaranteed for the author by routing their reads to the primary for a few seconds after a write.

- UI screens read the **patient timeline projection** (one query per screen) instead of assembling many FHIR searches.
- Growth path: table partitioning by patient hash from day one; if write throughput ever exceeds one primary, **shard by** `patient_id` (e.g., Citus) — almost every query is patient-scoped, so shards stay independent.
- No PHI in shared caches; the only cache of clinical data is the encrypted critical summary.

Design decisions & trade-offs

Decision	Alternative	Why this choice
FHIR R4 as the internal model (HAPI FHIR on PostgreSQL)	Custom relational schema	Interoperability (national services, hospitals), built-in validation, versioning and search; hiring and audit are easier. Cost: generic storage is less efficient → projections for UI screens.
Append-only history + hash chain + anchoring	In-place updates with an audit table	Integrity is a High priority: history cannot be rewritten silently, even by an administrator. Cost: storage growth (≈ +30 %), acceptable.
Break-glass with post-hoc review	Pre-approval by a supervisor	Minutes matter in an emergency; deterrence (audit, alerts, notice to patient, review, sanctions) controls misuse.
Per-patient encryption keys	Database encryption only	A leaked backup, replica or query result is useless without KMS; crypto-shredding at end of retention. Cost: key management and ~1 ms per request with key caching.
Separate Emergency Access deployment and database	Endpoint of the main Records API	Isolation from load and failures of the main API; can run active in two regions. Cost: one more deployable unit.

2.5 ⑤ Prescription Management Service

Priority	Zone	Criticality	Owns
Medium — M3 prescription management	Clinical	Tier 1	Prescriptions and their versions, signatures, safety-check results, fulfilment status, renewal requests

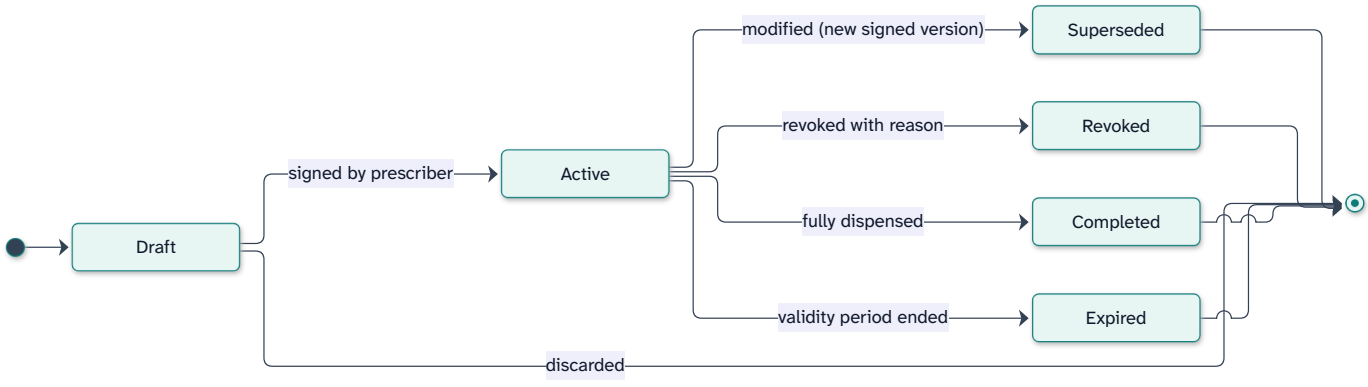
Mission — turn a doctor's decision into a **safe, signed, verifiable** electronic prescription that reaches the right pharmacy in seconds, can be corrected or revoked instantly, and whose progress the patient can follow.

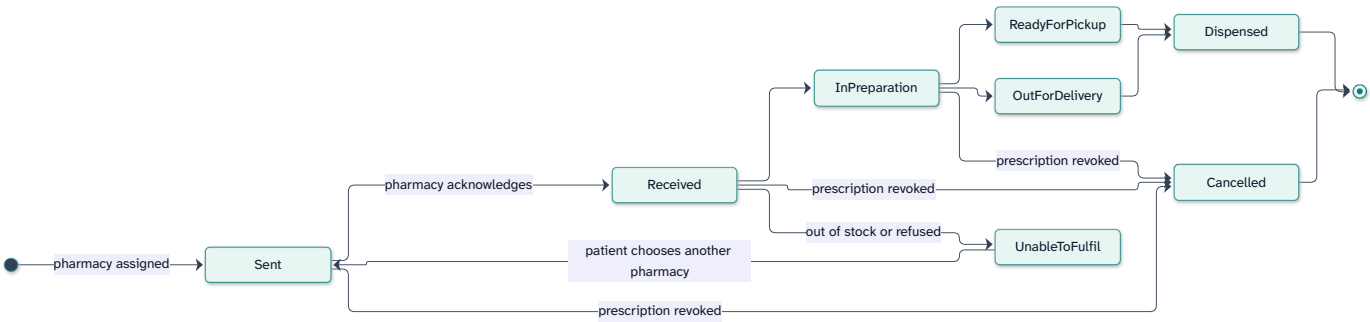
Responsibilities

Appendix 1 feature	How MedX delivers it
Doctors create, modify or revoke prescriptions post-consultation	Draft within an encounter → automatic **safety checks** → **e-signature** (step-up MFA) → active. Modification creates a new signed version that supersedes the previous one; revocation is a signed status change with a reason.
Notify pharmacies of new prescriptions or changes	`PrescriptionIssued / Modified / Revoked` events → Pharmacy Interface pushes to the pharmacy inbox and PMS webhook; revocations are **critical** (multi-channel, escalated until acknowledged).
Enable patients to view active prescriptions	Patient API and app screen: active prescriptions, remaining renewals, validity dates, printable PDF with QR code.
Track prescription fulfilment status	Fulfilment state machine fed by pharmacy updates; each change is visible to the patient and the prescriber.

Prescription and fulfilment lifecycles

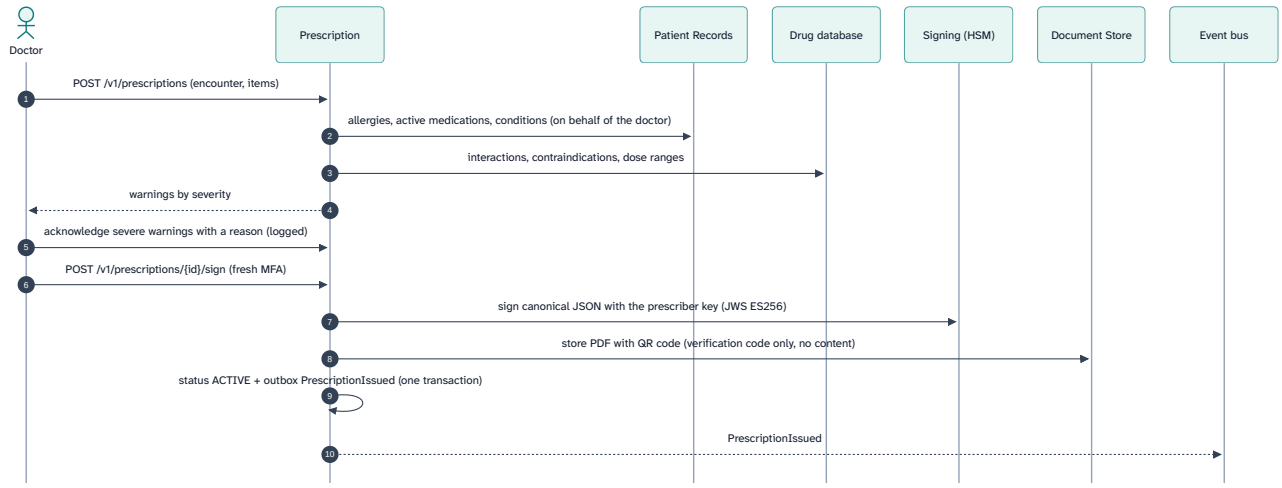
The **legal status** of the prescription and the **fulfilment status** at the pharmacy are two separate state machines — mixing them is a classic source of errors (e.g., "ready for pickup" on a revoked prescription).





Partial dispensing and renewals are tracked per line item (`dispensed_quantity` , `refills_remaining`).

Creating and signing a prescription



- **Safety checks (clinical decision support)**: drug–drug interactions, allergy and cross-allergy, duplicate therapy, contraindications (pregnancy, age, renal function when known), dose range. The call to Records uses **token exchange** (RFC 8693) so the request is authorised as the doctor, not as a privileged service. If a check is unavailable, the doctor is told so explicitly and must acknowledge before signing.
- **Signature**: the canonical prescription (RFC 8785 JSON) is signed with the prescriber's key, held in the HSM and usable only after a fresh MFA; alternatively a qualified signature from a trust service provider or the professional smart card (eIDAS). The signed version is **immutable**.
- **QR code / verification code**: lets a pharmacy verify authenticity and **current status** (not revoked, not already dispensed, not expired) — it does not contain the prescription content.

Modify and revoke

Action	Rules	Pharmacy side
Modify	Only if not fully dispensed. Creates version *n+1* (new signature) referencing version *n*, which becomes *Superseded*; dispensed quantities stay valid.	`PrescriptionModified` (high priority); if preparation had started, the pharmacist must acknowledge the change before continuing.
Revoke	Reason required (error, adverse reaction, change of treatment). Signed status change.	`PrescriptionRevoked` (**critical**): portal alert + PMS webhook; if not acknowledged within 15 min, SMS/phone escalation to the pharmacy. If already dispensed, the prescriber is alerted to contact the patient.
Renew	Patient requests a renewal from the app → task for the prescriber → new prescription linked to the previous one, or refusal with a message.	New `PrescriptionIssued` .

Controlled substances follow stricter, configurable rules per jurisdiction (secure prescription requirements, maximum durations, no remote renewal, additional identity checks at the pharmacy).

API (selection)

Method	Endpoint	Purpose
`POST`	`/v1/prescriptions`	Create a draft (encounter, items)

Method	Endpoint	Purpose
`POST`	`/v1/prescriptions/{id}/checks`	Run / re-run safety checks
`POST`	`/v1/prescriptions/{id}/sign`	Sign (step-up MFA) → Active
`POST`	`/v1/prescriptions/{id}/versions`	Modify → new signed version
`POST`	`/v1/prescriptions/{id}/revoke`	Revoke with reason
`PUT`	`/v1/prescriptions/{id}/pharmacy`	Patient assigns or changes the pharmacy
`GET`	`/v1/patients/me/prescriptions?status=active`	Patient view
`GET`	`/v1/prescriptions/{id}/fulfilment`	Fulfilment status timeline
`GET`	`/v1/prescriptions/verify?code=`	Pharmacy verification (authenticity + status)
`GET`	`/fhir/MedicationRequest?patient=`	Interoperability view

Events

- **Publishes** `prescription.v1`: PrescriptionIssued, PrescriptionModified, PrescriptionRevoked, PrescriptionExpired, PharmacyAssigned, FulfilmentStatusChanged.
- **Consumes** `pharmacy.fulfilment.v1` (acknowledgements and status updates, idempotent by sequence number), `identity.user.v1` (a suspended prescriber can no longer sign; active prescriptions are flagged for review).

Data owned

PostgreSQL: `prescription`, `prescription_version` (signed, immutable), `prescription_item` (drug code, structured dosage, quantity, refills, substitution allowed), `signature`, `status_history` (append-only), `fulfilment`, `safety_check` (warnings and overrides with reason), `renewal_request`, `outbox`. Drug reference data is a read-only replica refreshed from the drug database (public reference data, not PHI). Details in §3.4.

Security & privacy

- Signing requires a fresh strong authentication; signing keys never leave the HSM.
- **Minimum necessary for pharmacies**: patient name, birth date, (insurance number where required), items, prescriber identity and signature — never diagnosis or notes.
- Every read by a pharmacy is audited; verification endpoint rate-limited against enumeration of codes.

Scalability & resilience

- 120 k prescriptions/day at Year 3 (≈ 1.4/s average, ≈ 15/s peak) plus ≈ 5 status updates each — light load; the service scales horizontally and the drug search is cached.
- Revocation delivery is designed for **safety, not throughput**: outbox (no lost event), push + webhook + escalation, and a reconciliation job that compares prescription and pharmacy states every 5 minutes.

Design decisions & trade-offs

Decision	Alternative	Why this choice
Modification = new signed version	Edit in place	Legal traceability: the pharmacy and the auditor can always see what was signed, when, and what replaced it.

Decision	Alternative	Why this choice
Separate legal status and fulfilment status	One status field	Prevents contradictory states; each has a single owner (prescriber vs pharmacy).
FHIR `MedicationRequest` / `MedicationDispense` / `Task` mapping	Proprietary format	Direct path to national e-prescription services: where such a service exists (e.g., France's *ordonnance numérique*), an adapter registers each prescription there and the national service becomes the reference for dispensing.
Allergy check via Records at signing time	Local copy of allergies	Always current data for a safety decision; failure is explicit, never silent.

2.6 ⑥ Pharmacy Interface Service

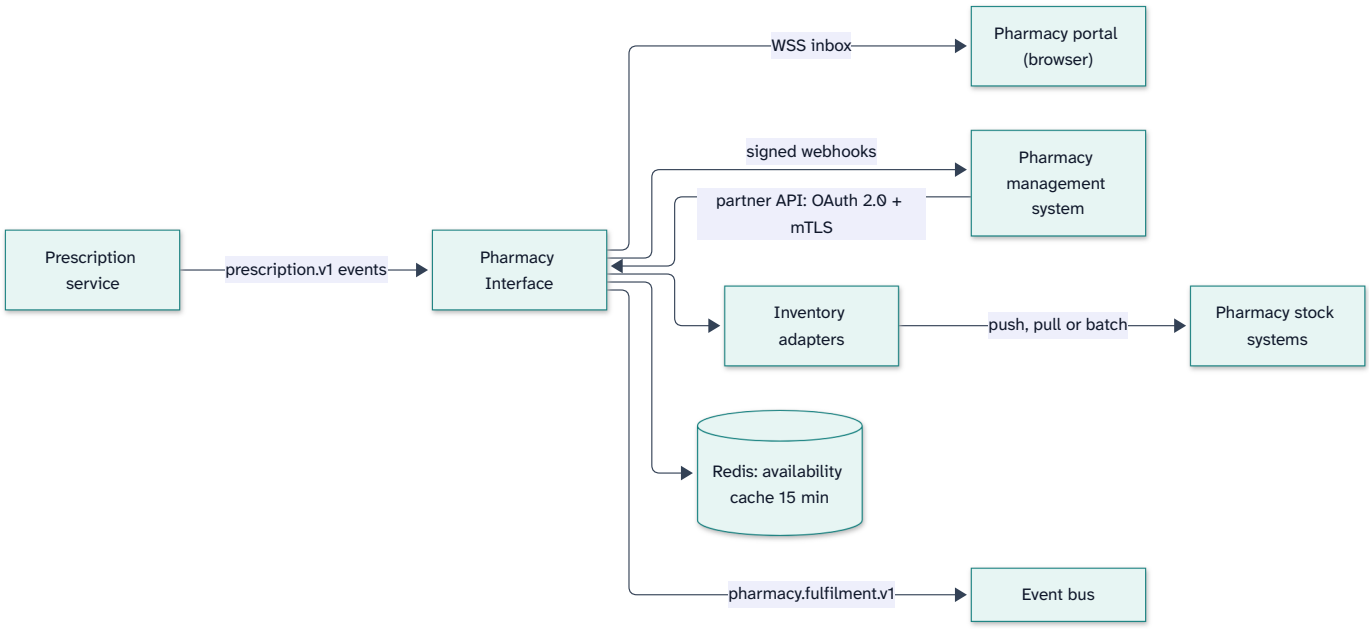
Priority	Zone	Criticality	Owns
Low — L1 pharmacy interface	Clinical (minimised prescription data)	Tier 2	Pharmacy registry, integrations, fulfilment tasks, patient–pharmacy conversations, inventory availability cache

Mission — connect every pharmacy, from the independent shop with a web browser to the chain with its own pharmacy management system (PMS), to MedX prescriptions — reliably and with the minimum patient data.

Responsibilities

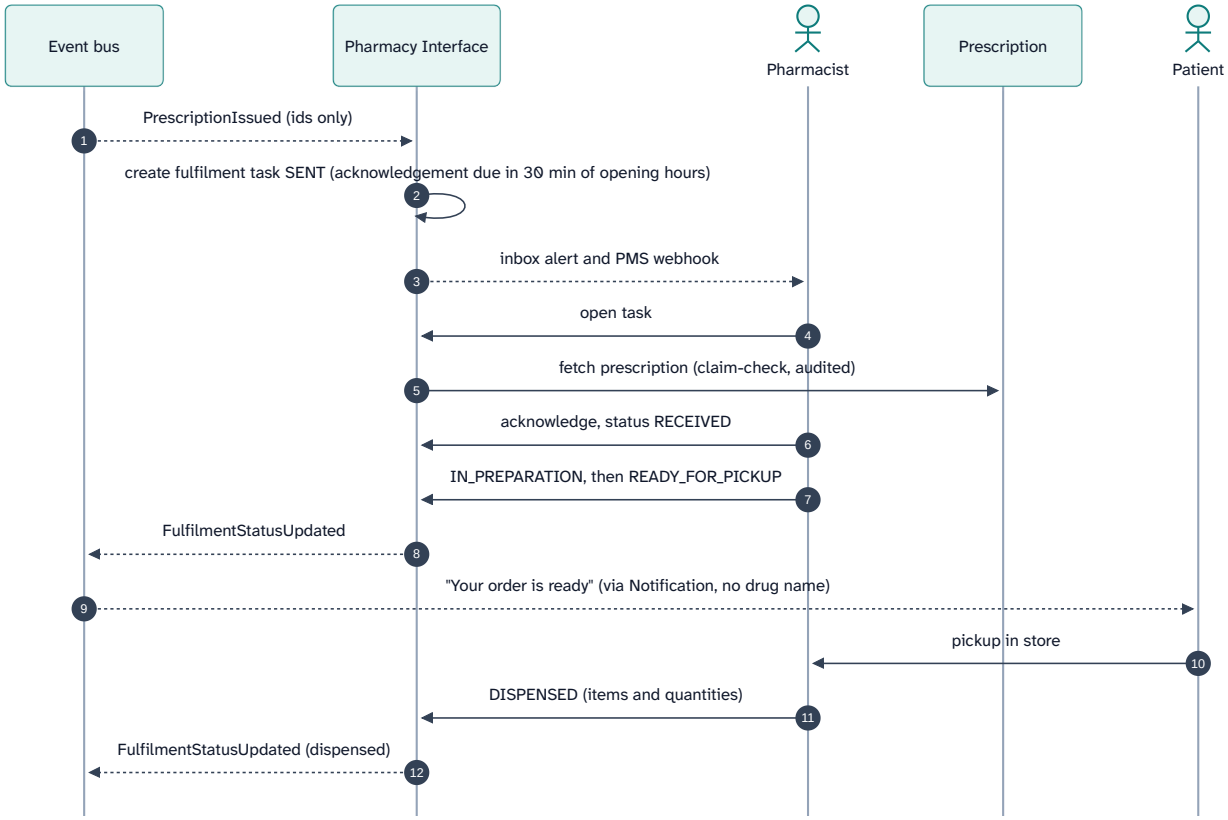
Appendix 1 feature	How MedX delivers it
Confirm prescription receipt and update statuses ("Ready for pickup", "Dispatched"…)	Real-time **inbox** (portal) or **webhook + partner API** (PMS); acknowledgement with SLA tracking; status updates validated against the fulfilment state machine and forwarded to the Prescription service.
Interaction between patients and pharmacies	Secure **conversation threads** attached to a prescription (availability questions, substitution proposals, pickup times), with notifications on both sides and message templates.
Integrate with inventory systems	**Inventory adapters** per PMS vendor behind a common availability port; cached availability levels; "find a pharmacy that has my medicine nearby".

Integration modes



Mode	For whom	Details
Pharmacy portal	Every pharmacy, no integration needed	Web app with real-time inbox, prescription view and printing, QR-code scan (webcam) for verification, status buttons, conversations.
Partner API + webhooks	Pharmacies whose PMS vendor integrates	REST and FHIR (`Task` , `MedicationDispense`) API; OAuth 2.0 client credentials with **mTLS**-bound tokens; outbound webhooks **signed (HMAC-SHA256 + timestamp)** with replay protection, retries with exponential backoff for 24 h, then dead-letter and portal fallback.
Inventory adapters	Pharmacies sharing stock data	Common port `checkAvailability(pharmacy, items)` returning IN_STOCK, LOW, OUT_OF_STOCK or UNKNOWN; strategies: push of stock deltas, real-time pull (2–3 s timeout) or nightly batch, depending on the vendor.

Fulfilment flow



- **Unable to fulfil** (out of stock, substitution refused): the patient is notified and offered nearby pharmacies that report the medicine in stock; reassignment restarts the flow.
- **Unacknowledged prescriptions**: if the SLA is missed, the patient is told and can choose another pharmacy.
- **Status updates are idempotent**: each update carries a sequence number; duplicates and out-of-order webhook retries are ignored.

Data minimisation

A pharmacy only ever sees prescriptions **assigned to it** (or presented by the patient via the QR code), and only the fields needed to dispense: patient name, birth date, insurance number where required, items, prescriber identity and signature. **No diagnosis, no consultation notes, no record access.** Inventory data is exposed to patients as availability levels, not quantities (commercial confidentiality).

API (selection)

Method	Endpoint	Purpose
`GET`	`/v1/pharmacy/inbox?status=`	Pharmacy inbox
`POST`	`/v1/pharmacy/fulfilments/{id}/acknowledge`	Confirm receipt
`POST`	`/v1/pharmacy/fulfilments/{id}/status`	Update status (sequence number, idempotent)
`POST`	`/v1/pharmacy/fulfilments/{id}/dispense`	Record dispensed items and quantities
`GET`	`/v1/pharmacies/search?lat=&lon=&item=`	Nearby pharmacies with availability (patient)
`POST`	`/v1/conversations` · `/v1/conversations/{id}/messages`	Patient–pharmacy conversation
`GET`	`/partner/fhir/Task?owner={pharmacy}&status=requested`	PMS pull mode

Method	Endpoint	Purpose
`POST`	`/partner/v1/fulfilments/{id}/status`	PMS status push

Events

- **Publishes** `pharmacy.fulfilment.v1` : FulfilmentAcknowledged, FulfilmentStatusUpdated, UnableToFulfil, InquiryMessagePosted.
- **Consumes** `prescription.v1` : Issued, Modified, Revoked, PharmacyAssigned.

Data owned

PostgreSQL: `pharmacy` (registry number, address, geolocation, opening hours, services), `integration` (mode, endpoint, credential references in Vault), `fulfilment_task` , `status_event` (append-only), `conversation` , `message` (encrypted), `outbox` . Redis: availability cache. Geo search with PostGIS. Details in §3.4.

Scalability & resilience

- 15 k pharmacies, ≈ 600 k status updates/day at Year 3 (≈ 7/s): light; webhook delivery workers scale on queue lag.
- A slow or failing PMS never blocks MedX: webhooks are asynchronous with retries and dead-letter, and the portal is always available as a fallback.
- Inventory queries are cached (15 min) and time-boxed; "unknown" is an acceptable answer.

Design decisions & trade-offs

Decision	Alternative	Why this choice
Portal **and** API	API only	Inclusion: small pharmacies can join on day one; integrated ones get automation. Cost: two channels to maintain.
Adapter per PMS vendor behind one port	Force one format on all vendors	Vendors integrate at their own pace; MedX's core stays clean (anti-corruption layer).
Availability levels, not quantities	Exact stock	Enough for the patient's decision; protects pharmacies' commercial data.
Low priority, delivered after the clinical core	Early delivery	Follows MedX priorities; until then, prescriptions are printable with a verifiable QR code usable in any pharmacy.

2.7 ⑦ Notification Service

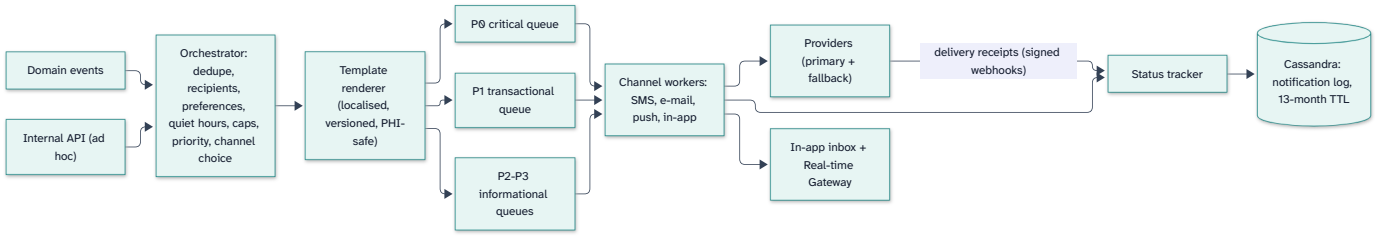
Priority	Zone	Criticality	Owns
Low — L2 preferences (applied) · L3 delivery & read tracking — **a minimal version ships with the Medium features** (booking confirmations, reminders)	Engagement (no PHI)	Tier 2 (critical messages have their own lane)	Templates, local replica of preferences, notification log, delivery statuses, in-app inbox

Mission — tell the right person the right thing on the right channel at the right time, **without ever putting health information in an SMS, an e-mail or a push banner**.

Responsibilities

Appendix 1 feature	How MedX delivers it
Send alerts, reminders, updates (SMS, e-mail, in-app)	Event-driven pipeline with channel workers for **SMS, e-mail, push (APNs/FCM) and in-app** (Real-time Gateway + inbox); two providers per external channel with automatic failover.
Handle notification preferences (opt-in/opt-out, frequency)	Preferences owned by User Management and replicated locally from `PreferencesUpdated`; evaluated per category × channel, with quiet hours, frequency caps and digests.
Track delivery and read status	Status machine fed by provider receipts and client read receipts, stored in an append-only notification log; bounce handling; delivery dashboards.

Pipeline



Priority lanes

Lane	Examples	Behaviour	Target
P0 critical	Prescription revoked (to the pharmacy), urgent consultation assigned (to a doctor), break-glass alert (to the privacy office)	Ignores quiet hours; several channels in parallel; escalation if not read (e.g., SMS after 5 min, then phone)	dispatched < 10 s
P1 transactional	Booking confirmation or change, reminders, "medicine ready", payment receipt	Mandatory category (channel can be chosen, not disabled)	dispatched < 60 s
P2 informational	New document or message available	Fully controlled by preferences; digest possible	< 5 min
P3 marketing	Newsletter, surveys	**Explicit opt-in only** (GDPR/ePrivacy), frequency caps	batch

Each lane has its own Kafka topic and worker pool, so a burst of reminders can never delay a revocation alert.

PHI-safe content

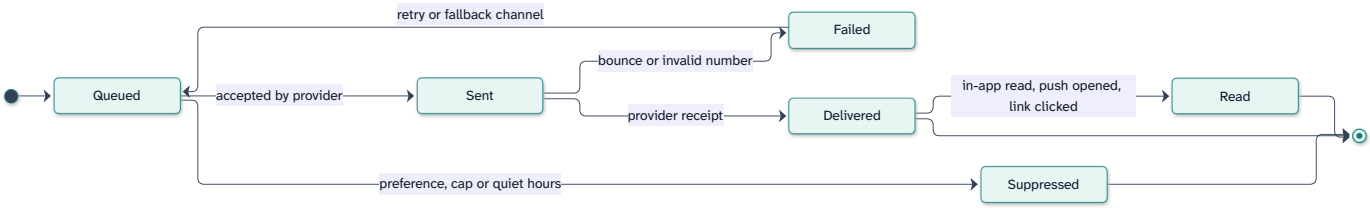
Channel	Example of what MedX sends	What it never contains
SMS	"MedX: your teleconsultation is tomorrow at 10:30. Join from the app."	Doctor name or specialty, reason, drug names, results
E-mail	Generic subject, deep link to the app, sender authentication (SPF, DKIM, DMARC)	Attachments with health data (invoices and documents are downloaded in the app)
Push	"You have a new message"	Message content on the lock screen
In-app	Full details after authentication	—

Templates can only use **allow-listed variables**; a CI check rejects any template referencing a clinical field.

Preferences

- Owned and edited in User Management (single source of truth); replicated here through `PreferencesUpdated` events into a local table and a Redis cache → no synchronous dependency at send time.
- Evaluated per **category × channel** (appointments, prescriptions, documents, messages, marketing), with quiet hours in the user's time zone, frequency caps (e.g., max 3 non-critical SMS per day) and daily digests.
- Mandatory notifications (security, legal, P0/P1) cannot be switched off entirely, only routed.

Delivery and read tracking (L3)



- Status events are appended (never updated in place) with provider references and timestamps.
- **No tracking pixels in e-mails** (privacy): "read" for e-mail means a link was clicked or the item was opened in the app.
- Hard bounces mark the contact point invalid and emit `ContactPointInvalid` so the user is asked to update it.
- Dashboards: delivery rate per channel and provider, latency per lane; alert when a provider degrades (automatic failover to the second provider).

Reliability

- **Idempotency**: deduplication key = event ID + recipient + channel (Redis, 24 h) — replays never double-send.
- Retries with exponential backoff and jitter, provider failover, dead-letter queue with operator replay.
- Reminder batches (e.g., all 9:00 appointments) are spread with jitter across the sending window.

API (selection)

Method	Endpoint	Purpose
`POST`	`/v1/notifications`	Internal ad hoc send (service-to-service only)
`GET`	`/v1/users/me/notifications?cursor=`	In-app inbox
`POST`	`/v1/users/me/notifications/{id}/read`	Read receipt
`POST`	`/v1/webhooks/providers/{provider}`	Delivery receipts (signature verified)
`GET`	`/v1/admin/notifications/metrics`	Delivery statistics (aggregated, no content)

Events

- **Consumes** almost every business topic (see the event catalogue) and `identity.preferences.v1`.
- **Publishes** `notification.delivery.v1`: NotificationSent, NotificationDelivered, NotificationRead, NotificationFailed, ContactPointInvalid.

Data owned

PostgreSQL: `template` (versioned, per locale), `preference_replica`, `provider_config`. Cassandra: `notification_by_recipient` (partition = recipient + month), `notification_status_event`, TTL 13 months. Redis: deduplication keys, frequency counters, rate limits. Details in §3.4.

Scalability & resilience

- 2 M notifications/day at Year 3 (≈ 25/s average, **≈ 500/s at reminder peaks**): Kafka absorbs bursts; workers autoscale on consumer lag per lane; providers are rate-limited per account.
- Cassandra handles the write-heavy log (≈ 4 status events per notification) linearly, with TTL-based expiry instead of costly deletes.

Design decisions & trade-offs

Decision	Alternative	Why this choice
Generic, PHI-free external messages	Rich messages ("your dermatology appointment...")	SMS and e-mail are neither private nor within the HDS scope; convenience is kept by deep links into the authenticated app.
Wide-column store (Cassandra) for the log	PostgreSQL	Append-heavy time series queried by recipient, with TTL expiry: a natural fit; PostgreSQL stays for templates and preferences.
Build the orchestrator, buy the channels	Notification SaaS for everything	Preference, PHI and priority rules are MedX-specific and sensitive; message delivery is a commodity.
Separate lanes per priority	One queue	Guarantees that critical safety messages are never stuck behind bulk reminders.

2.8 ⑧ Billing & Payment Service

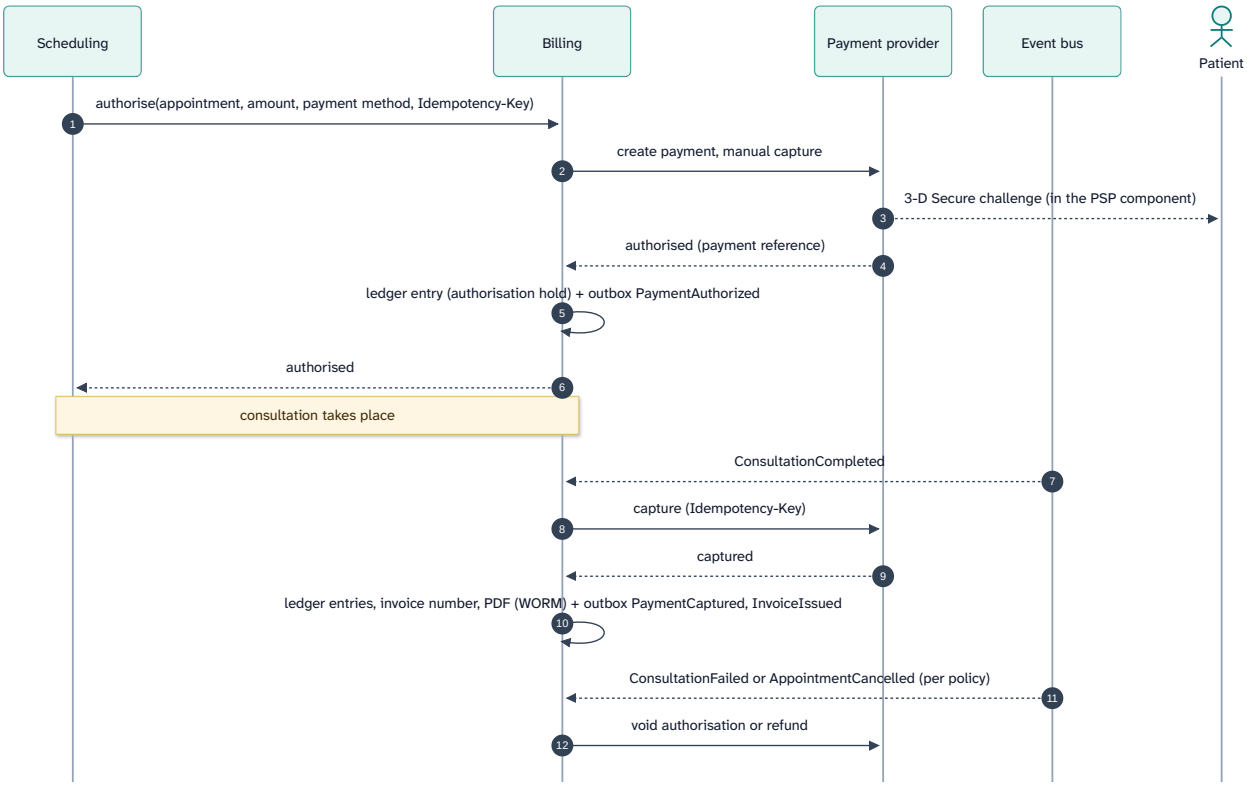
Priority	Zone	Criticality	Owns
Low — L5 secure payment · L6 refunds	Payment (PCI scope minimised)	Tier 2	Prices, payments (PSP references), double-entry ledger, invoices and credit notes, refunds, payouts

Mission — charge the right amount at the right time, never touch a card number, keep books that reconcile to the cent, and refund quickly and traceably when something goes wrong.

Responsibilities

Appendix 1 feature	How MedX delivers it
Handle consultation fees and process payments	Price per practitioner and consultation type; **pre-authorisation at booking, capture after the consultation**; cancellation policy; marketplace payouts to practitioners and pharmacies through the PSP.
Generate invoices for consultations and medication	Invoice issued on capture (on behalf of the practitioner or pharmacy): gapless numbering per issuer, immutable PDF stored WORM, **no clinical detail** on the invoice.
Process refunds if necessary	Automatic (doctor cancels, consultation failed technically) or manual (support within a threshold, 4-eyes above it); credit notes; full ledger trace.
Provide secure transaction mechanisms	**PSP hosted payment fields** (card data never reaches MedX) → PCI DSS **SAQ A**; 3-D Secure 2 (PSD2 strong customer authentication); tokenised payment methods; signed webhooks; idempotency everywhere.

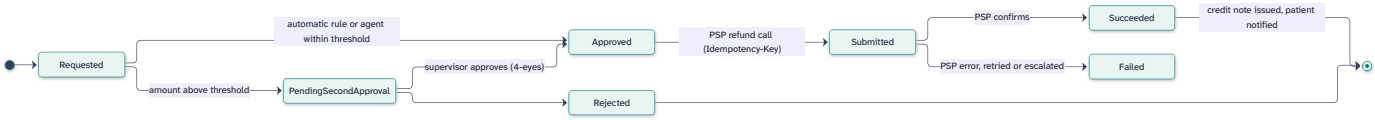
Payment lifecycle



- **Authorisation expiry** — card authorisations expire after about 7 days, so for appointments booked further ahead the card is saved with strong authentication (setup intent) and authorised automatically 24 h before.
- **Cancellation policy** — free until 24 h before; late cancellation or no-show fee as disclosed at booking; **doctor cancellation or technical failure → automatic full release/refund**.

- **Medication charges** — when a pharmacy offers delivery or click-and-collect with online payment, the order is paid through the same flow; the invoice is issued on behalf of the pharmacy.
- **Payouts** — marketplace accounts at the PSP split each payment between the practitioner or pharmacy and the MedX fee; the PSP performs the payees' identity checks (KYC).

Refunds (L6)



Ledger and reconciliation

- **Double-entry, append-only ledger**: every money movement is a balanced journal entry (patient receivable, practitioner/pharmacy payable, MedX revenue, PSP clearing, refunds). Entries are never updated — corrections are new entries.
- **Idempotency**: every PSP call carries an idempotency key; every PSP webhook is de-duplicated by event ID and its raw payload kept for audit.
- **Daily reconciliation** with the PSP settlement report; any difference opens an incident.
- **Invoices** are immutable; a refund produces a **credit note**, never an edited invoice. Retention 10 years (commercial law), WORM storage.

PCI DSS scope minimisation

Measure	Effect
PSP-hosted fields / drop-in components (iframes) in the apps	Card numbers go from the device straight to the PSP; MedX stores only tokens, brand and last 4 digits.
Payment page script integrity (CSP, SRI, change detection — PCI DSS v4 req. 6.4.3 and 11.6.1)	Protects the page that embeds the PSP component against script injection (e-skimming).
Restricted PSP API keys in Vault, rotated	A leaked key cannot read card data or move funds beyond its permissions.
Payment namespace isolated by network policy	Only Scheduling, the gateway (webhooks) and the event bus can reach Billing.
No PHI in billing	Invoices say "Teleconsultation", never a specialty or a diagnosis.

API (selection)

Method	Endpoint	Purpose
`POST`	`/v1/payment-methods/setup`	Returns a PSP client secret for the hosted fields
`POST`	`/v1/payments/authorizations`	Internal (Scheduling): authorise a fee
`POST`	`/v1/payments/{id}/capture`	Internal: capture after the consultation
`POST`	`/v1/refunds`	Request a refund (automatic rules, support agent)
`POST`	`/v1/refunds/{id}/approve`	Second approval above threshold
`GET`	`/v1/users/me/invoices` · `/v1/invoices/{id}/pdf`	Patient invoices
`POST`	`/v1/webhooks/psp`	PSP events (signature verified, deduplicated)

Events

- **Publishes** `billing.payment.v1` : PaymentAuthorized, PaymentFailed, PaymentCaptured, PaymentVoided, RefundSucceeded, RefundFailed, InvoiceIssued.
- **Consumes** `consultation.session.v1` (capture or refund), `scheduling.appointment.v1` (cancellations, no-shows), `pharmacy.fulfilment.v1` (paid medication orders).

Data owned

PostgreSQL: `customer` (PSP customer reference), `payment_method` (token, brand, last 4), `payment` , `ledger_account` , `journal_entry` + `journal_line` (append-only), `invoice` + `invoice_line` , `credit_note` , `refund` , `psp_event` (raw, deduplication), `payout` , `outbox` . Amounts stored as **integers in minor units + ISO 4217 currency**. Invoice PDFs in the Document Store (WORM). Details in §3.4.

Scalability & resilience

- ≈ 200 k payments/day at Year 3 (≈ 2–3/s): throughput is not the challenge, **correctness** is — hence idempotency, outbox, ledger and reconciliation.
- If the PSP is unavailable, bookings can proceed in "pay later" mode (configurable by risk), captured when the PSP recovers; no request is lost thanks to the outbox and retries.

Design decisions & trade-offs

Decision	Alternative	Why this choice
PSP hosted fields (SAQ A)	Handle card data (SAQ D)	Removes the heaviest compliance scope and the most attractive target. Cost: less control of the payment UI.
Own double-entry ledger	Rely on the PSP dashboard	Auditability, marketplace splits, refunds and reconciliation require MedX's own books.
Pre-authorise then capture	Charge at booking	Patient pays only for consultations that happened; fewer refunds. Cost: authorisation-expiry handling.
Low priority, but security by design from day one	Postpone security with the feature	Payment is delivered last, but its design already keeps card data out of MedX.

2.9 Platform Components

Shared capabilities used by every business service. Several of them carry **High-priority** requirements (H1 encryption, H4 authorization, H5 integrity and secure files), so they are built once, by the platform team, and reused everywhere.

Component	Priority served	Tier
API Gateway & edge	H1, H4	0
Real-time Gateway	M2	1
Policy Decision Point (OPA)	H4, M4	0
Event bus (Kafka) & outbox	H5	1
Secure Document Store	H1, H5	1
Audit Trail	H2, H3, H5	1 (writes buffered in outboxes)
Keys & secrets	H1	0
Observability & security monitoring	all	1
De-identified data platform	L4	2

API Gateway & edge

- **CDN** serves only static assets (app bundles, images); API responses containing personal data are sent with `Cache-Control: no-store` and never cached at the edge.
- **WAF** with OWASP core rules, bot management, geo/IP reputation, virtual patching; **DDoS protection** L3–L7.
- **API Gateway** (Envoy-based): TLS 1.3 termination (HSTS), routing by audience (`/patient` , `/clinician` , `/pharmacy` , `/partner` ; `/admin` is **not exposed publicly** — reachable only through the zero-trust access proxy), JWT validation for three issuers (realms) with cached JWKS, DPoP proof validation, mTLS for partners, per-user / per-IP / per-client **rate limits and quotas** (stricter on login, search and verification endpoints), request size limits, OpenAPI schema validation, CORS, security headers, correlation IDs, webhook ingress per provider with signature verification.
- Access logs contain metadata only (route, status, latency, pseudonymous subject) — never bodies or query strings with personal data.

Real-time Gateway

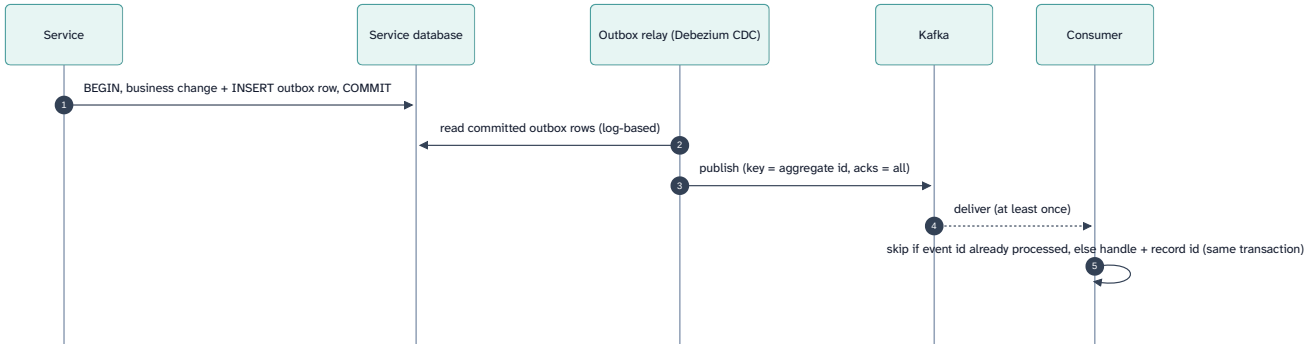
- WebSocket (WSS) connections for chat, presence, live statuses and in-app notifications — **one connection per app**, multiplexing channels.
- Authentication at connection time (access token); every channel subscription (`user:{id}` , `consultation:{id}` , `pharmacy:{id}`) is authorised by the owning service; connections are re-validated on token refresh and closed on revocation.
- Stateless fan-out through Redis pub/sub (or NATS): any node can deliver to any user; heartbeats, resume tokens after reconnect, connection draining on deployment.
- Sizing: 200 k concurrent connections at Year 3 peak ≈ 10 pods at ~25 k connections each, autoscaled on connection count.

Policy Decision Point (OPA)

- **Sidecar** next to every service: decisions in < 1 ms, no network hop, keeps working if the policy server is down.

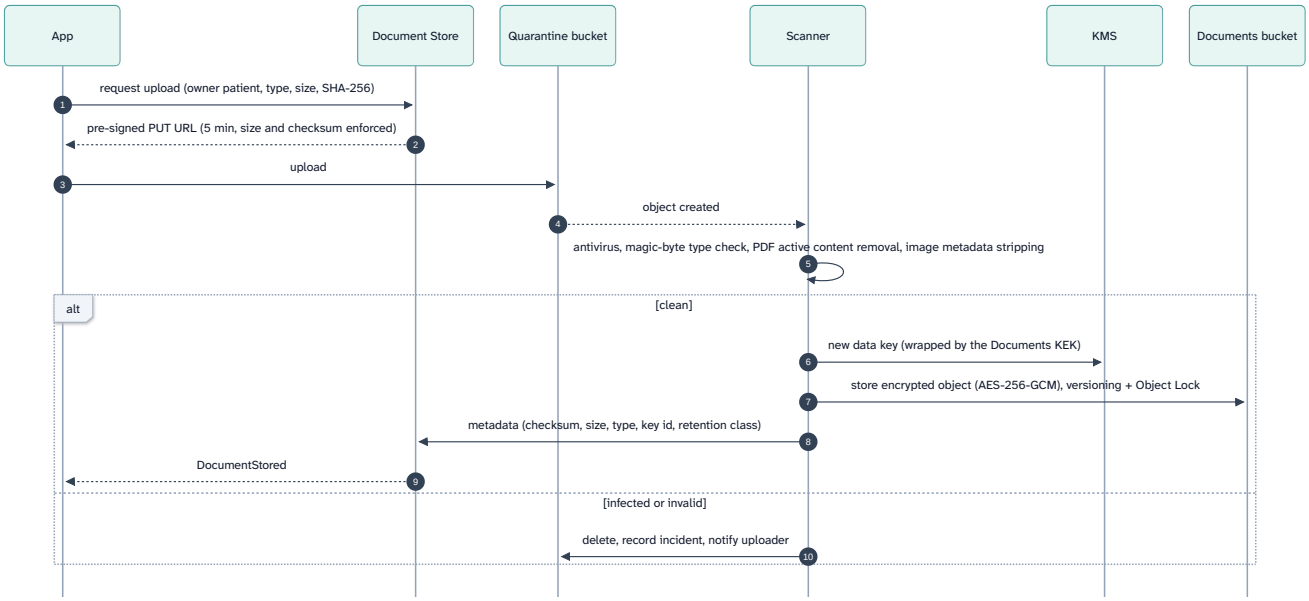
- Policies written in Rego, stored in Git, reviewed, **unit-tested in CI** (positive and negative cases for every rule), compiled into **signed bundles**, distributed to sidecars (pull every 60 s, last good bundle kept).
- Inputs: subject (from the validated token), action, resource (type, owner patient, security labels), purpose of use, and per-request **facts** looked up by the service (care grant, consent, emergency grant).
- Outputs: `allow` + **obligations** (masking, exclusions, audit level).
- Decision logs (without PHI) go to the audit pipeline for forensics.

Event bus (Kafka) & outbox



- **No dual writes**: an event exists if and only if the business change was committed.
- Kafka: 3 brokers over 3 AZs, replication factor 3, `min.insync.replicas = 2`, `acks = all`, idempotent producers; TLS + SASL/mTLS authentication, **ACLs per service** (a service can only produce its own topics and consume what it needs), encryption at rest.
- Schema registry with backward-compatibility checks in CI; breaking changes = new topic version.
- Retention 7 days (30 days for `audit.events.v1`); dead-letter topic per consumer group with alerting and replay.

Secure Document Store



- **Double encryption**: application-level AES-256-GCM with a per-file data key (wrapped by the Documents KEK and registered against the owner patient for crypto-shredding) **plus** bucket-level SSE-KMS.

- **Integrity**: SHA-256 computed by the client, verified at upload, re-verified at every download and by a weekly scrub job; object versioning; **Object Lock** (compliance mode) for documents under legal retention.
- **Access**: downloads go through the Document Store, which asks the owning service's policy (Records, Consultation, Billing) whether the caller may read this document; streamed decryption; every download audited. Quarantine objects are deleted after 24 h at the latest.
- **Limits**: 50 MB per file; allowed types PDF, JPEG, PNG, HEIC (DICOM imaging is out of scope for v1).
- **Durability**: object storage durability of 11 nines + cross-region replication + immutable backup copy.

Audit Trail

- **What is audited**: every read, creation, update, export, sharing, break-glass, authentication event and administrative action on personal data. Schema aligned with FHIR `AuditEvent` / IHE BALP: who (subject, role, organisation, assurance level, device, IP), what (action, resource type/ID, patient), when, why (**purpose of use**: treatment, emergency treatment, patient request, payment, operations, support), outcome, correlation ID.
- **Pipeline**: services write audit events to their **outbox in the same transaction** as the access → Kafka `audit.events.v1` (restricted ACL) → Audit service → (1) **WORM archive**: batches hash-chained, Merkle root signed by an HSM key and timestamped daily, stored with Object Lock in the **security account**; (2) **search index** (restricted OpenSearch, 13 months) for investigations and for the patient's "who accessed my record" page; (3) **SIEM** stream for real-time rules.
- **Who can read it**: privacy officers and auditors (read-only, themselves audited); patients see their own access history; no engineer can modify or delete audit data.
- **Retention**: at least 6 years, configurable up to the medical-record retention.

Keys & secrets

- **KMS backed by HSMs** (FIPS 140-3 level 3): key-encryption keys per service × environment × data class; usage restricted by IAM policy (the Billing role cannot decrypt Records keys); automatic yearly rotation; multi-region keys for DR; every use logged.
- **Envelope encryption** in the services (see §5.6): per-patient / per-file / per-consultation data keys, cached unwrapped in memory ≤ 5 min.
- **HashiCorp Vault**: dynamic database credentials (1 h TTL), Transit engine for encrypting integration secrets (calendar OAuth tokens, PMS credentials), PKI for internal certificates, secrets injected at runtime — **no secret in code, images or plain environment variables**; separation of duties between key custodians and data administrators.

Observability & security monitoring

- **OpenTelemetry** everywhere: traces (W3C `traceparent` propagated through HTTP and Kafka headers), metrics (RED for services, USE for resources), structured logs. Backends: Prometheus/Grafana, Loki, Tempo.
- **PHI never in telemetry**: log fields are allow-listed, identifiers tokenised, request bodies never logged; a scanner samples logs for PHI patterns and alerts.
- **SLOs and error budgets** per service tier; multi-window burn-rate alerts; **synthetic probes** call the emergency path every minute from both regions.
- **SIEM** correlation rules: break-glass events, mass record access, access without care grant attempts, impossible travel, MFA disabled, privilege escalation, anomalous exports; **user and entity behaviour analytics** to detect "curious insiders" (e.g., a doctor opening records of patients with no appointment, or of a celebrity).

De-identified data platform

- Purpose: product analytics, capacity planning, quality indicators, research (with consent), and **datasets for staging, demo and tests** (L4).
- Change-data-capture from service replicas into a **secure processing enclave** in the production account, where pseudonymisation/anonymisation jobs run; only de-identified outputs leave the enclave (columnar files queried by an analytics engine).
- Techniques and controls are described in §5.7: removal of direct identifiers, keyed pseudonyms, generalisation, date shifting, k-anonymity checks, DPO approval.
- Staging and demo use **synthetic patients** (e.g., generated with Synthea in FHIR format) — never real PHI.

3. Data Model & Storage Strategy

Deliverable 3. Synthetic analysis of the data model and of the storage chosen for each component. Component behaviour is described in 2. Detailed component designs.

3.1 Principles

1. **Database per service.** Each service owns its schema, credentials and migrations; no other service reads its tables. Cross-service references are **identifiers only** (UUIDv7), resolved through APIs or events.
2. **PostgreSQL by default.** Transactions, constraints, JSONB, mature replication and point-in-time recovery, a managed Multi-AZ offering inside the HDS scope, and skills that are easy to hire. A specialised store is added only when it solves a problem PostgreSQL does not solve well (§3.3).
3. **Classify before storing.** Every field has a data class (§3.2) that decides its encryption, masking, logging and retention rules.
4. **Append-only for anything clinical, legal or financial.** Records, prescriptions, fulfilment statuses, ledger and audit are never updated in place — changes are new versions or new entries.
5. **Derived data is disposable.** Caches, search indexes and projections can be rebuilt from the systems of record; they are never the only copy of anything.
6. **Every store is encrypted, private, backed up, monitored and has a retention rule** — no exception.

3.2 Data classification

Class	Definition	Examples	Mandatory controls
C4 — Sensitive health data	Clinical content, sensitive categories, national health identifier	Notes, diagnoses, results, documents, chat content, mental/sexual health, genetics, INS	Clinical zone only · per-patient application encryption + storage encryption · ABAC with purpose of use · never cached (except encrypted emergency summary) · never in logs, events, search indexes or non-prod
C3 — Health-related personal data	Data revealing that a person receives care	Appointments, doctor–patient links, prescriptions, fulfilment statuses	Clinical zone · storage encryption, sensitive fields encrypted · minimised for pharmacies · audited access
C2 — Personal data	Identity and contact data without health information	Name, e-mail, phone, address, preferences, invoices	Field encryption for contact data · masking for support · GDPR rights applied
C1 — Internal	Operational, non-personal	Configuration, metrics, pseudonymous telemetry	Access control, standard encryption
C0 — Public	Meant to be published	Doctor public profiles, drug reference data, static content	Integrity controls

3.3 Storage technology choices

Technology	Role at MedX	Chosen because	Deliberately not used for
PostgreSQL (managed, Multi-AZ, synchronous standby, read replicas, cross-region replica)	System of record of all eight services, the FHIR store, emergency grants, document metadata	ACID, rich constraints (e.g., exclusion constraint against double booking), JSONB for FHIR resources, row-level security, PITR, mature HA	Very high-volume append-only logs with TTL (chat hot store, notification log)

Technology	Role at MedX	Chosen because	Deliberately not used for
Redis / Valkey (cluster, replicas in 3 AZs, TLS, encryption at rest)	Slot holds, availability cache, sessions and token denylist, rate limits, idempotency keys, presence, pub/sub	Sub-millisecond latency, atomic `SET NX`, TTLs, pub/sub	Any source of truth (everything in Redis can be rebuilt)
Cassandra (RF 3 across 3 AZs, TLS, encryption at rest)	Consultation chat hot store (30 days), notification log (13 months)	Linear write scalability, time-ordered partitions, native TTL, multi-datacenter replication	Relational integrity, ad hoc queries, long-term clinical storage
Object storage (S3-compatible: versioning, Object Lock, SSE-KMS, cross-region replication)	Medical documents, chat transcripts, prescription PDFs, invoices, audit archive, backups	11-nines durability, WORM retention, lifecycle tiers, low cost at 150 TB	Small structured records
OpenSearch (private, encrypted, fine-grained access control)	Doctor directory, drug catalogue search, restricted audit investigation index	Full-text, facets, geo queries, fast aggregation	Storing clinical content (it is **not** a PHI store)
Kafka (3 brokers, RF 3)	Event log, 7 days (30 days for audit)	Durable, ordered per key, replayable	Long-term storage
Columnar analytics store (Parquet files + query engine)	De-identified analytics	Cheap scans over large volumes	Identifiable data

Alternatives considered — a document database for FHIR JSON (PostgreSQL JSONB gives the same flexibility with cross-resource transactions and one less technology to secure); a ledger database for audit (hash-chained batches on WORM object storage give tamper evidence without a niche product); DynamoDB-style key-value stores (vendor lock-in and harder portability across HDS providers).

3.4 Logical data models per service

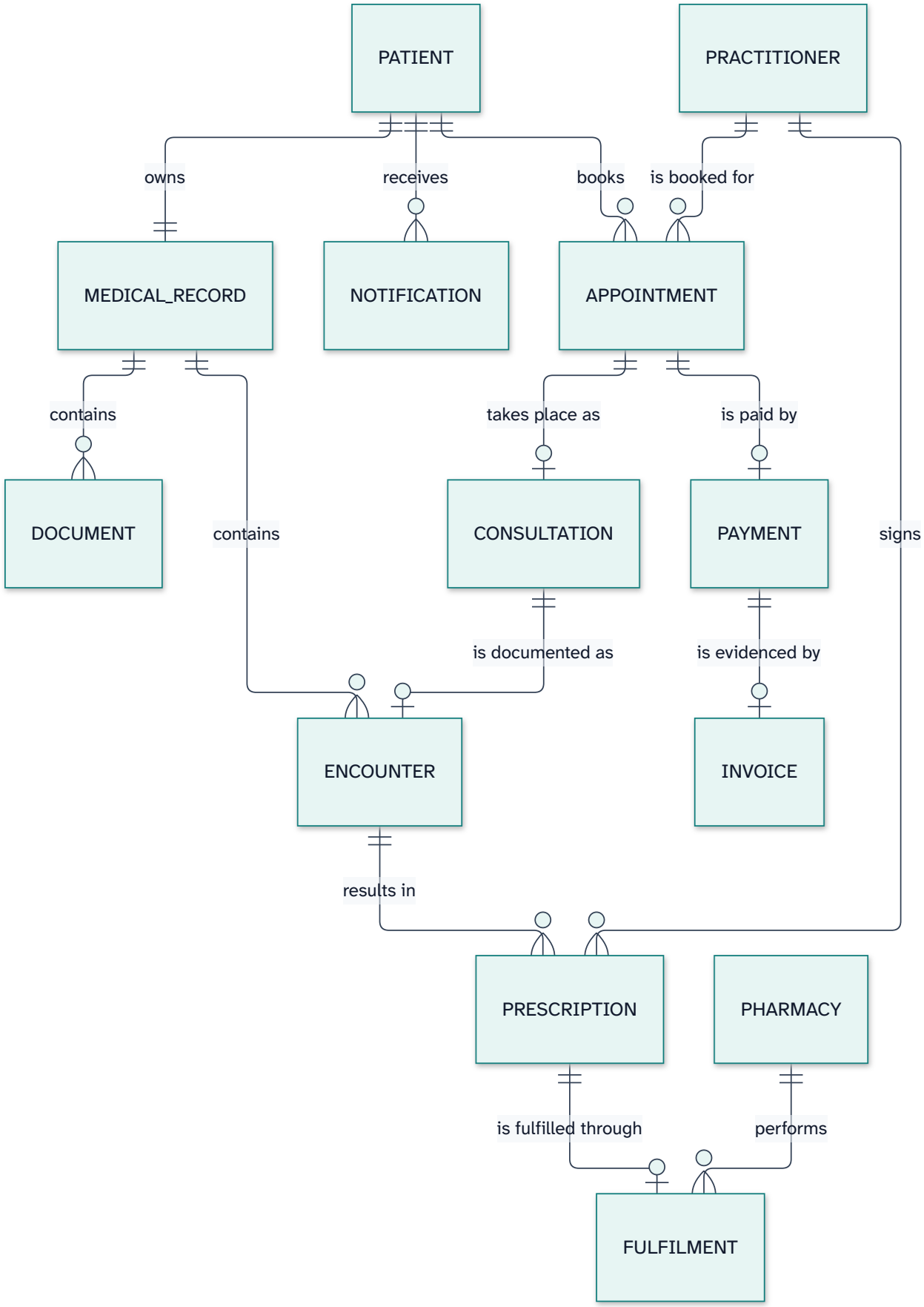
Storage strategy per component — synthesis

Component	System of record	Secondary stores	Key design points	Volume (Year 3)
① User Management	PostgreSQL `usermgmt` + Keycloak's PostgreSQL	Redis (sessions, denylist)	Contact data encrypted, e-mail blind index, preferences as versioned JSONB	~50 GB
② Scheduling	PostgreSQL `scheduling`	Redis (holds, availability, queues), OpenSearch (directory)	Exclusion constraint, append-only history, reason encrypted	~250 GB/year
③ Consultation	PostgreSQL `consultation`	Cassandra (chat, 30-day TTL), Redis (routing, presence), Document Store	Chat encrypted per consultation, transcript archived to the record	~50 GB/year + ~120 GB hot chat
④ Patient Records	PostgreSQL `records` (FHIR resources + history)	Read replicas (3 AZs + DR), Document Store	Append-only history, hash chain, per-patient keys, projections	**~10 TB** structured
④ Emergency Access	PostgreSQL `emergency` (own, per region)	Records read replicas	Grant + audit in one transaction, active in two regions	< 10 GB
⑤ Prescription	PostgreSQL `prescription`	Drug reference replica, OpenSearch (drug search)	Signed immutable versions, two state machines	~250 GB/year
⑥ Pharmacy Interface	PostgreSQL `pharmacy` (+ PostGIS)	Redis (availability cache)	Append-only status events, credentials in Vault	~100 GB/year
⑦ Notification	Cassandra (log, 13-month TTL)	PostgreSQL (templates, preference replica), Redis (dedupe, caps)	No PHI stored at all	~1.2 TB (13 months, before replication)

Component	System of record	Secondary stores	Key design points	Volume (Year 3)
⑧ Billing & Payment	PostgreSQL `billing`	Document Store (invoice PDFs, WORM)	Double-entry append-only ledger, minor-unit integers, PSP tokens only	~100 GB/year
Document Store	Object storage	PostgreSQL (metadata, wrapped keys)	Quarantine → scan → encrypt → Object Lock	**~150 TB** cumulative
Audit Trail	Object storage (WORM, security account)	OpenSearch (13 months), SIEM	Hash-chained batches, signed daily Merkle roots	~0.7 TB/year compressed

Cross-service domain model

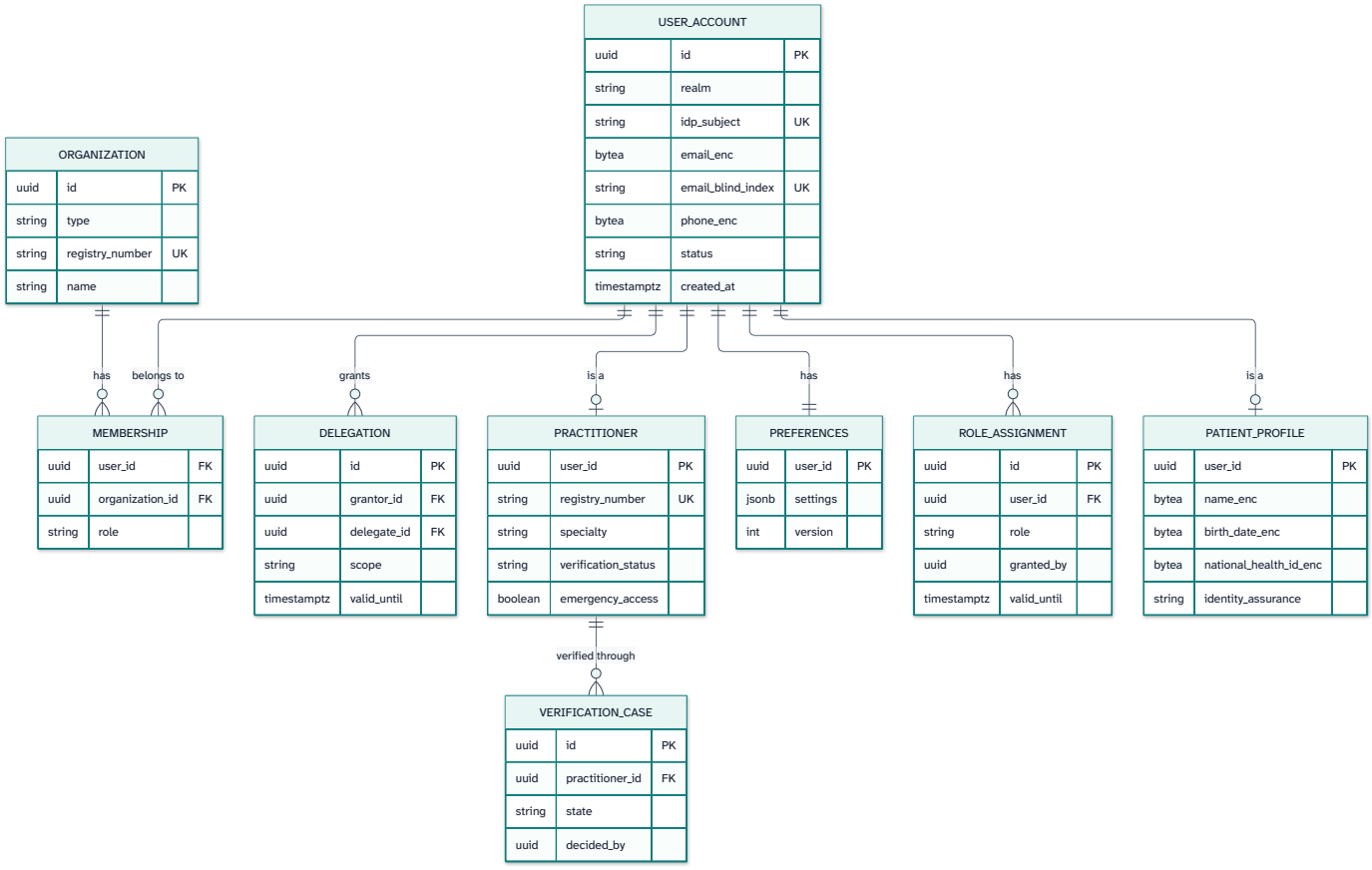
Entities belong to one service; relationships across services are references by identifier.



Entity	Owner service	Store
Patient, Practitioner, Pharmacy (identity, organisation)	① User Management	PostgreSQL
Appointment	② Scheduling	PostgreSQL

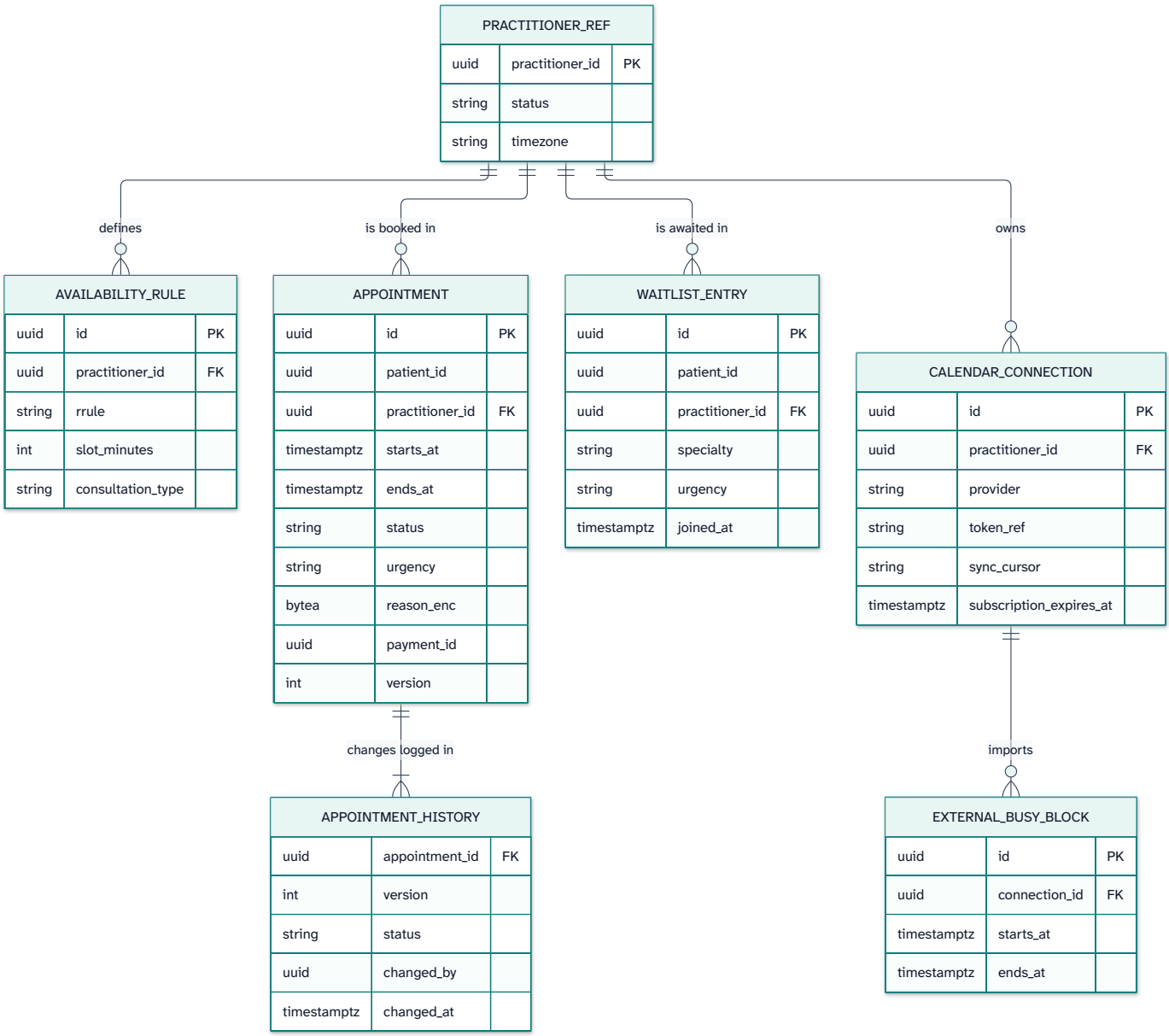
Entity	Owner service	Store
Consultation	③ Consultation	PostgreSQL + Cassandra (chat)
Medical record, Encounter, clinical resources	④ Patient Records	PostgreSQL (FHIR)
Document	Document Store (metadata) / ④ (clinical reference)	Object storage + PostgreSQL
Prescription	⑤ Prescription	PostgreSQL
Fulfilment	⑥ Pharmacy Interface (operational) / ⑤ (legal status)	PostgreSQL
Notification	⑦ Notification	Cassandra
Payment, Invoice	⑧ Billing & Payment	PostgreSQL + object storage

① User Management



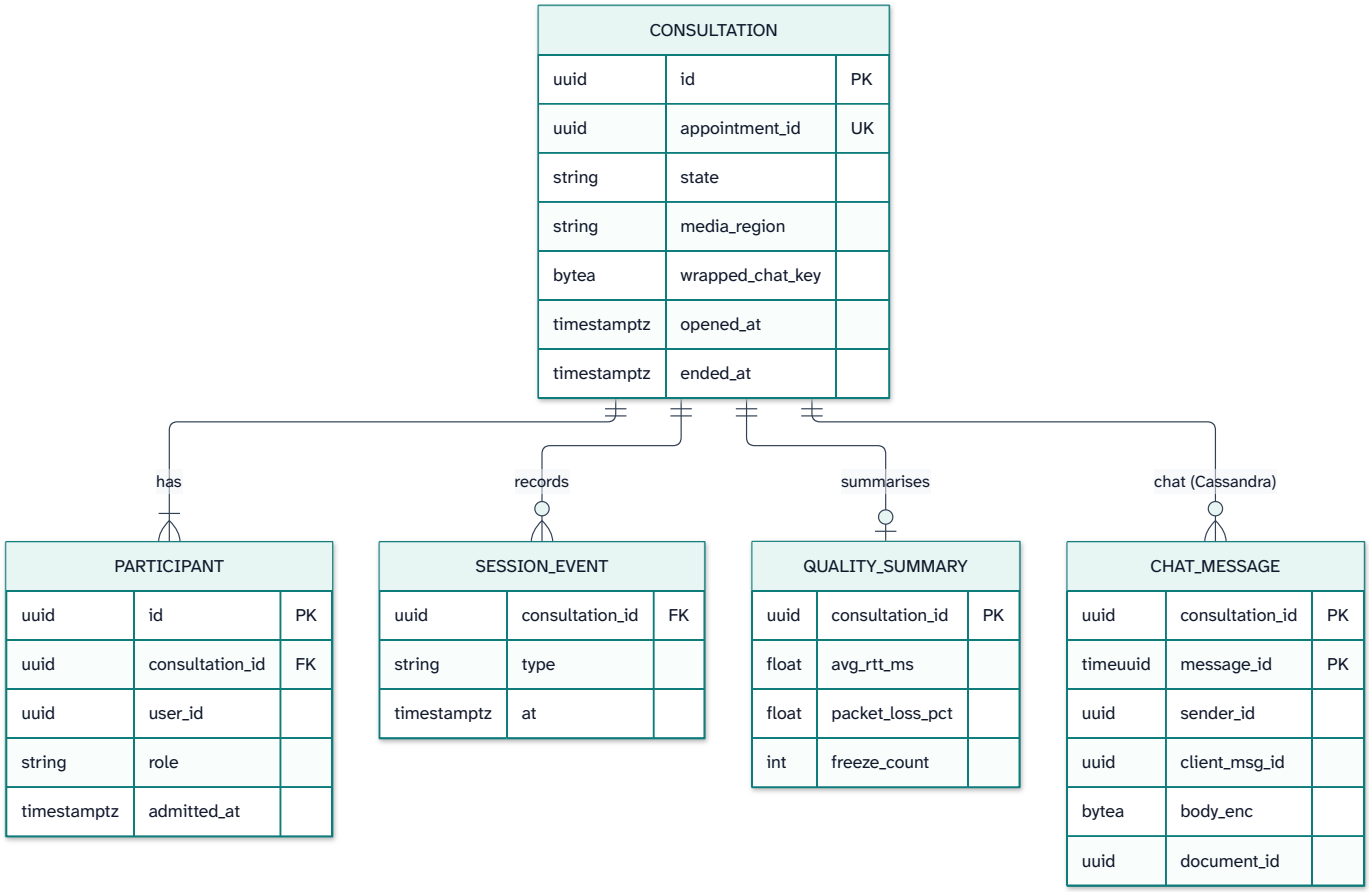
Blind index: `email_blind_index = HMAC-SHA256(k, normalised e-mail)` lets the service find an account by e-mail without storing the address in clear; the HMAC key lives in Vault.

② Scheduling



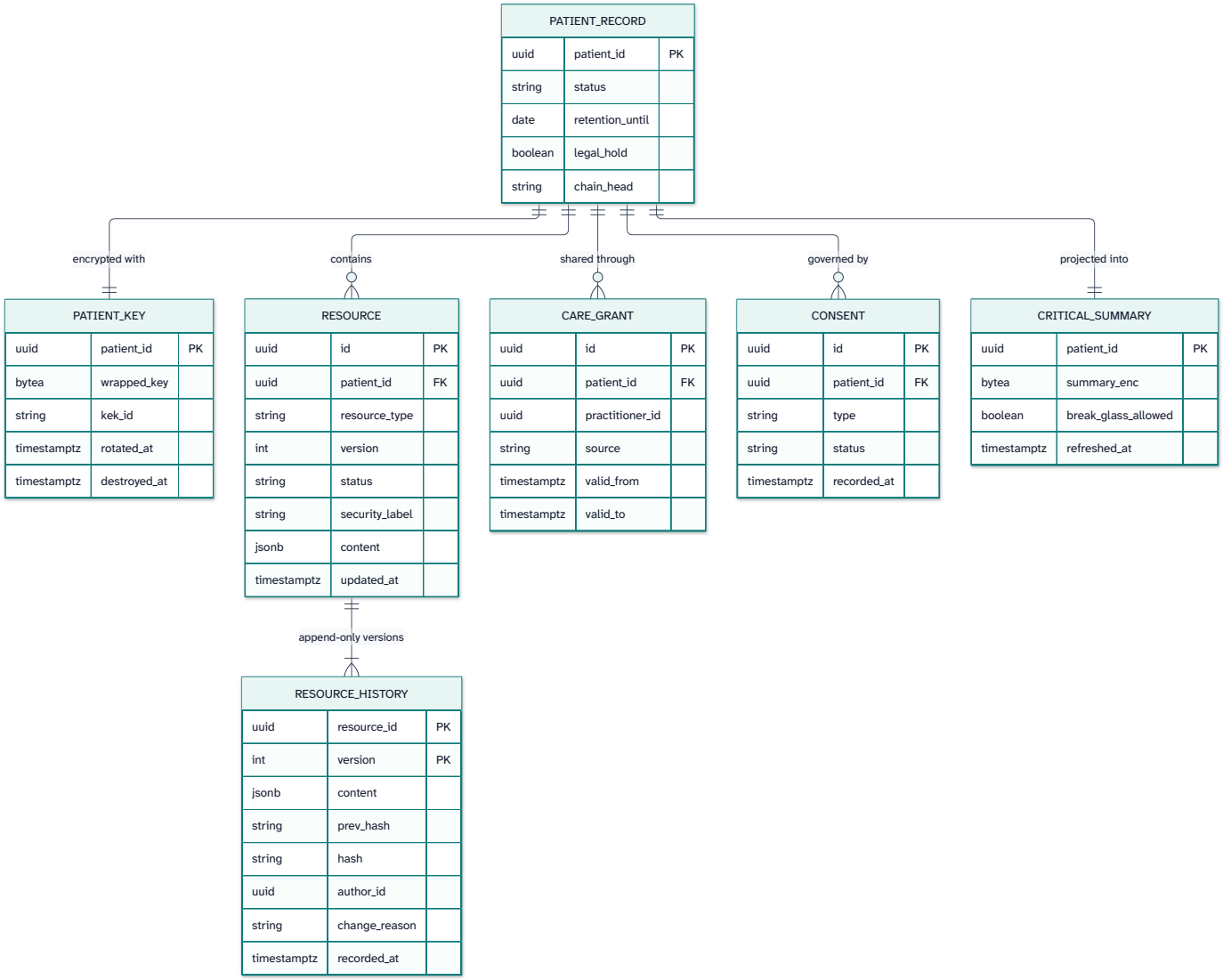
The exclusion constraint on `(practitioner_id, tstzrange(starts_at, ends_at))` for active statuses is described in §2.2. Redis keys: `hold:{practitioner}:{start}` (TTL 10 min), `avail:{practitioner}:{date}` (hash of free slots), `waitlist:{practitioner|specialty}` (sorted set by urgency then time).

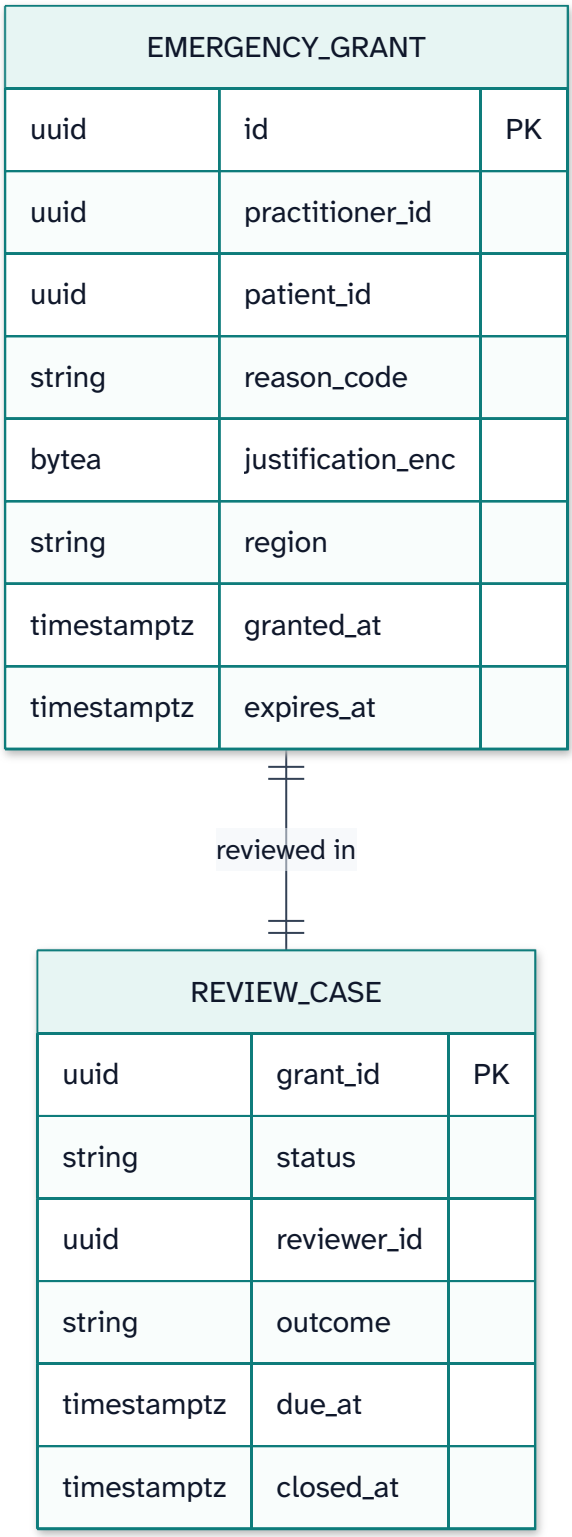
③ Consultation



Cassandra partition = `consultation_id` (all messages of one consultation together, ordered by `timeuuid`); `default_time_to_live` = 30 days; the transcript is archived into the record before expiry.

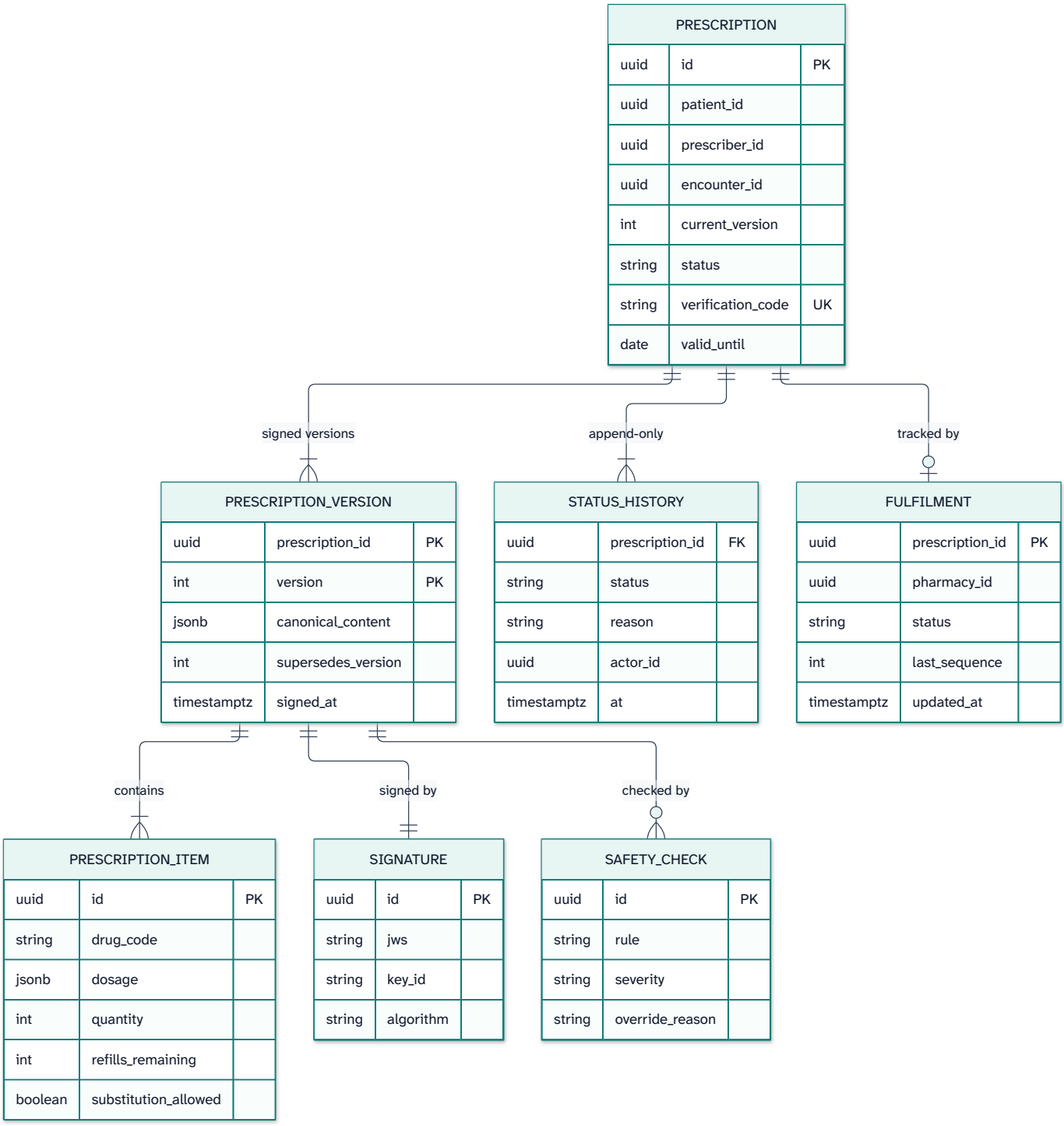
④ Patient Records and Emergency Access



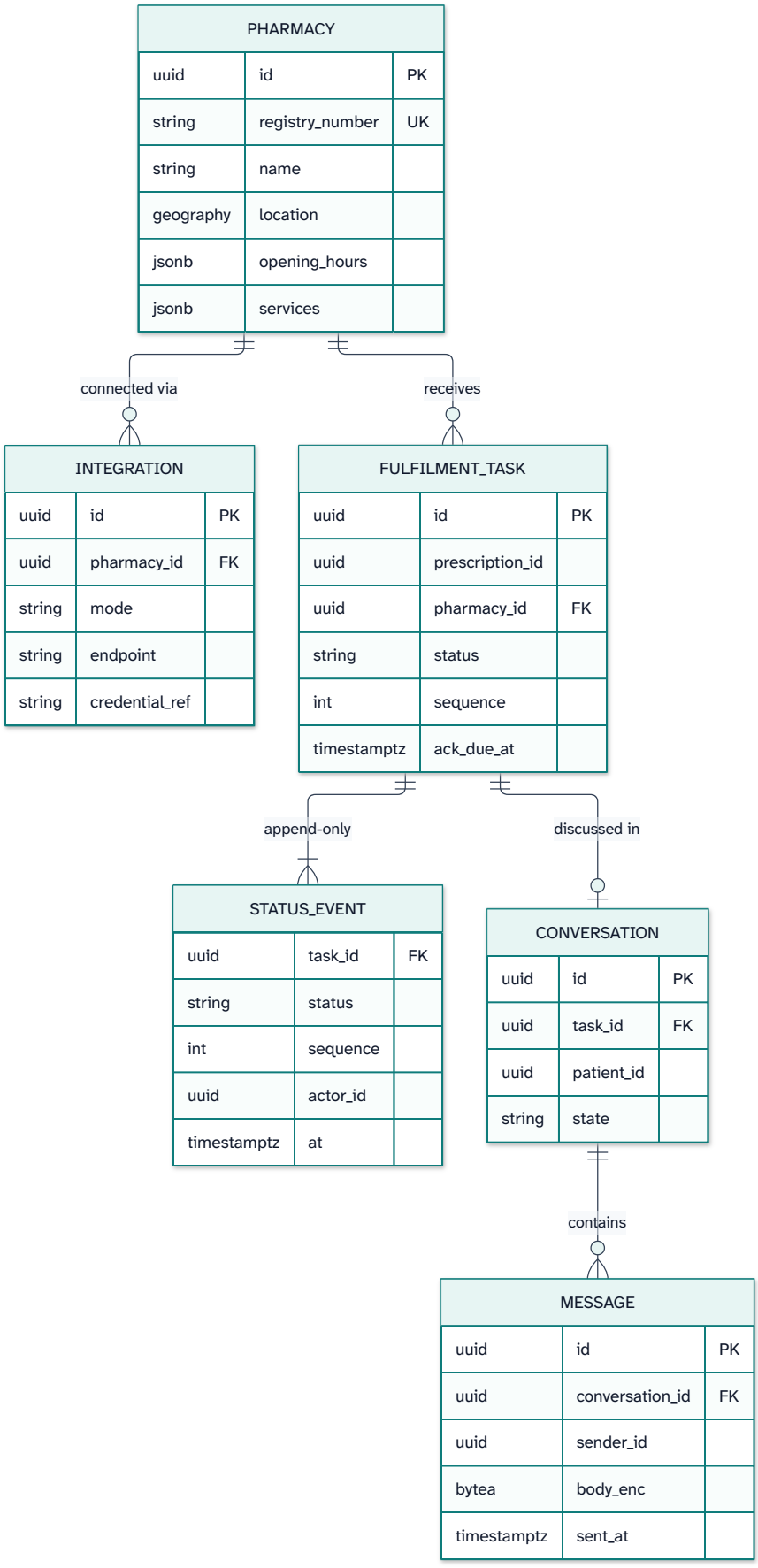


- `RESOURCE.content` holds the FHIR JSON; elements classified C4 (free-text notes, sensitive categories, national identifier) are replaced by **ciphertext encrypted with the patient key** (AES-256-GCM, additional authenticated data = resource ID + version, so ciphertext cannot be moved to another resource).
- FHIR search parameters are extracted into indexed tables at write time (as HAPI FHIR does) — only non-sensitive search parameters (codes, dates, statuses) are indexed in clear.
- Tables are **partitioned by hash of** `patient_id` from day one, preparing for sharding by patient.
- The database role of the service has `INSERT/SELECT` only on `RESOURCE_HISTORY`; a trigger rejects any update.

⑤ Prescription



⑥ Pharmacy Interface



⑦ Notification (Cassandra)

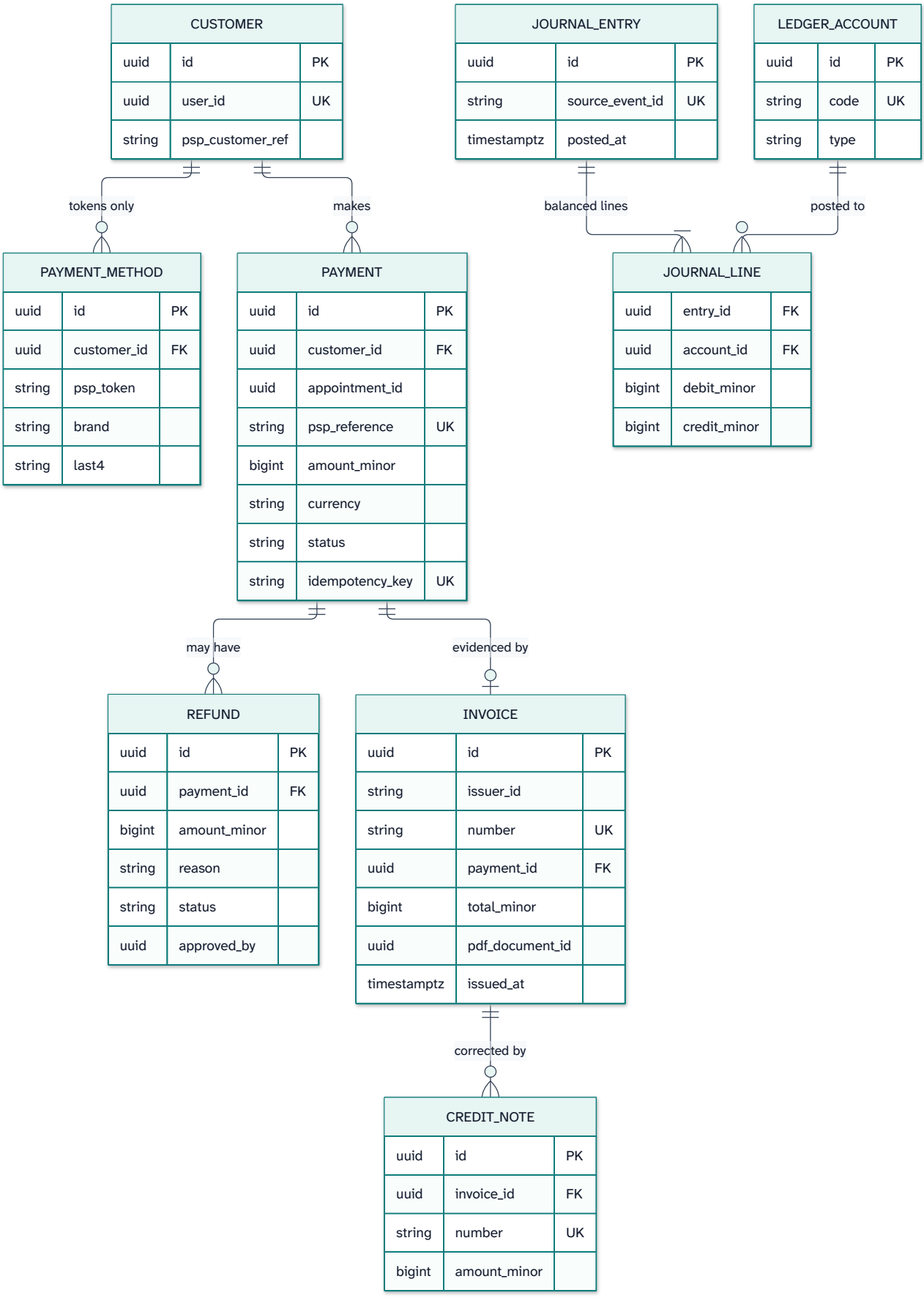
Query-driven tables: "a user's notifications, newest first" and "the status history of one notification".

```
CREATE TABLE notification_by_recipient (
    recipient_id    uuid,
    month           text,          -- '2026-03': bounds partition size
    created_at      timeuuid,
    notification_id uuid,
    category        text,
    priority        text,
    channel         text,
    template_id     text,
    status          text,
    PRIMARY KEY ((recipient_id, month), created_at)
) WITH CLUSTERING ORDER BY (created_at DESC)
  AND default_time_to_live = 34214400;   -- 396 days, about 13 months

CREATE TABLE notification_status_event (
    notification_id uuid,
    at              timeuuid,
    status          text,
    provider        text,
    provider_ref    text,
    PRIMARY KEY ((notification_id), at)
) WITH default_time_to_live = 34214400;
```

No message body with personal content is stored — only the template ID and the rendering variables' references.

⑧ Billing & Payment



Invariant enforced by a deferred constraint trigger: for every `JOURNAL_ENTRY`, Σ debits = Σ credits. `source_event_id` is unique → the same PSP event can never be posted twice.

Audit Trail (WORM archive + index)

```
{
  "id": "0192a4c1-8f3e-7b21-9c4d-2e5f6a7b8c9d",
  "recorded": "2026-03-02T03:12:44.182Z",
  "action": "read",
  "purposeOfUse": "ETREAT",
  "outcome": "success",
  "breakGlass": true,
  "agent": { "id": "0192…", "role": "doctor", "organization": "0192…", "aal": 3, "ip":
"203.0.113.10", "device": "web" },
  "entity": { "patientId": "0192…", "resourceType": "Composition", "resourceId": "0192…" },
  "source": { "service": "emergency-access", "region": "eu-central", "traceId":
"4bf92f3577b34da6a3ce929d0e0e4736" },
  "batch": { "id": "2026-03-02T03:12Z-eu-central-17", "prevHash": "9f2c…", "hash": "b71e…" }
}
```

3.5 Consistency model

Data	Consistency needed	Mechanism
Slot uniqueness	**Strong**	Exclusion constraint (single primary) + Redis hold
Clinical record writes	**Strong**, no lost update	Single primary, `If-Match` optimistic concurrency, read-your-writes routing for the author
Prescription legal status	**Strong** in the Prescription service; propagated to pharmacies in seconds	State machine + outbox + reconciliation every 5 min
Payments and ledger	**Strong**, effect exactly once	Idempotency keys, unique source event IDs, balanced entries
Booking + payment across services	**Eventually consistent business transaction**	Saga with compensation (release slot, void authorisation)
Availability cache, directory, timeline projection	Eventual (seconds)	Recomputed from events; rebuildable
Critical (emergency) summary	Eventual (p99 < 5 s)	Event-driven projection; the emergency view also lists changes of the last 24 h read directly from the replica
Notification preferences replica	Eventual (seconds)	`PreferencesUpdated` events

3.6 Identifiers

- **UUIDv7** for every entity: globally unique across services and regions, time-ordered (good index locality), not guessable in sequence — never auto-increment integers exposed in URLs (prevents enumeration attacks).
- **Patient identity**: MedX patient ID (internal, pseudonymous) + national health identifier (INS) stored encrypted and used only for identity matching and national-service exchanges.
- Human-facing references (invoice numbers, prescription verification codes) are separate, purpose-specific identifiers.

3.7 Data lifecycle and retention

Data	Retention	End of life	Basis
Medical record (resources, documents, transcripts, prescriptions)	**20 years after the last encounter** (France; longer for minors), configurable per jurisdiction; legal hold possible	**Crypto-shredding** (patient keys destroyed) then physical deletion	Public-health law
Chat hot store	30 days (transcript already archived in the record)	TTL expiry	Minimisation
Video/audio	**Not recorded**	—	Privacy by design
Appointments	5 years after the appointment, then anonymised for statistics	Anonymisation job	Evidence of care, disputes
User account (patient)	While active; closed after 3 years of inactivity with prior notice	Deletion of C2 data; medical record kept under its own rule	GDPR storage limitation
Practitioner identity	As long as records reference the practitioner	Kept (attribution of records)	Integrity of records
Invoices, ledger	10 years	Deletion	Commercial law
Payment tokens	Until removed by the user, or 13 months after last use	Deleted at the PSP and locally	PCI DSS, minimisation
Notification log	13 months	TTL expiry	Evidence of delivery
Audit trail	≥ 6 years, configurable up to record retention	Object Lock expiry then deletion	Accountability
Technical/security logs	12 months	Deletion	Security investigations
Backups	PITR 35 days · daily snapshots 35 days · monthly snapshots 12 months	Automatic expiry (immutable until then)	Recovery
Key registries (wrapped patient keys)	Backups kept 35 days only	Expiry	Makes crypto-shredding effective in backups
De-identified analytics	Anonymous data: unlimited · pseudonymised datasets: 2 years	Deletion	Research, statistics

Right to erasure vs medical retention — GDPR art. 17(3) exempts data that must be kept for healthcare or legal obligations: a patient can close their account and delete contact data, but the medical record stays under its legal retention (the patient is told so clearly).

3.8 Data integrity mechanisms

Threat	Mechanisms
Concurrent edits (lost update)	Optimistic concurrency (`If-Match` /version), single primary for writes
Invalid or inconsistent data	FHIR profile & terminology validation, foreign keys and check constraints, state machines, exclusion constraint
Partial failures between services	Transactional outbox, idempotent consumers, sagas with compensation, reconciliation jobs
Silent tampering (insider, compromised admin)	Append-only history, per-patient hash chains, daily Merkle root signed in HSM and timestamped, WORM audit archive, verification jobs
Corrupted files	SHA-256 checksum at upload, at every download and in weekly scrub jobs; object versioning
Accidental or malicious deletion, ransomware	No delete privilege for application roles on history; Object Lock (compliance mode); **immutable, cross-account backup vault**; deletion protection on databases

Threat	Mechanisms
Loss of a zone or region	Synchronous standby in another AZ (RPO 0), cross-region replicas (RPO ≤ 1 min), replicated object storage
Undetected backup failure	Automated restore tests every week (sample) and full restore drill every quarter, with integrity verification

3.9 Backup and recovery

Store	Backup	Restore objective
PostgreSQL (all services)	Continuous WAL archiving → **PITR 35 days**; daily snapshots; monthly snapshots 12 months; copies in the immutable vault (other account, other region)	Point-in-time restore of one service without touching others
Cassandra	Daily snapshots + incremental backups to object storage	Chat is rebuildable from record transcripts; notification log is non-critical
Object storage	Versioning + Object Lock + cross-region replication + vault copy	Object or version restore; region failover
Kafka	Not backed up as a store (events are also in databases); MirrorMaker to DR for continuity	Replay from outboxes if needed
Redis / OpenSearch	Not backed up (derived data) — rebuilt from systems of record	Rebuild jobs (minutes to hours)
Keys (KMS/HSM)	Provider-managed, multi-region keys; Vault DR replication	—

3.10 Data residency and replication

- All health data is stored and processed in the **EEA**, in HDS-certified regions (primary France, DR Germany).
- Cross-region replication is encrypted in transit and re-encrypted at rest with multi-region keys.
- Expansion to another jurisdiction (e.g., the US) is done with a **separate regional cell**: a full copy of the platform with its own data, keys and compliance scope (HIPAA); only non-personal configuration is shared globally. Patient data never flows between cells except at the patient's request.

4. Scalability & Performance

Deliverable 4. How MedX handles a growing user base and keeps its performance — with numbers. Targets come from §0.7 capacity targets and §0.8 service levels.

4.1 Strategy in one page

Lever	What MedX does
Scale out, not up	Stateless services on Kubernetes, autoscaled on the metric that really limits each one (requests, connections, queue lag, participants).
Separate what scales differently	Media plane, records, notifications and payments have independent scaling and failure domains.
Keep the hot path short	Local token validation and policy decisions, pre-computed availability and emergency summaries, at most two synchronous hops.
Absorb peaks asynchronously	Kafka and priority lanes turn spikes into backlogs that drain in seconds, never into errors.
Protect what matters most	Bulkheads and load-shedding order: emergency access → ongoing consultations → clinical writes → booking → the rest.
Grow in steps with explicit triggers	Replicas → partitioning → sharding by patient → regional cells; each step triggered by measured thresholds.
Prove it continuously	Load, soak, chaos and WebRTC tests at 2× the forecast peak before each major release; SLOs with error budgets.

4.2 Workload model

Pattern	Description	Design response
Daily cycle	Peaks Monday 8–10 am and 6–8 pm; nights ≈ 5 % of peak	Scheduled pre-scaling + autoscaling; batch jobs at night
Seasonal surges	Flu season, epidemics: **×3 to ×5** demand in days	Surge playbook (§4.10), headroom ×2 by design, DR region can absorb read traffic
Reminder bursts	Thousands of reminders due at round hours	Jittered sending window, dedicated notification lane
Read-heavy clinical data	Records read ≈ 5–10× more than written	Replicas, projections, patient-scoped queries
Long-lived connections	WebSocket for chat/status, WebRTC for media	Dedicated gateways and media nodes scaled on connections/participants

4.3 Capacity estimates (Year 3)

Quantity	Estimate	Result	Design capacity
API throughput	~75 M calls/day ÷ 86 400 s × peak factor 5	**≈ 4.3 k req/s**	10 k req/s
Concurrent video sessions	200 k consultations/day × 12 % in peak hour × 20 min	**≈ 8 k**	10 k (autoscale to 20 k)
SFU egress bandwidth	20 k participants × ~1.2 Mbit/s received	**≈ 24 Gbit/s**	48 Gbit/s

Quantity	Estimate	Result	Design capacity
Media nodes (8 vCPU)	20 k participants ÷ ~200 per node	**≈ 100 nodes**	up to 200
WebSocket connections	1 M daily users × ~20 % connected at peak	**≈ 200 k**	400 k
Records reads	~10 M/day, peak factor 5	**≈ 600 req/s**	2 k req/s
Records writes	200 k consultations × ~10 resources, over working hours	**≈ 55/s, peaks ≈ 110/s**	500/s
Domain events	~50 M/day	**≈ 600/s avg, 3 k/s peak**	trivial for a 3-broker Kafka cluster
Audit events	20 M/day	**≈ 230/s avg, 1.2 k/s peak**	batched writes to WORM storage
Notifications	2 M/day	**≈ 25/s avg, 500/s at reminder peaks**	2 k/s (provider limits)
Document storage	~1 MB/consultation + patient uploads	**≈ 75 TB/year** at Year-3 run-rate, 150 TB cumulative	object storage (no practical limit)
Structured clinical data	~50 KB/consultation incl. history & indexes	**≈ 4 TB/year**, 10 TB at Year 3	single primary + replicas; sharding path ready

4.4 Performance objectives and latency budgets

SLOs are defined per criticality tier in §0.8. Each critical request has a **latency budget** that the design must respect:

Emergency critical summary (target p95 < 300 ms server-side)

Step	Budget
Gateway: JWT signature (cached JWKS), DPoP check, rate limit	≤ 5 ms
mTLS hop to the Emergency Access pod	≤ 2 ms
OPA decision (sidecar, in memory)	≤ 1 ms
Insert grant + audit outbox (synchronous Multi-AZ commit)	≤ 15 ms
Read pre-computed summary (one indexed row, local replica)	≤ 10 ms
Unwrap patient key (in-memory cache; KMS on a miss)	≤ 1 ms (≤ 30 ms on miss)
Decrypt, apply obligations, serialise	≤ 5 ms
Typical total	**≈ 40–70 ms**, leaving a wide margin under 300 ms

Booking confirmation (target p95 < 800 ms excluding 3-D Secure) — hold ≤ 20 ms (Redis + insert), payment authorisation 300–600 ms (PSP, the dominant term), commit + outbox ≤ 20 ms.

Chat message (target p95 < 500 ms) — gateway ≤ 5 ms, authorisation + encryption ≤ 3 ms, Cassandra write (`LOCAL_QUORUM`) ≤ 10 ms, pub/sub fan-out ≤ 5 ms; the rest is network.

4.5 Scaling by layer and by service

Layer	Mechanism
Edge	CDN offloads static assets; managed WAF/load balancers scale automatically; gateway pods autoscaled on RPS and CPU.

Layer	Mechanism
Services	Kubernetes Horizontal Pod Autoscaler on CPU and custom metrics (RPS, p95 latency, Kafka lag via KEDA); cluster autoscaler adds nodes; pod anti-affinity spreads replicas across 3 AZs; minimum 3 replicas for Tier 0/1.
Data	Vertical sizing + read replicas + connection pooling (PgBouncer) + partitioning; sharding path for Records (§4.7).
Events	Topics partitioned by aggregate key (12–48 partitions); consumer groups scale up to the partition count.
Real-time	WebSocket pods scaled on open connections; Redis pub/sub cluster for fan-out.
Media	SFU nodes scaled on participants and egress bandwidth; region-aware placement; drain before scale-in.

Service	Load driver	Autoscaling signal	Replicas (Year 3, min–max)	Main bottleneck → mitigation
API Gateway	Requests	RPS, CPU	6 – 40	—
Real-time Gateway	Connections	Connections per pod (~25 k)	4 – 20	Memory per connection → lean frames, heartbeats
① User Management + IdP	Logins, refreshes	CPU	3 – 12	Password hashing CPU → rate limits, passkeys
② Scheduling	Searches, bookings	RPS	3 – 30	Availability computation → pre-computed Redis cache
③ Consultation (control plane)	Joins, chat	RPS, WebSocket messages	3 – 20	—
Media plane (SFU + TURN)	Participants	Participants per node, bandwidth	10 – 200 nodes	Bandwidth → simulcast, adaptive stream, audio-only
④ Patient Records	Reads	RPS, p95 latency	4 – 30	Database → replicas, projections, sharding path
Emergency Access	Break-glass requests	**Reserved capacity** + RPS	3 per region (fixed floor) – 12	Isolated by design (bulkhead)
⑤ Prescription	Writes, checks	RPS	3 – 12	Drug checks → cached reference data
⑥ Pharmacy Interface	Webhooks, status updates	Queue lag	2 – 12	Slow pharmacy systems → asynchronous webhooks
⑦ Notification	Events	Consumer lag per priority lane	2 – 40	Provider rate limits → two providers per channel
⑧ Billing & Payment	Payments	RPS	3 – 8	PSP latency → asynchronous capture
Audit Trail	Audit events	Consumer lag	2 – 10	Object-storage writes → 1-minute batches

4.6 Caching strategy

Cache	Content	Location	Freshness	Health data?
CDN	App bundles, images	Edge	Versioned, immutable	No
Keys & policies	JWKS, OPA bundles	In-process	5 min / pulled every 60 s	No
Availability	Free slots per doctor and day (60 days)	Redis	Recomputed on events (seconds)	No patient data
Directory	Doctor public profiles + next free slot	OpenSearch	Event-driven	No (public)
Drug reference	Drug catalogue, interactions	In-process + OpenSearch	Daily refresh	No

Cache	Content	Location	Freshness	Health data?
Emergency summary	International Patient Summary per patient	PostgreSQL projection + replicas	Event-driven (p99 < 5 s)	**Yes — encrypted**
Data keys	Unwrapped patient/file keys	Service memory only	≤ 5 min, bounded LRU	Key material, never persisted
Inventory	Availability level per pharmacy and item	Redis	15 min	No

Rules — no health data in shared caches (the encrypted emergency summary is the only exception); responses with personal data carry `Cache-Control: no-store`; caches use jittered TTLs and request coalescing to avoid stampedes; every cache is rebuildable from its system of record.

4.7 Database scaling path

Stage	Trigger (measured)	Action
1 — Launch	—	One Multi-AZ primary (synchronous standby) per service; Records with 2 read replicas; PgBouncer; `statement_timeout` and connection quotas per service
2 — Growth	Records > 2 TB, replica lag > 1 s, or primary CPU > 60 % at peak	Larger instances, more replicas, partition pruning, older history partitions moved to a cheaper storage tier
3 — Scale-out	Records writes > ~5 k/s or > 20 TB	**Shard by `patient_id` hash** (e.g., Citus or application-level routing); each shard with its own replicas — nearly every query is patient-scoped, so shards are independent
4 — New jurisdiction	New country or regulation	New **regional cell** (§4.12)

Query hygiene is part of the definition of done: indexes reviewed with `EXPLAIN` in CI for new queries, `pg_stat_statements` dashboards, keyset pagination (no deep `OFFSET`), no N+1 access patterns.

4.8 Video at scale

- **One room per SFU node** for 1-to-1 consultations (no cross-node forwarding); multi-party rooms (interpreter) can cascade across nodes.
- **Placement** chooses the region with the lowest measured RTT for both participants, then the least-loaded node (CPU, bandwidth, participants).
- **Simulcast + dynacast + adaptive stream** send each receiver only the resolution its screen and network need; average received bitrate ~1.2 Mbit/s instead of 2.5 Mbit/s for full 720p.
- **Audio-first**: under congestion video degrades first; below ~150 kbit/s the call becomes audio-only.
- **TURN** fleet sized for ~20 % of sessions relayed; TURN over TLS 443 for restrictive networks.
- **Scale-in by draining**: a node marked for removal accepts no new rooms and is removed when its last call ends.
- **Load testing**: headless-browser bots generate thousands of simultaneous calls with injected packet loss and jitter; the node capacity figure (~200 participants per 8 vCPU) is re-measured at every SFU upgrade.
- **Cost lever to evaluate**: direct peer-to-peer media for 1-to-1 calls when networks allow would cut SFU egress, at the price of weaker quality monitoring and control — kept as an optimisation, not the baseline.

4.9 High availability and disaster recovery

Component	Within the region (AZ failure)	Across regions (region failure)
Gateways, services	≥ 3 replicas over 3 AZs; load balancer health checks	Pre-deployed in DR, scaled up by the runbook
Emergency Access, IdP	≥ 3 replicas over 3 AZs	**Active in both regions** (DNS health-checked routing)
PostgreSQL	Synchronous standby in another AZ, automatic failover < 60 s (RPO 0)	Asynchronous cross-region replica (lag < 1 s, RPO ≤ 1 min), promoted by the runbook
Redis	Replicas in other AZs, automatic failover	Rebuilt (derived data)
Kafka	3 brokers / 3 AZs, RF 3, `min.insync.replicas = 2`	MirrorMaker 2 to the DR cluster
Cassandra	RF 3 across AZs, `LOCAL_QUORUM`	Second datacenter in DR (asynchronous)
Object storage	Regional service (multi-AZ by design)	Cross-region replication
Media plane	Nodes in 3 AZs, clients reconnect	Media nodes in DR scale up

Regional failover runbook (rehearsed twice a year):

1. Detection by synthetic probes and health checks → incident declared, communication started.
2. Tier 0 continues immediately: emergency access and sign-in are already served by the DR region.
3. Promote DR database replicas for Tier 1 services (scripted, in dependency order), switch producers to the DR Kafka cluster (translated consumer offsets).
4. Scale DR Kubernetes node groups to production size (pre-approved capacity reservations) → **Tier 1 back ≤ 1 h**.
5. Restore Tier 2 services and rebuild derived stores (caches, search) → **Tier 2 back ≤ 4 h**.
6. Fail back after recovery with reverse replication, outside peak hours.

4.10 Resilience patterns and graceful degradation

- **Timeouts everywhere** (no call without a deadline), **retries with exponential backoff and jitter** only for idempotent operations, **circuit breakers** in the mesh, **bulkheads** (separate pools and deployments for the emergency path and each priority lane), **idempotency keys** on every create.
- **Load-shedding order** under extreme load, enforced by gateway route priorities, Kubernetes priority classes and queue lanes: ① emergency access → ② ongoing consultations → ③ clinical writes (notes, prescriptions) → ④ booking → ⑤ non-critical notifications → ⑥ analytics and exports.

Failure	Degraded behaviour
Payment provider down	Holds keep slots; retry; optional "pay later" booking for returning patients
Calendar provider down	Last known busy times used; doctor warned that sync is delayed
SMS / e-mail provider down	Automatic failover to the second provider; in-app notifications continue
Drug database down	Prescribing possible only after explicit acknowledgement that checks are unavailable
Event bus unavailable	Core transactions continue; events wait in outboxes; notifications and projections catch up
Records primary database down	Automatic failover (< 60 s); reads continue on replicas; emergency access unaffected
Media node failure	Clients reconnect to another node within seconds
Search cluster down	Fallback search by specialty served from PostgreSQL
Whole region down	Tier 0 continues in DR; Tier 1 ≤ 1 h; Tier 2 ≤ 4 h

Surge playbook (epidemic, ×3–×5 demand): scheduled pre-scaling; "surge mode" raising autoscaling floors; additional on-call doctors in the urgent queue; feature flags that switch off non-essential features (exports, recommendations); a **virtual waiting room** in front of booking if needed, so the platform stays responsive for people already in care.

4.11 Performance engineering practice

Practice	Cadence
Performance budgets checked in CI (key endpoints, p95)	Every merge
Load tests at 1× and **2× forecast peak** (k6 / Gatling), including WebRTC bots with network impairment	Every major release
Soak tests (24 h) for leaks and slow degradation	Monthly
Chaos experiments (pod kill, AZ loss, latency injection, dependency outage)	Monthly in staging, quarterly game days in production
DR failover drill	Twice a year
Capacity review vs forecast (users, bandwidth, storage)	Monthly
SLO review with **error budgets** — feature work pauses for reliability when a budget is burnt	Weekly

4.12 Growth beyond Year 3 — cells

When MedX outgrows one region or enters a new jurisdiction, it scales by **cells**: a cell is a complete, isolated copy of the platform (services, data, keys) serving a population (a country, or a shard of patients). A thin global routing layer — holding **no health data** — sends each user to their home cell. Cells limit the blast radius of any incident, keep data residency simple (a US cell under HIPAA, EU cells under GDPR/HDS) and turn growth into a repeatable deployment rather than a redesign.

4.13 Cost efficiency

- Autoscaling floors lowered at night; non-production environments scaled to zero outside working hours.
- Reserved capacity for the steady baseline; spot/preemptible capacity only for stateless, non-critical workloads (never Tier 0, never media).
- Storage lifecycle: documents older than 1 year move to an infrequent-access tier, older than 5 years to an archive tier — with Object Lock retention preserved.
- Media bandwidth is the largest variable cost: simulcast, audio-only fallback and tuned bitrates are both quality and cost measures; **cost per consultation** is tracked as a KPI.

5. Security & Compliance

Deliverable 5. Security protocols, encryption methods and compliance measures that protect MedX's health data. This is where the High-priority requirements H1 (data privacy), H3 (emergency access controls), H4 (authentication & authorization) and H5 (integrity) become concrete controls.

5.1 Objectives and principles

Objective	Meaning for MedX
Confidentiality	Health data is seen only by the patient and the professionals who treat them, for a legitimate purpose.
Integrity	Records, prescriptions and payments are complete, correct and tamper-evident.
Availability	Critical information is reachable in an emergency, even during failures (Tier 0).
Accountability	Every access to health data can be attributed, explained and reviewed.
Privacy	Collect the minimum, use it only for its purpose, keep it only as long as required, respect patients' rights.

Principles: defence in depth · zero trust (no implicit trust from the network: every call is authenticated, authorised and encrypted) · least privilege · deny by default · privacy by design and by default · assume breach (limit blast radius, detect fast, recover from immutable backups) · security as code (policies, infrastructure and controls versioned and tested).

5.2 Regulatory and standards framework

Framework	Why it applies	Key obligations	How the design complies
GDPR (EU 2016/679)	Health data is a special category (art. 9)	Lawful basis (art. 9(2)(h) provision of health care), privacy by design (art. 25), security (art. 32), **DPIA** (art. 35), DPO (art. 37), records of processing (art. 30), data-subject rights, breach notification within **72 h** (art. 33–34), processor contracts (art. 28), no transfer outside the EEA without safeguards	§5.11 privacy operations; encryption, minimisation and retention built in; EEA-only hosting
HDS certification (French Public Health Code, art. L.1111-8)	Hosting health data of French patients	Hosting by HDS-certified providers; HDS v2 (2024) requires hosting in the EEA; MedX certified for the activities it operates itself	HDS-certified cloud regions (France + Germany); only certified services used for PHI; MedX certification plan (§5.12)
National e-health frameworks (ANS: PGSSI-S, INS, Pro Santé Connect, interoperability framework)	French health information systems	Security policy, national patient identifier, professional authentication, interoperable formats	PGSSI-S-aligned policies, INS in the Patient resource, Pro Santé Connect federation, FHIR/IPS model
Telemedicine rules (Public Health Code, art. R.6316-1 et seq.)	Teleconsultations	Authenticated professionals, identified and consenting patients, secure exchanges, report in the record	Verified practitioners + MFA, patient identity assurance, consent at booking, E2EE video, encounter note in the record
HIPAA (US) — if MedX expands	US patients' PHI	Privacy, Security and Breach Notification Rules; BAAs; minimum necessary	Separate US cell, same controls, BAAs with sub-processors

Framework	Why it applies	Key obligations	How the design complies
PCI DSS v4.0	Card payments	Protect cardholder data	PSP-hosted fields → **SAQ A**; payment-page script integrity
eIDAS (EU 910/2014 as amended by 2024/1183)	E-signatures and timestamps	Legal value of signatures and timestamps	Prescriber signatures (qualified options), qualified timestamps for integrity anchors
NIS2 (EU 2022/2555)	Healthcare is a high-criticality sector	Risk-management measures; incident reporting: early warning 24 h, notification 72 h, final report 1 month	Measures adopted by design (§5.8–5.10); reporting process in the incident plan
European Health Data Space (EU 2025/327)	Patient access and exchange of electronic health data (phased obligations)	Patient access & sharing rights, interoperability and logging requirements for EHR systems	FHIR/IPS model, patient export and access log, audit trail
Medical Device Regulation (EU 2017/745)	Software that supports clinical decisions may be a medical device	Qualification, CE marking if in scope	Triage and prescription checks assessed for qualification; kept rule-based and validated
Standards	Assurance and best practice	ISO/IEC 27001 & 27701, ISO 27799 (health), OWASP ASVS & API Top 10, NIST SP 800-63B, CIS Benchmarks, FHIR security (SMART scopes), IHE BALP (audit)	Used as control baselines (§5.8)

5.3 Threat model

Built with STRIDE on the architecture's data flows; the table lists the threats with the highest impact on MedX.

#	Threat	Example	Main mitigations	Residual risk
T1	Account takeover	Credential stuffing on patients, phishing a doctor	MFA (phishing-resistant for professionals), breached-password check, rate limiting, DPoP-bound tokens, new-device alerts	Low
T2	Broken object-level authorization	Changing an ID in a URL to read another patient's record	ABAC decision on every object, UUIDs, negative authorization tests in CI, gateway schema validation	Low
T3	Insider snooping	Staff or doctor browsing a celebrity's record	No staff access to PHI, care-grant model, audit of every read, UEBA alerts, patient-visible access log, sanctions	Medium → Low
T4	Ransomware / destructive attack	Encryption or deletion of databases and backups	Immutable cross-account backup vault, Object Lock, least privilege, segmentation, EDR, tested restores	Low
T5	Data exfiltration	Leaked snapshot, misconfigured bucket, compromised service	Per-patient application encryption (stolen data is ciphertext), KMS key policies, egress allow-lists, CSPM, DLP on logs	Low
T6	Injection & web attacks	SQL injection, XSS, SSRF	Parameterised queries, output encoding, strict CSP, WAF, SAST/DAST, egress control	Low
T7	Eavesdropping on consultations	Man-in-the-middle on video	DTLS-SRTP + E2EE, TLS 1.3, certificate pinning in apps	Low
T8	Malicious file upload	Malware disguised as a medical PDF	Quarantine, antivirus, magic-byte type checks, PDF sanitisation, isolated scanners	Low
T9	Supply-chain compromise	Malicious dependency or base image	Dependency scanning, pinned versions, SBOM, signed images verified at admission, build provenance	Medium

#	Threat	Example	Main mitigations	Residual risk
T10	DDoS and abuse	Booking bots, directory scraping	CDN/WAF, bot management, rate limits, virtual waiting room	Low
T11	Prescription fraud	Forged or re-used prescriptions	Signed prescriptions, verification endpoint with live status, QR codes without content	Low
T12	Break-glass abuse	Fake emergency to read a record	Entitlement, justification, step-up MFA, real-time alert, patient notice, mandatory review, sanctions	Low–Medium
T13	Payment fraud	Stolen cards	3-D Secure (SCA), PSP risk scoring, velocity limits	Low
T14	Leakage through notifications	An SMS revealing a diagnosis to a family member	No PHI in external messages, allow-listed template variables	Low
T15	Cloud misconfiguration	Public bucket, disabled logging	Organisation guardrails (deny public access, deny disabling encryption/logging), IaC scanning, CSPM	Low

5.4 Identity and access management

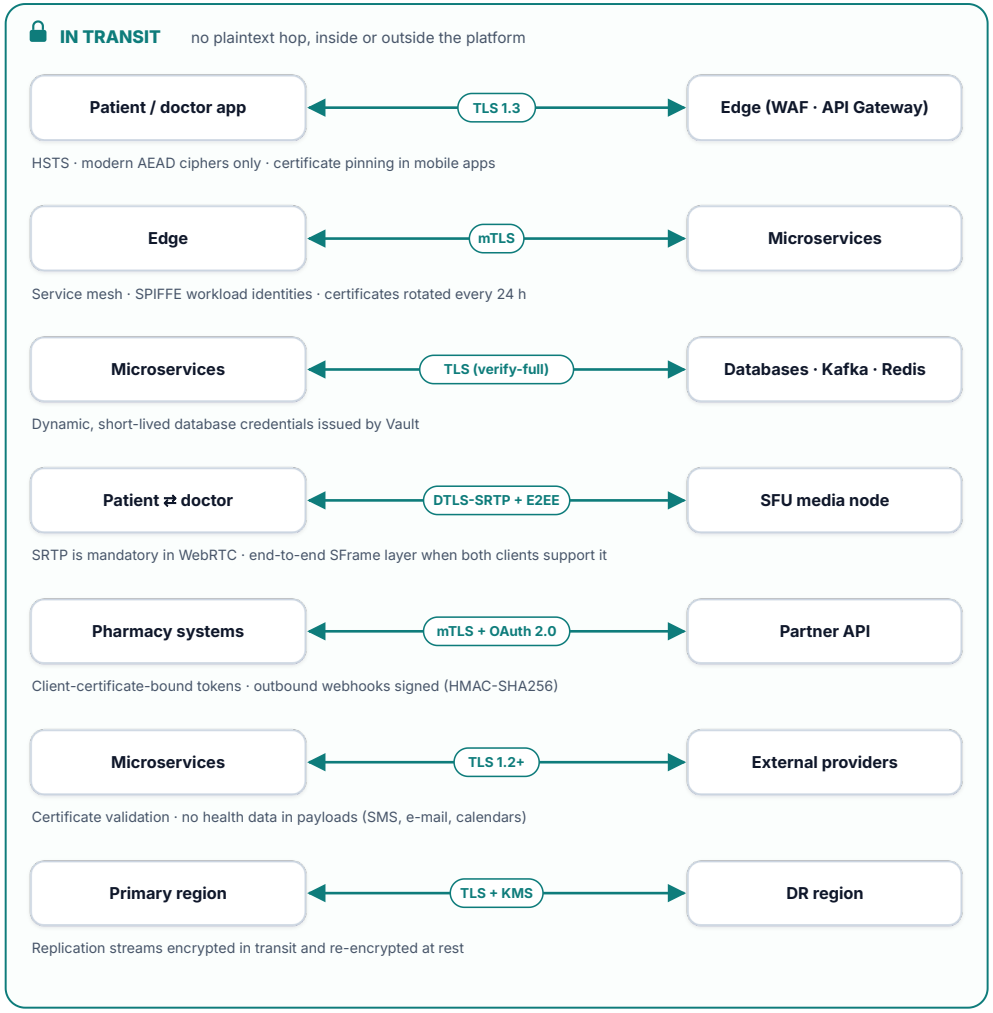
Details are in §2.1 User Management; the security-relevant rules are:

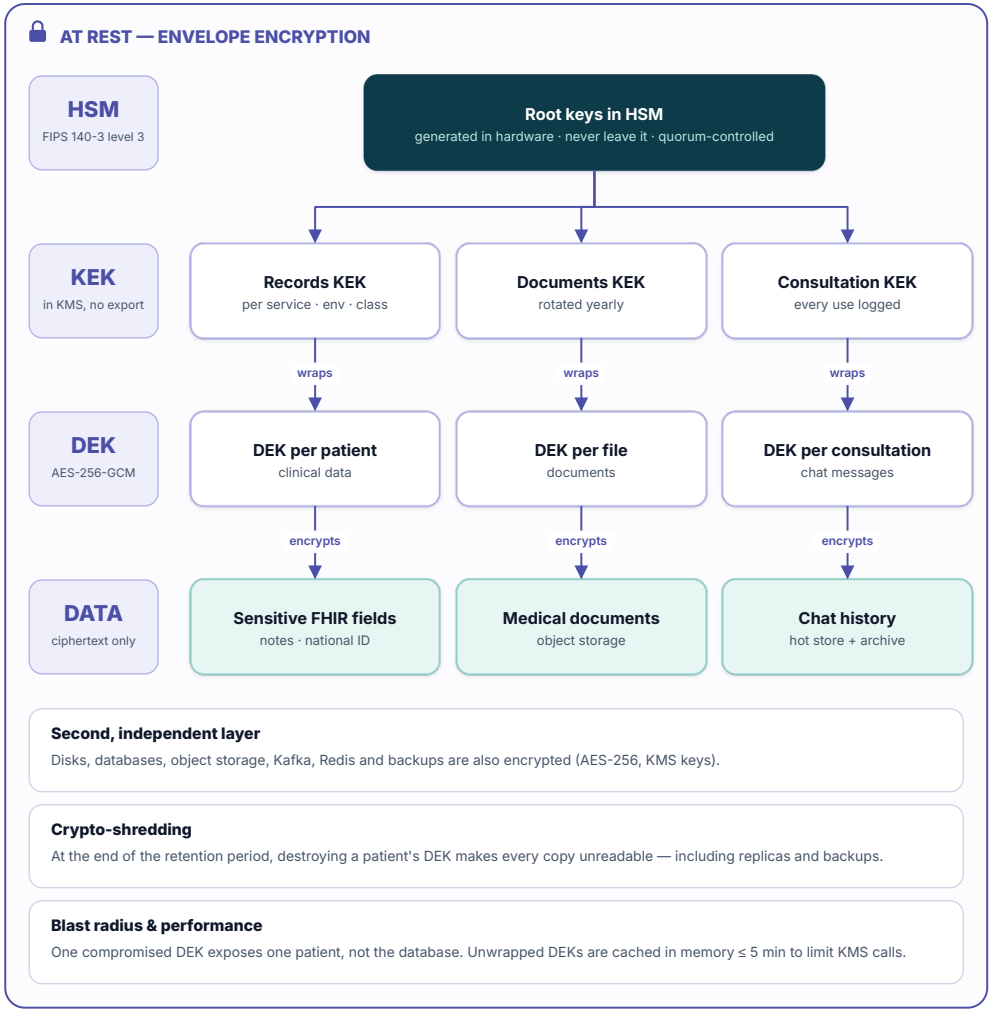
- **Authentication assurance** — patients: password/passkey + risk-based MFA; professionals: phishing-resistant MFA always (passkeys, security keys, professional smart card via Pro Santé Connect); **prescription signing and break-glass require a fresh hardware-bound authentication** (step-up within 15 minutes). Short-lived, audience- restricted, DPoP- or certificate-bound tokens.
- **Authorization** — RBAC for coarse permissions + ABAC (care relationship, consent, purpose of use, sensitivity labels, organisation) evaluated by OPA next to every service; deny by default; obligations for masking.
- **Privileged access** — no standing administrator access to production: just-in-time elevation with approval, MFA, a 4-hour maximum and session recording; platform administrators **cannot read PHI** (application-level encryption keys are not usable by their roles).
- **Service identities** — SPIFFE identities through the mesh (mTLS), mesh authorisation policies define which service may call which; one cloud IAM role per service with least privilege (e.g., only the Records role may use the Records KEK).
- **Lifecycle** — automated deprovisioning for staff leavers; monthly registry sync suspends struck-off practitioners; **quarterly recertification** of entitlements such as `emergency_access`; segregation of duties between developers, database administrators and key custodians; 4-eyes for sensitive operations.

5.5 Encryption in transit

Encryption in Transit & at Rest

High-priority requirement H1 · every hop encrypted · two independent layers at rest · keys generated and held in HSM-backed KMS





Encryption in transit and at rest

Channel	Protocol	Configuration
Apps ↔ edge	**TLS 1.3** (TLS 1.2 with ECDHE + AEAD suites only for older devices; TLS ≤ 1.1 disabled)	HSTS with preload, OCSP stapling, automatic certificate rotation, public-key pinning (with backup keys) in mobile apps
Edge ↔ services, service ↔ service	**mTLS** (service mesh, STRICT mode — plaintext refused)	SPIFFE workload identities, certificates rotated every 24 h, authorisation policies per caller
Services ↔ datastores	TLS with server verification	PostgreSQL `sslmode=verify-full`; TLS for Redis, Kafka (plus mTLS client authentication), Cassandra (client and inter-node), OpenSearch
Audio/video	**DTLS-SRTP** (mandatory in WebRTC) + **E2EE** (SFrame) for 1-to-1 when supported	TURN over TLS 443 on restrictive networks
Pharmacy systems, partners	**mTLS + OAuth 2.0** with certificate-bound tokens	Outbound webhooks signed HMAC-SHA256 with timestamp and replay protection
External providers (SMS, e-mail, push, calendars, PSP)	TLS 1.2+ with certificate validation	No PHI in payloads; SMTP with MTA-STS, SPF/DKIM/DMARC
Region ↔ region replication	TLS	Re-encrypted at rest with multi-region keys

Cipher policy — TLS 1.3 suites (AES-128/256-GCM, ChaCha20-Poly1305) with forward secrecy (X25519, P-256). Because health data must stay confidential for decades, MedX enables **hybrid post-quantum key exchange** (X25519 + ML-KEM) wherever clients and load balancers support it, against "harvest now, decrypt later" attacks (crypto-agility is a design requirement: algorithms are configuration, not code).

5.6 Encryption at rest and key management

Two independent layers

Layer	Scope	Mechanism	Protects against
1 — Storage encryption	Every volume, database, snapshot, backup, object, Kafka log, Redis, Cassandra, OpenSearch	AES-256 with customer-managed KMS keys	Lost disks, stolen snapshots, provider-side media access
2 — Application (envelope) encryption	C4 data: sensitive FHIR elements, documents, chat, national identifiers, contact data	**AES-256-GCM** in the service with per-patient / per-file / per-consultation data keys; additional authenticated data binds ciphertext to its record	Database-level compromise, over-privileged administrators, misconfigured replicas or exports, SQL injection returning raw rows

Key hierarchy — root keys in **HSMs (FIPS 140-3 level 3)** → **KEKs** in KMS per service × environment × data class (never exported) → **data keys (DEKs)** generated by KMS, stored only in wrapped form next to the data → ciphertext.

Lifecycle step	Rule
Generation	In the HSM/KMS only; no application-generated long-term keys
Access	KMS key policies: only the owning service's role may decrypt with its KEK; key administrators cannot use keys to decrypt; every use logged in the security account
Caching	Unwrapped DEKs kept in process memory ≤ 5 minutes (bounded LRU), never written to disk or logs
Rotation	KEKs rotated automatically every year (old versions kept for decryption; DEKs re-wrapped progressively); DEKs rotated on suspicion of compromise
Separation	Production keys unusable from non-production accounts → a production snapshot cannot be decrypted outside production

Lifecycle step	Rule
Disaster recovery	Multi-region keys for the DR region; Vault replicated
Destruction	**Crypto-shredding**: at the end of retention the patient's data keys are destroyed; key registries keep only 35 days of backups, so every remaining copy of the data — replicas, backups — becomes unreadable

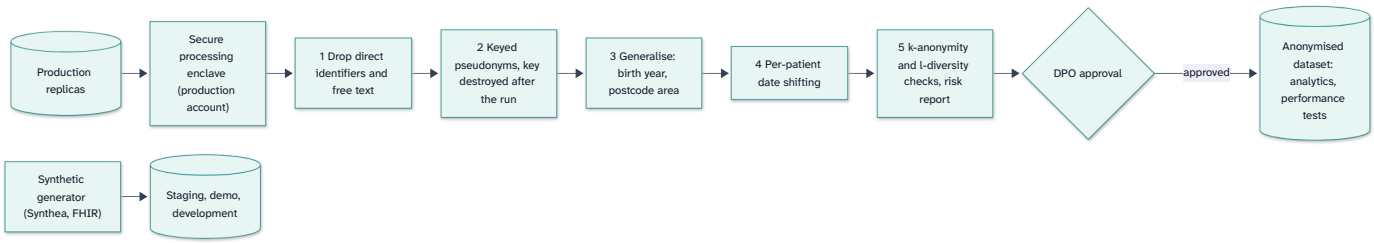
Other protections — passwords hashed with Argon2id; secrets in Vault (dynamic database credentials, Transit encryption for integration tokens); payment cards tokenised by the PSP; mobile apps keep no health data beyond the session except documents the user explicitly downloads, protected by OS keystore-backed file encryption and wiped on logout.

5.7 Data masking, redaction and anonymization

Dynamic masking (M4) — the policy engine returns masking obligations applied before any response leaves a service (support agents see `+33 6 •• •• •• 42`, pharmacists see no diagnosis, break-glass excludes psychotherapy notes…). The complete matrix is in §2.4.

Redaction — documents shared or exported can have tagged sections removed; PDF redaction removes the text layer (true redaction, not a black rectangle); logs and traces never contain PHI (allow-listed fields, tokenised identifiers, automated scanner on log samples).

Anonymization for staging and demo (L4) — rule: *production health data never leaves production*.



Environment	Data	Controls
Development, staging, demo	**Synthetic** patients (e.g., Synthea FHIR bundles) and generated appointments, prescriptions, payments; demo tenant watermarked "DEMO"	Separate accounts, no network path or key access to production
Performance tests needing realistic distributions	**Anonymised** extracts produced by the pipeline above	k ≥ 10 on quasi-identifiers, l-diversity on sensitive attributes, documented re-identification risk assessment, DPO approval, dataset expiry
Analytics and research	Anonymised data; pseudonymised data only inside production-grade controls and with legal basis/consent	Purpose registry, access reviews

5.8 Application and API security

- **Secure development lifecycle** — threat modelling for each significant feature, security champions in every team, **OWASP ASVS level 2** baseline and **level 3** for User Management, Records, Emergency Access and Prescription; security acceptance criteria in the definition of done.
- **Automated checks in CI** — SAST, dependency scanning (SCA), secret scanning, infrastructure-as-code scanning, container image scanning; DAST and API fuzzing against staging; **authorization test suites** with negative cases (patient A must never read patient B).

- **Independent testing** — penetration test before launch and every year, after major changes; bug-bounty programme after launch.
- **API protections (OWASP API Top 10)** — object- and property-level authorization, response allow-lists (no mass exposure), explicit request models (no mass assignment), rate limits and quotas, SSRF protection through an egress gateway with allow-listed destinations, complete API inventory in the gateway.
- **Web** — strict nonce-based CSP, Trusted Types, SRI, HSTS, `frame-ancestors 'none'` (except the PSP component), SameSite cookies, CSRF tokens for cookie sessions.
- **Mobile (OWASP MASVS)** — certificate pinning, app attestation, sensitive screens hidden from screenshots and task switcher, no health data in device logs, rooted/jailbroken devices treated as a risk signal.
- **Supply chain** — pinned dependencies, SBOM for every image, minimal (distroless) base images patched weekly, **signed images verified at admission**, build provenance (SLSA), reviewed changes only (protected branches).

5.9 Infrastructure and network security

- **Landing zone** — separate accounts (production, DR, security & log archive, shared services, non-production) under organisation guardrails: deny public storage, deny disabling encryption or logging, deny regions outside the EEA.
- **Network** — private subnets for all services and datastores; public exposure limited to load balancers and media nodes; Kubernetes network policies **default deny**; mesh authorisation policies; private endpoints to cloud services; egress only through controlled gateways with allow-lists.
- **Workload hardening** — CIS benchmarks, Pod Security "restricted" profile, non-root and read-only containers, seccomp/AppArmor, runtime threat detection, EDR on nodes; vulnerability SLAs: critical < 7 days, high < 30 days.
- **Staff access** — identity-aware zero-trust proxy (no VPN, no SSH); production access only through JIT elevation.
- **Physical security** — inherited from the HDS / ISO 27001-certified cloud provider.

5.10 Logging, detection and incident response

- **Security logging** — authentication events, authorisation denials, admin actions, KMS usage, network flows, audit trail; shipped to the **security account** (immutable, 12 months for technical logs, ≥ 6 years for the audit trail).
- **Detection (SIEM + UEBA)** — break-glass events, access without care relationship attempts, mass record reads or exports, impossible travel, MFA disabled, privilege escalation, anomalous KMS decrypt volumes, new public endpoint.
- **Response** — documented plans for data breach, ransomware, account takeover and break-glass abuse; 24/7 on-call; severity levels; forensic readiness (retained logs, snapshot procedures); tabletop exercises twice a year.

Notification duty	Deadline
GDPR — supervisory authority (CNIL)	**72 h** after awareness (art. 33)
GDPR — affected patients	Without undue delay when the risk is high (art. 34)
NIS2 — national CSIRT / authority	Early warning **24 h**, notification **72 h**, final report **1 month**
French health sector — serious incidents	Declaration to the health authorities' incident portal / CERT Santé
HIPAA (US cell)	Individuals within 60 days; HHS as required

5.11 Privacy compliance operations

Area	Measure
Governance	DPO appointed; records of processing; **DPIA before launch** and for every major change (large-scale health data makes it mandatory); privacy review in the design process
Legal bases	Care: art. 9(2)(h); contract/legal obligations for billing; **explicit consent** only where required (research use, marketing, optional sharing, non-essential cookies)
Transparency	Layered privacy notices in the apps; **"who accessed my record"** page for patients
Data-subject rights	Self-service export (FHIR bundle / PDF), rectification through the treating doctor (amendment keeps history), erasure of account data (medical record kept under its legal retention), objection and restriction where applicable (e.g., research, break-glass where allowed); answered within one month
Sub-processors	Data processing agreements, HDS-certified hosting, EU-based providers for any personal data; **no health data sent to messaging or calendar providers**; BAAs for a future US cell
Minors	Parental delegation, adolescent confidentiality rules enforced by policy
Privacy by default	Research and marketing opt-in off by default; minimal sharing; no third-party trackers in authenticated areas

5.12 Compliance and assurance roadmap

Milestone	When
DPIA, records of processing, security policies (PGSSI-S-aligned), HDS-certified hosting contract	Before the first real patient
External penetration test	Before launch, then yearly
HDS certification for MedX's own operating activities	Before launch (scope-dependent)
PCI DSS SAQ A attestation	At payment launch
ISO/IEC 27001 (+ 27701) certification	Year 1–2
SOC 2 Type II, HIPAA readiness	Before US expansion

Compliance is continuous: guardrails and policies as code, automated evidence collection, quarterly access reviews, yearly DR and incident exercises.

5.13 High-priority requirements — controls and evidence

Requirement	Controls	How it is verified
H1 Data privacy (encryption at rest & in transit)	TLS 1.3 / mTLS everywhere; AES-256 storage encryption; per-patient envelope encryption; HSM-backed keys; no PHI outside the clinical zone	Encryption compliance checks on 100 % of stores; TLS configuration scans; mesh in STRICT mode; PHI-in-logs scanner
H2 Patient records	FHIR store, validation, versioning, consent registry, audit of every access	Nightly hash-chain verification; audit coverage tests; access-control test suite
H3 Emergency access	Break-glass with entitlement, step-up MFA, time-boxed grant, Tier-0 deployment in two regions, post-hoc review	Synthetic probes (p95, availability); **100 % of break-glass cases reviewed within 72 h**; DR drills
H4 Authentication & authorization	Keycloak (OIDC, MFA), OPA RBAC + ABAC, JIT privileged access	100 % MFA for professionals; quarterly recertification completed; policy unit tests in CI

Requirement	Controls	How it is verified
H5 Data integrity & secure files	ACID, append-only, hash chains + anchoring, checksums, Object Lock, immutable backups, file scanning	Weekly restore tests + quarterly full drill; checksum scrub with zero mismatch; WORM configuration checks

6. Architecture Decisions & Trade-offs

Why the design looks the way it does. Each decision is traced to MedX's priorities (Appendix 2) and states **what was given up** — a design without acknowledged costs is not a design.

6.1 The tensions that shaped the architecture

Tension	How MedX resolves it	Priority that decided
Privacy ⟷ emergency availability	Access in an emergency is never blocked by a workflow; it is narrow, time-boxed, fully audited and reviewed (break-glass).	H1 + H3
Security ⟷ usability	Strict for professionals (phishing-resistant MFA, step-up), smooth for patients (passkeys, risk-based MFA, biometrics).	H4, then M1–M2
Consistency ⟷ availability	Strong consistency where errors harm (records writes, bookings, prescriptions, payments); eventual elsewhere; the emergency path favours availability with bounded staleness (< 5 s).	H2, H3, H5
Isolation ⟷ operational complexity	Eight business services (no more), grouped in zones, on a shared golden-path platform.	H1, H3
Resilience ⟷ cost	Active-active only for Tier 0 (emergency access, sign-in); warm standby for the rest.	H3
Rich messages ⟷ confidentiality	Generic external messages + deep links into the authenticated app.	H1
Build ⟷ buy	Build what is differentiating or sensitive (records, emergency access, care workflows, policies); buy commodities (IdP software, PSP, messaging, SFU software).	All — frees capacity for High priorities

6.2 Decision log

ADR	Decision	Priority link	Main alternative rejected	What we give up
01	Microservices by bounded context, grouped into security zones	H1, H3, scalability	Modular monolith	Simpler operations
02	HL7 FHIR R4 (+ IPS) as the clinical model, on PostgreSQL	H2, interoperability	Custom relational schema	Some storage efficiency
03	PostgreSQL by default; specialised stores only when justified	H5, simplicity	NoSQL-first polyglot persistence	Extreme write scale by default
04	Append-only clinical history + hash chains + signed daily anchors	H2, H5	In-place updates + audit table	~30 % more storage
05	Two encryption layers with per-patient envelope keys	H1	Storage encryption only	Complexity, ~1 ms per request
06	Break-glass with post-hoc review on an isolated, two-region Tier-0 path	H3	Pre-approval workflow	Some risk of misuse (mitigated)
07	Proven IdP (Keycloak) + OPA policies (RBAC + ABAC) enforced locally	H4	Custom auth; RBAC only	Running a stateful IdP cluster
08	Event-driven integration (Kafka) with transactional outbox	H5, scalability	Synchronous call chains	Eventual consistency to reason about
09	Self-hosted WebRTC SFU (LiveKit), E2EE by default for 1-to-1	H1, M2	Video CPaaS; P2P; MCU	Operating media servers; no recording

ADR	Decision	Priority link	Main alternative rejected	What we give up
10	No PHI outside the trust boundary (messages, calendars, events, logs)	H1	Rich notifications	Some convenience
11	PSP-hosted payment fields → PCI DSS SAQ A	L5, H1	Handling card data	Control over payment UI
12	Multi-AZ everywhere + warm-standby DR; active-active only for Tier 0	H3, H5	Full active-active	RTO of 1–4 h for Tier 1–2
13	Synthetic data by default outside production; anonymisation under DPO control	L4, H1	Masked copies of production	Some realism in tests
14	Deliver by priority: secure core → care workflows → ecosystem	All	Feature-first MVP	Later commercial launch

6.3 Decision records

ADR-01 — Microservices by bounded context, grouped into security zones

- **Context** — eight business capabilities with very different data sensitivity (PHI, card-adjacent, contact data), load profiles (video bandwidth vs record reads vs payment correctness) and criticality (Tier 0 to 2).
- **Decision** — one service per Appendix-1 component, plus platform services; zones (identity, clinical, engagement, payment) with dedicated node pools and network policies.
- **Alternatives** — monolith (one blast radius); modular monolith (good start, but no isolation of the emergency path nor of compliance scopes); function-as-a-service (cold starts, poor fit for WebRTC/WebSocket).
- **Consequences** — ✅ isolation of PHI and failures, independent scaling and releases; ❌ distributed-systems complexity → golden-path template, service mesh, platform team, strict limit on the number of services.

ADR-02 — HL7 FHIR R4 (+ International Patient Summary) as the clinical data model

- **Context** — records must be complete, valid, versioned and exchangeable with national services and hospitals; emergency access needs a standard summary.
- **Decision** — FHIR resources stored by a FHIR server (HAPI FHIR on PostgreSQL); IPS profile for the critical summary; national profiles for validation.
- **Alternatives** — custom schema (faster to start, costly integrations later); document database (flexible, but weaker transactions and one more technology).
- **Consequences** — ✅ interoperability, validation, history and search out of the box; ❌ generic storage is less efficient → UI projections and partitioning.

ADR-03 — PostgreSQL by default, specialised stores only when justified

- **Decision** — PostgreSQL for all systems of record; Redis (ephemeral), Cassandra (append-heavy TTL logs), object storage (files), OpenSearch (search) only where PostgreSQL is a poor fit.
- **Why** — fewer technologies to secure, certify, back up and staff; constraints and transactions protect integrity (e.g., exclusion constraint against double booking).
- **Consequences** — ✅ integrity and simplicity; ❌ extreme write scale needs the sharding path (§4.7).

ADR-04 — Append-only clinical history, hash chains and signed anchors

- **Decision** — no in-place update or physical deletion of clinical data during retention; each version chained by hash; daily Merkle root signed in the HSM, timestamped and stored in WORM.
- **Why** — H5 data integrity and H2 compliance: history must be provable, not just "logged".
- **Consequences** — ✅ tamper evidence even against administrators; ❌ ~30 % extra storage, verification jobs.

ADR-05 — Two encryption layers with per-patient envelope keys

- **Decision** — storage encryption everywhere + application-level AES-256-GCM for C4 data with per-patient / per-file / per-consultation keys wrapped by HSM-backed KEKs.
- **Alternatives** — storage encryption only (does not protect against database-level compromise or leaked exports); one KMS key per patient (millions of KMS keys: cost and quota limits).
- **Consequences** — ✅ stolen data is useless, blast radius of one key = one patient, crypto-shredding at end of retention; ❌ key-management complexity, ~1 ms overhead with key caching, careful key backup policies.

ADR-06 — Break-glass with post-hoc review, on an isolated Tier-0 path

- **Context** — H3 requires *immediate* access for authorised professionals; H1 requires strict confidentiality.
- **Decision** — self-service break-glass (entitlement + justification + step-up MFA) → 4-hour read-only grant → pre-computed IPS summary; real-time alert, patient notice, mandatory review within 72 h; separate deployment and database, active in two regions.
- **Alternatives** — supervisor pre-approval (too slow at 3 am in an emergency room); open access for all doctors (unacceptable privacy risk).
- **Consequences** — ✅ access in seconds, even during failures; ❌ misuse is possible but visible, attributable and sanctioned; one more deployable unit.

ADR-07 — Proven IdP + policy-as-code authorization

- **Decision** — Keycloak for authentication (OIDC, MFA, passkeys, federation); OPA sidecars evaluating RBAC + ABAC policies from signed bundles; short-lived tokens validated locally.
- **Alternatives** — custom authentication (high risk); RBAC only (cannot express care relationships, consent or emergencies); central authorization server called on every request (latency and single point of failure).
- **Consequences** — ✅ strong, standard, testable, fast; ❌ revocation is not instantaneous (bounded by 5–10 min tokens + denylist), a stateful IdP to operate in two regions.

ADR-08 — Event-driven integration with the transactional outbox

- **Decision** — services publish domain events (ids and statuses only) through an outbox relayed to Kafka; consumers are idempotent; synchronous calls only when a user is waiting (≤ 2 hops).
- **Alternatives** — synchronous chains (cascading failures, tight coupling); publish-after-commit without outbox (lost or phantom events).
- **Consequences** — ✅ decoupling, resilience, natural audit feed and projections; ❌ eventual consistency to design for (sagas, reconciliation jobs).

ADR-09 — Self-hosted WebRTC SFU with end-to-end encryption

- **Decision** — LiveKit SFU and TURN servers on HDS-certified infrastructure; simulcast and adaptive bitrate; E2EE on by default for 1-to-1 consultations; no recording.
- **Alternatives** — CPaaS (fast to integrate, but media processed outside MedX's certified scope and costly at scale — kept as fallback); peer-to-peer mesh (no quality control, relaying issues); MCU (CPU-heavy, breaks E2EE).
- **Consequences** — ✅ data sovereignty, confidentiality, cost control; ❌ media operations skills, no server-side recording or transcription while E2EE is on.

ADR-10 — No health data outside the trust boundary

- **Decision** — SMS, e-mails, push banners, external calendars, event payloads, logs and traces never contain health data; details are shown only after authentication in the app.
- **Consequences** — ✅ removes whole classes of leaks and keeps third-party providers out of the HDS scope; ❌ less informative messages (mitigated by deep links).

ADR-11 — PSP-hosted payment fields (PCI DSS SAQ A)

- **Decision** — card data goes from the device to the PSP; MedX stores tokens only; 3-D Secure; own double-entry ledger for money movements.
- **Consequences** — ✅ minimal PCI scope and no card data to steal; ❌ payment UI constrained by the PSP components.

ADR-12 — Multi-AZ everywhere, warm-standby DR, active-active only for Tier 0

- **Decision** — every tier redundant across 3 AZs (RPO 0 in-region); a second EU region as warm standby (RPO ≤ 1 min); emergency access and sign-in active in both regions.
- **Alternatives** — full active-active (multi-master conflicts on clinical data, double cost); single region (unacceptable for Tier 0).
- **Consequences** — ✅ emergency access survives a regional outage, costs stay reasonable; ❌ Tier 1 and 2 recover in 1 h and 4 h respectively.

ADR-13 — Synthetic data by default outside production

- **Decision** — development, staging and demo use synthetic FHIR patients; realistic datasets only through the anonymisation pipeline, with DPO approval; production keys unusable outside production.
- **Consequences** — ✅ no real PHI in non-production, ever; ❌ some edge cases found later → targeted anonymised datasets and production-like load tests.

ADR-14 — Deliver by priority, buy what is not differentiating

- **Decision** — secure core first (High), care workflows second (Medium), ecosystem last (Low) — see roadmap. Low-priority capabilities rely on partners (PSP, messaging providers) and enter earlier only as thin slices required by Medium features.
- **Consequences** — ✅ the foundations every later feature depends on are right from day one; ❌ commercial launch waits for the secure core (a pilot with partner clinics runs during Phase 2).

7. Requirements Traceability Matrix

Proof of completeness: every priority of Appendix 2, every feature of Appendix 1 and every deliverable is mapped to the design element that satisfies it.

7.1 Priorities (Appendix 2)

ID	Requirement	Design response	Where
H1	Data privacy — encryption at rest and in transit	TLS 1.3 at the edge, mTLS in the mesh (STRICT), TLS to all stores, DTLS-SRTP + E2EE for media; AES-256 storage encryption everywhere + per-patient envelope encryption with HSM-backed KMS; no PHI outside the clinical zone	§5.5, §5.6, §1.5
H2	Patient records management — secure, integrity, compliance	Dedicated FHIR-based Records service, validation, signed notes and addenda, consent registry, care grants, full audit, retention and crypto-shredding	§2.4, §3.4
H3	Emergency access for healthcare professionals	Break-glass: entitlement + justification + step-up MFA → 4-hour read-only grant → IPS critical summary in < 300 ms; isolated Tier-0 deployment active in two regions; alerts, patient notice, 72-hour review	§2.4 Emergency access, §4.9
H4	Authentication & authorization	Keycloak (OIDC, MFA, passkeys, federation), realms per population, short-lived bound tokens, OPA RBAC + ABAC with local enforcement, JIT privileged access, quarterly recertification	§2.1, §5.4
H5	Data integrity — no loss/corruption, secure file storage	ACID + constraints, optimistic concurrency, append-only history, hash chains + signed anchors, outbox, checksums, Object Lock, immutable backups, scanned & encrypted Document Store, restore tests	§3.8, §2.9 Document Store
M1	Teleconsultation scheduling with notifications to doctors	Scheduling service: holds, saga, exclusion constraint, waitlist, urgent queue, real-time agenda updates + notifications	§2.2
M2	Reliable video/audio, chat, file sharing	Consultation control plane + SFU media plane, adaptive quality, E2EE, chat with receipts, Document Store uploads	§2.3
M3	Prescription management with notifications	Signed versions, safety checks, modify/revoke rules, critical revocation alerts, fulfilment tracking	§2.5
M4	Data masking and redaction	Policy obligations applied by a shared masking library; partial masking, allow-lists, category exclusion, true PDF redaction, PHI-free logs	§2.4 Masking, §5.7
M5	Integration with external calendars	Google, Microsoft 365, CalDAV adapters; OAuth minimal scopes; push + delta sync; free/busy only; no PHI written	§2.2 Calendars
L1	Pharmacy interface & inventory	Portal + partner API + signed webhooks, fulfilment state machine, inventory adapters, nearby availability	§2.6
L2	User preferences management	Preferences API in User Management, replicated to Notification by events	§2.1, §2.7
L3	Notification delivery and read tracking	Status machine, provider receipts, in-app read receipts, append-only log (Cassandra, 13 months), no e-mail pixels	§2.7
L4	Data anonymization for staging/demo	Synthetic data by default; anonymisation pipeline (identifier removal, keyed pseudonyms, generalisation, date shifting, k-anonymity), DPO approval	§5.7, §2.9
L5	Secure payment gateway	PSP hosted fields (SAQ A), 3-D Secure, tokens only, pre-authorisation + capture, ledger	§2.8
L6	Transaction refunds	Refund state machine, automatic rules, 4-eyes above threshold, credit notes, ledger entries	§2.8 Refunds

Coverage: 5/5 High, 5/5 Medium and 6/6 Low requirements are designed. Delivery order follows the priorities (roadmap).

7.2 Features of Appendix 1

Component	Feature	Mechanism	Where
① User Management	Registrations and profile updates	Patient self-registration with identity assurance; professional onboarding with registry check; pharmacy organisations	§2.1
	Authenticate patients, doctors, pharmacy staff	Keycloak realms, OIDC + PKCE, MFA, passkeys, e-CPS federation	§2.1
	Authorize by role	RBAC matrix + ABAC policies (OPA)	§2.1
	Preferences	Preferences API + `PreferencesUpdated` events	§2.1
② Scheduling	Book, reschedule, cancel	Holds, saga with Billing, cancellation policy	§2.2
	Notify doctors	Events → Notification + live agenda via Real-time Gateway	§2.2
	Doctors' calendars	Google / Microsoft / CalDAV adapters	§2.2
	Waitlists and urgent priority	Waitlist matcher, triage, urgent queue, reserved capacity	§2.2
③ Consultation	Virtual rooms	Room lifecycle, waiting room, scoped tokens	§2.3
	Video and audio streaming	WebRTC SFU + TURN, simulcast, audio-only fallback	§2.3
	Text chat	WSS chat, receipts, archive into the record	§2.3
	File transfers	Document Store pipeline	§2.3
④ Patient Records	Store histories, notes, records	FHIR resources + documents	§2.4
	Doctors add/edit post-consultation	Versioning, signature, addenda	§2.4
	Patients view (with restrictions)	Patient scope, release rules, redaction	§2.4
	Integrity & compliance	Hash chains, anchors, audit, consent, retention	§2.4
⑤ Prescription	Create, modify, revoke	Signed versions, safety checks	§2.5
	Notify pharmacies	Events → Pharmacy Interface, critical lane for revocations	§2.5
	Patients view active prescriptions	Patient API	§2.5
	Track fulfilment	Fulfilment state machine	§2.5
⑥ Pharmacy Interface	Confirm receipt, update statuses	Inbox, acknowledgement SLA, idempotent status updates	§2.6
	Patient–pharmacy interaction	Conversation threads + notifications	§2.6
	Inventory integration	Adapters + availability cache	§2.6
⑦ Notification	Alerts, reminders (SMS, e-mail, in-app)	Pipeline with channel workers and priority lanes	§2.7
	Preferences (opt-in/out, frequency)	Local replica, quiet hours, caps, digests	§2.7
	Delivery & read status	Status machine + log	§2.7
⑧ Billing & Payment	Fees and payments	Pre-authorisation + capture	§2.8

Component	Feature	Mechanism	Where
	Invoices (consultations, medication)	Gapless numbering, WORM PDFs, credit notes	§2.8
	Refunds	Refund workflow	§2.8
	Secure transactions	PSP hosted fields, 3-D Secure, signed webhooks	§2.8

7.3 Deliverables

#	Deliverable	Artefacts
1	High-level architecture diagram	High-level architecture · PNG / SVG · system context · deployment view
2	Detailed component designs	Component designs (8 services + platform components)
3	Data model and storage strategy	Data model & storage
4	Scalability and performance considerations	Scalability & performance
5	Security and compliance measures	Security & compliance · encryption diagram
6	Presentation	Slides and speaker notes

7.4 Evaluation criteria

Criterion (weight)	Where it is addressed
Clarity of communication (20 %)	One-page overviews at the top of each document, consistent numbering ①–⑧, H/M/L badges, diagrams for every key flow, presentation with speaker notes
Architectural soundness (20 %)	§1 style rationale, zones, communication rules, acyclic synchronous dependencies (§2.2 of the component index), ADRs
Scalability & performance (20 %)	§4: capacity math, latency budgets, per-service scaling, DB scaling path, video at scale, DR, degradation, cells
Priorities alignment (20 %)	§0.4, §6 decisions & trade-offs, §8 roadmap
Data handling strategy (10 %)	§3 and §5.6–5.7
Completeness (10 %)	This matrix: 16/16 priority requirements and 30/30 Appendix-1 features covered

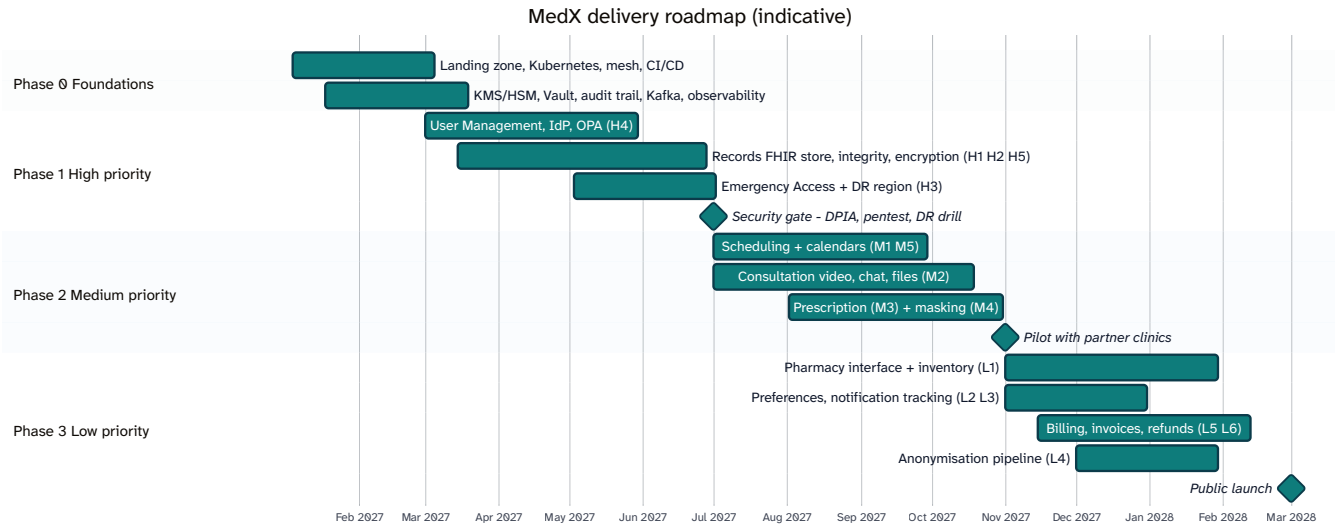
8. Roadmap & Risks

How the design is delivered **in the order of MedX's priorities**, and what could go wrong.

8.1 Delivery principles

1. **High-priority capabilities are foundations, not features**: encryption, identity, records, integrity and emergency access are built first and are **release gates** for everything else.
2. Medium-priority care workflows are built on top of that foundation.
3. Low-priority capabilities come last and rely on partners (payment provider, messaging providers). Thin, bought slices enter earlier **only** when a Medium feature needs them (e.g., a transactional e-mail for a booking confirmation).
4. Each phase ends with exit criteria that are **measured**, not declared.

8.2 Phased plan (indicative, kick-off January 2027)



Phase	Scope	Priorities	Exit criteria
0 — Foundations (≈ 2 months)	Accounts and guardrails, Kubernetes + mesh (mTLS STRICT), CI/CD with signing and scanning, KMS/HSM + Vault, event bus + outbox relay, audit trail (WORM), Document Store, observability, synthetic data generator	H1, H5 (enablers)	Every environment built from code; 100 % of stores encrypted; audit events immutable end-to-end
1 — Secure clinical core (≈ 4 months)	User Management + Keycloak + OPA; Patient Records (FHIR, versioning, hash chains, per-patient keys, consent, care grants); Emergency Access active in two regions; DR region; DPIA	**H1–H5**	Authorization test suite green; hash-chain verification green; break-glass p95 < 300 ms in both regions; DR drill passed; external pentest with no critical/high finding open
2 — Care workflows (≈ 4 months)	Scheduling (holds, saga, waitlist, urgent queue) + calendar connectors; Consultation (SFU, TURN, chat, file sharing, E2EE); Prescription (signing, checks, revoke); masking & redaction; thin transactional notifications	**M1–M5**	Load test at 2× Year-1 peak passed; video quality KPIs met on impaired networks; revocation reaches pharmacy portal < 5 s; **pilot with partner clinics**
3 — Ecosystem & business (≈ 4 months)	Pharmacy portal + partner API + inventory adapters; preferences UI; notification tracking; billing, invoices, refunds, ledger; anonymisation pipeline	**L1–L6**	PCI SAQ A attestation; reconciliation to the cent over 30 days; first PMS vendor integrated; **public launch**

Phase	Scope	Priorities	Exit criteria
4 — Scale & expand (continuous)	National services (e-prescription, INS, shared record), ISO 27001, sharding when triggered, regional cells, HIPAA-ready US cell	—	Triggered by growth and market decisions

Team topology — one platform team (infrastructure, security building blocks, observability) and five stream-aligned teams (identity & access · clinical records & emergency · scheduling & consultation · prescription & pharmacy · engagement & billing), plus shared web/mobile front-end capacity; a security champion in each team.

8.3 Risk register

#	Risk	Likelihood	Impact	Mitigation	Owner
R1	Regulatory scope changes (HDS scope of MedX's own activities, medical-device qualification of triage or checks)	Medium	High	Early legal and DPO review; DPIA; triage kept rule-based and medically validated; certification plan	DPO / legal
R2	Poor video experience on weak networks harms adoption	Medium	High	Audio-first design, TURN over 443, pre-call tests, quality telemetry per region, phone fallback	Consultation team
R3	Operational load of self-operated components (Keycloak, LiveKit, Kafka)	Medium	Medium	Managed services where they are inside the HDS scope, golden paths, runbooks, platform team on-call	Platform
R4	Break-glass misuse	Low	High	Entitlements, alerts, patient notice, mandatory reviews, sanctions, UEBA	Privacy officer
R5	Key-management error making data unrecoverable	Low	Very high	HSM-backed multi-region keys, no manual key deletion outside the crypto-shredding workflow, dual control, recovery drills	Security
R6	FHIR store performance at scale	Medium	Medium	Projections, partitioning from day one, sharding path, load tests at 2×	Records team
R7	Slow pharmacy adoption (integrations)	High	Medium	Portal for every pharmacy from day one, adapters per PMS vendor, partnerships	Product
R8	Third-party provider outage (PSP, SMS, calendars)	Medium	Medium	Two providers per messaging channel, graceful degradation, circuit breakers	Engagement team
R9	Vendor lock-in	Medium	Medium	Open standards (FHIR, OIDC, WebRTC), open-source components, Kubernetes, infrastructure as code	Architecture
R10	Scarce skills (FHIR, security, WebRTC)	Medium	Medium	Standard technologies, training, pairing, security champions	CTO
R11	Supply-chain attack on dependencies	Medium	High	SBOM, signed images, admission control, dependency pinning and scanning	Security

8.4 Open questions for MedX

1. Legal roles between MedX and practitioners (controller / processor) and resulting HDS certification scope.
2. Timeline and mandatory scope of national integrations (e-prescription, national identifier, shared record).
3. Pricing and reimbursement model (patient pays, insurers, third-party payment) — impacts Billing.
4. Target markets after France and the order of regional cells.

5. Appetite for AI-assisted features (triage, note-taking): would bring EU AI Act and MDR obligations and constrain end-to-end encryption choices.